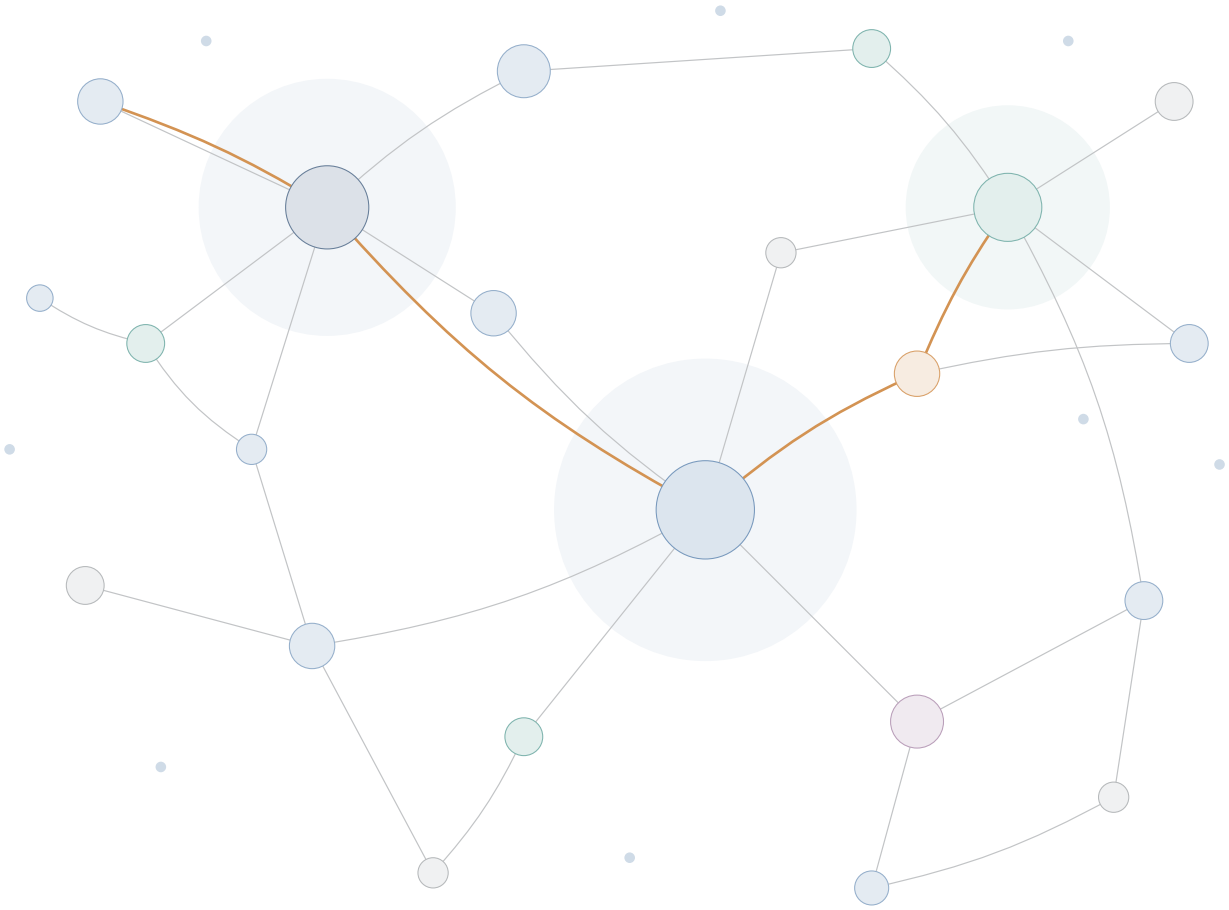
A textbook for students of archival science

Records in Contexts
An Introduction

Understanding the new international standard for archival description

English edition 1.0 2026



Tetsuji Kuboyama

Gakushuin University

About this book

This book is an introduction to Records in Contexts (RiC), the standard for archival description developed by the International Council on Archives (ICA), written for students of archival science and for practitioners who are meeting the standard for the first time. It is based on the following primary sources.

- *Records in Contexts—Foundations of Archival Description* (RiC-FAD) Version 1.0, ICA EGAD, November 2023
- *Records in Contexts—Conceptual Model* (RiC-CM) Version 1.0, ICA EGAD, November 2023
- *Records in Contexts—Ontology* (RiC-O) Version 1.1, ICA EGAD, May 2025 (including the OWL source file, the official documentation and the official examples)
- *Records in Contexts—Application Guidelines* (RiC-AG) Version 0.1 (draft), ICA EGAD, October 2025 (including the text, the official crosswalks and the diagrams)

The two documents at the centre of the standard, RiC-CM and RiC-O, are specifications. They say what the model contains, not how to work with it. Definitions are stated in the abstract, worked procedures are largely absent, and the reader who asks "so what do I actually do?" gets no answer. The fourth part, RiC-AG, is the part that takes that question seriously, and it appeared only in October 2025, as a first draft. Its chapter on implementation rules out, in its opening paragraph, the thing a beginner most wants: "We cannot provide a step-by-step guide to implemententing [sic] RiC—the huge variety of specific use-cases make this impossible." What it offers instead is "a list of actions that might be considered", which, it says, "should not be regarded as complete or definitive" (RiC-AG, *Implementation strategies*).

A distance therefore remains between the specifications and everyday practice. This book tries to close it. It follows all four parts, keeps close to what the sources actually say, and adds what the sources leave out: where the ideas come from, how they relate to techniques developed in neighbouring fields, and what a first implementation looks like.

A note on the examples The book was first written in Japanese, for readers in Japan, and this English edition is a translation. Where the original used Japanese records, institutions and legislation to make a point, the examples have been replaced rather than translated, so that they can be followed without knowledge of Japanese archival practice. What has not been changed is the vantage point. The author works in a country whose archival tradition was not represented in the group that drafted RiC, and Section 1.4 says so plainly. Readers inside the traditions that were represented may find that observation useful.

How the book is arranged Chapter 1 gives the shape of the standard and the structure of the primary sources. Chapter 2 looks at the standards RiC replaces, and at the history of how one description has been made to refer to another. Chapter 3 sets out what is new in RiC. Chapter 4 reads RiC-CM, the conceptual model; Chapter 5 reads RiC-O, the ontology. Chapter 6 is about inference, which is where the advantage of RiC-O shows most clearly. Chapter 7 is about doing

the work, and Chapter 8 about how much technology this requires. The appendices hold reference tables, hints for the exercises, and a glossary of terms that shift meaning between fields.

How to read it The chapters are written to be read in order, and reading them in order works. Not everyone needs to read every chapter at the same depth, though. Figure 1 gives a rough guide for each of the four roles set out in Section 8.1. Chapters 5 and 6 contain long stretches of Turtle, which can look forbidding; an archivist who writes descriptions needs to be able to *read* it, not to write it. Chapter 7 is close to self-contained, section by section, so it can be entered at whichever section is of interest. The reference tables in the appendices are not for reading through. They are for opening while designing a description.

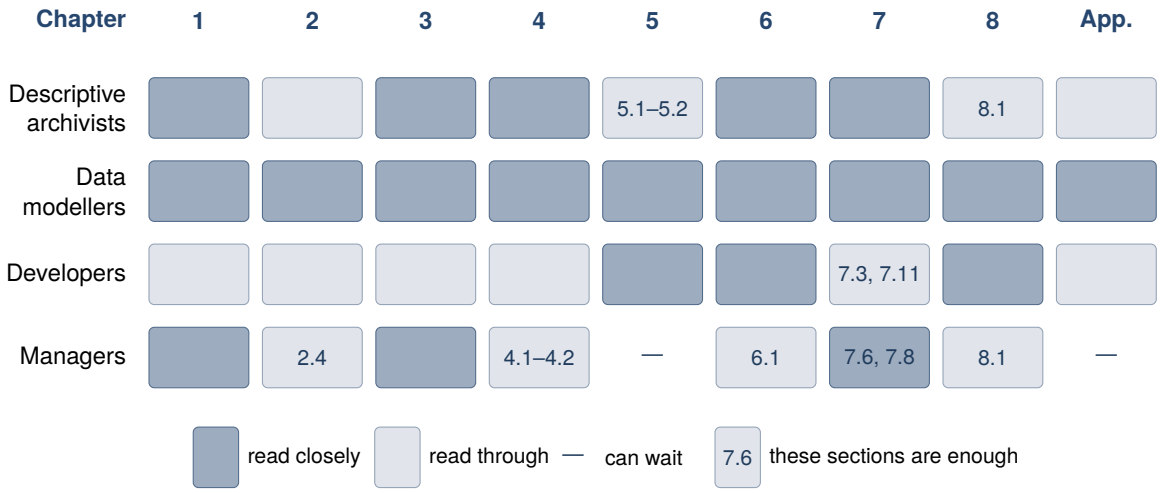


Figure 1: A guide to reading by role. The four roles are the ones tabulated in Section 8.1. Shading shows how closely to read; a section number inside a cell means that those sections alone will do.

Conventions Terms from the RiC-O vocabulary appear in a monospaced face with their prefix, as in `rico:Record`. References to the primary sources are given by section number, as in "(RiC-CM §1.6)". Read with the sources open. This book is a map; the territory is on their side.

Contents

Chapter 1

What RiC Is — The Standard and Its Sources

1.1 The shape of the standard

Records in Contexts (RiC) is the international standard for archival description developed by the Expert Group on Archival Description (EGAD) of the International Council on Archives (ICA). As the name says, its subject is records *in contexts*: not the record on its own, but the record together with the people who made and used it and the activities those people pursued. It is a single standard, and it replaces four earlier ones: ISAD(G), ISAAR(CPF), ISDF and ISDIAH (RiC-CM §1.2).

The difference that matters most is stated by RiC-CM itself in a single sentence: "The existing ICA standards model description, that is, they model a finding aid, whereas RiC-CM models the entities as such, as a basis for describing but without anticipating any particular end product" (§1.2). The older standards told you how to compose a description. RiC describes the world that a description is about. A hierarchical, fonds-level description can still be expressed, but so can the whole network of relations that surrounds a record. Chapter 3 takes up what that shift costs and what it buys.

1.2 Four parts

RiC is not one document. It has four parts, and they are documents of different kinds (RiC-CM §1.1). Knowing which is which is the first thing that helps when reading the sources.

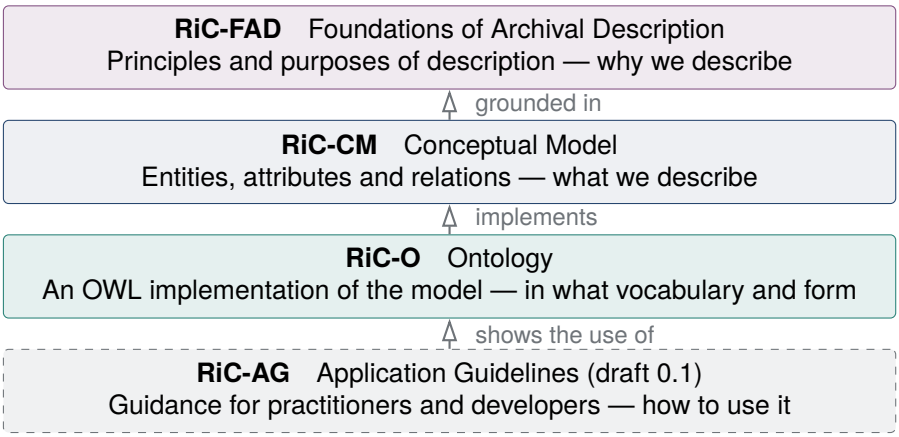


Figure 1.1: The four parts of RiC, running from the abstract at the top to the practical at the bottom.

1.2.1 RiC-FAD — the principles

RiC-FAD (Foundations of Archival Description) is short, about eight pages, and states the principles and purposes that underlie archival description. It covers the history of record keeping (going back to the clay tokens of Mesopotamia), the emergence of the principle of provenance and its later widening, the role of the archivist, and the three purposes of description: managing records, preserving them, and making them usable. It is the ground on which RiC-CM justifies its choices, and the CM refers back to it repeatedly. It began as part of RiC-CM 0.1 and was later separated out, which is why the two are meant to be read together.

1.2.2 RiC-CM — the conceptual model

RiC-CM (Conceptual Model) is the central document, around 130 pages. Using the framework of the entity-relationship model, it defines what description is about in terms of entities, attributes and relations. Version 1.0 defines 19 entities, 42 attributes and 85 relations, most of the relations coming in pairs pointing in opposite directions.

RiC-CM calls itself "a high-level conceptual model" (§1.2). The qualifier is not modesty. The document then lists what the model is *not*: four things it is none of, "though it may inform the development of each":

- a standard or set of rules for composing or forming descriptive content;
- an implementation specification for developing records management and public access systems;
- a model for physically managing records, "though it does provide a framework for the intellectual component of such management";
- a data communication or exchange standard.

So RiC-CM does not do the work that EAD does (Encoded Archival Description, the standard for encoding a finding aid so that it can be exchanged between systems), nor the work that a content standard such as DACS or RAD does. It settles how the world is to be carved into concepts, and leaves the expression and the operation of it to whoever implements. That is the source both of the freedom discussed later in this book and of the shortage of guidance that practitioners complain about. What exactly is missing when you try to begin describing from this standard alone is shown in Section 7.1.

1.2.3 RiC-O — the ontology

RiC-O (Ontology) formalises RiC-CM in OWL 2, the web ontology language standardised by the W3C. RiC-CM calls it "a specific implementation of RiC-CM" (§1.1), and RiC-O itself speaks of "a technical implementation of RiC-CM that also extends and refines it". If RiC-CM is a diagram on paper, RiC-O is a vocabulary a machine can read: the means to publish archival description as Linked Open Data (data published on the web with links between datasets; Chapter 5), to query it with SPARQL (Chapter 6), and to reason over it. Version 1.1 defines 107 classes, 75 datatype properties and 480 object properties, so it is a good deal more detailed than the high-level CM. It also contains constructs that have no counterpart in the CM at all, such as the Proxy class. Chapter 5 takes it up in detail.

RiC-O is not the only possible implementation, and the official documentation says so: "other technical implementations of RiC-CM may be developed later on". It makes a separate point about the older standards, that the existing XML schemas of EAD and EAC-CPF "should be made closer to RiC-CM in the future". The relation between CM and O is that of a concept to one realisation of it.

1.2.4 RiC-AG — the application guidelines (draft)

RiC-AG (Application Guidelines) is the fourth part, addressed to practitioners, developers and managers. The first draft, 0.1, was released on 29 October 2025 at the ICA congress in Barcelona. EGAD calls it a living document and intends to develop it in response to community feedback.

Its contents fall into two groups. One is about RiC as a whole: the benefits of RiC for users, for practitioners and for managers; strategies for implementation; using RiC alongside other standards; and the official crosswalks, both from the four existing standards to RiC-CM and from EAD 2002 to RiC-O 1.1. The other is practical guidance: a worked conversion of an existing ISAD(G)-style catalogue (*Getting started*, using a catalogue from The National Archives of the UK), a set of FAQs (record or record set, instantiation, order, describing activities, using vocabularies, how to extend), a case study on Indigenous communities, and an extension example built around Cervantes and the publication of *Don Quixote*.

What to expect of it is set by its own opening. The part on implementation strategies states in its opening paragraph: "We cannot provide a step-by-step guide to implemententing [sic] RiC—the huge variety of specific use-cases make this impossible." In its place comes "a list of actions that might be considered", which "should not be regarded as complete or definitive". The same posture holds throughout: examples and diagrams where rules might have been. With the fourth part published, RiC still is not a standard of the follow-these-steps kind. It supports the judgement; it does not make it. Chapters 3 and 7 return to what that design choice means in practice.

What a specification will, and will not, give you

Many readers finish RiC-CM for the first time thinking: I still do not know what to write, or where. That reaction follows from what the document is. RiC-CM is a set of definitions, a way of carving up the world, and nothing in it is a form. ISAD(G) laid out 26 elements of description with rules for applying them, so it could be used directly as a list of things to fill in. RiC-CM has no equivalent. The AG draft of 2025 exists to close that gap, and it does so by showing how to judge rather than what to enter: on the question of record versus record set it states that "there is no definitive right or wrong approach". In practice this means reading the parts for different purposes. For procedure, go to the AG and to Chapter 7 of this book; for the meaning of a term, go to the CM.

1.3 Where to find the primary sources

All of the RiC sources are free and carry a Creative Commons Attribution 4.0 licence. Development and publication both happen on GitHub, so alongside the specifications you get the version history, the community comments filed as issues, and the official sample data, all in one place.

The RiC-O repository repays a closer look, because it is the most useful of the four as a working source.

- `ontology/current-version/RiC-O_1-1.rdf` — the ontology itself, an OWL file in RDF/XML. Every class and property carries labels in English, French and Spanish, with German labels added to the classes in 1.1, and, where a counterpart exists in the CM, a note naming it (`rico:RiCCMCorrespondingComponent`). The definitions themselves are in English only. In effect this file is the detailed reference manual. RDF/XML, incidentally, is one of the ways of writing RDF in XML syntax; RDF itself is taken up from Section 2.3.
- `CSV_lists_of_components/` — the classes and properties as CSV tables. Useful for surveying the whole vocabulary without an ontology editor, and approachable without any RDF or OWL.
- `examples/` — real data. Actual EAD, EAC-CPF (the encoding standard for authority records; Section 2.1) and METS files (METS gathers the structure and the various metadata of a digital

Table 1.1: Where the RiC sources live (as of July 2026)

Document	Latest version	Where to find it, and what is there
RiC-FAD	1.0 (November 2023)	GitHub `ICA-EGAD/RiC-FAD`. The PDF, plus earlier versions (RiC-IAD 0.2 and others)
RiC-CM	1.0 (November 2023)	GitHub `ICA-EGAD/RiC-CM` and the ICA website. The PDF, earlier versions (0.1, 0.2, 1.0-beta), and the record of responses to public comment
RiC-O	1.1 (May 2025)	GitHub `ICA-EGAD/RiC-O`. The OWL source (`RiC-O_1-1.rdf`), HTML documentation, CSV lists of classes and properties, official examples (Archives nationales de France and others), and a set of diagrams
RiC-AG	0.1 draft (October 2025)	GitHub `ICA-EGAD/RiC-AG` and the web edition. The text (Markdown and HTML), the official crosswalks (xlsx), and diagrams (draw.io)
Related	—	The RiC-O documentation site (`ica-egad.github.io/RiC-O`), a list of projects (`RiC-ResourceList`), and the users' mailing list (Records in Contexts users, Google Group)

object into one XML document) are paired with their RiC-O equivalents in RDF. There is a digital preservation package from docuteam in Switzerland (used with PREMIS), conversions of catalogues and authority records from the Archives nationales de France, and an example from the University of Strathclyde Archives. Nothing teaches you faster what the triples actually look like. EGAD does caution in the accompanying notes that the examples are neither current nor representative.

- `diagrams/` — conceptual diagrams, in draw.io format and as PNG.

1.4 How RiC was developed

Following the development history is not antiquarianism: it tells you which version answers to which. RiC-O 0.2 implements RiC-CM 0.2; RiC-O 1.1 implements RiC-CM 1.0. When an article or a tool on the web assumes a particular version, this correspondence is how you work out which one.

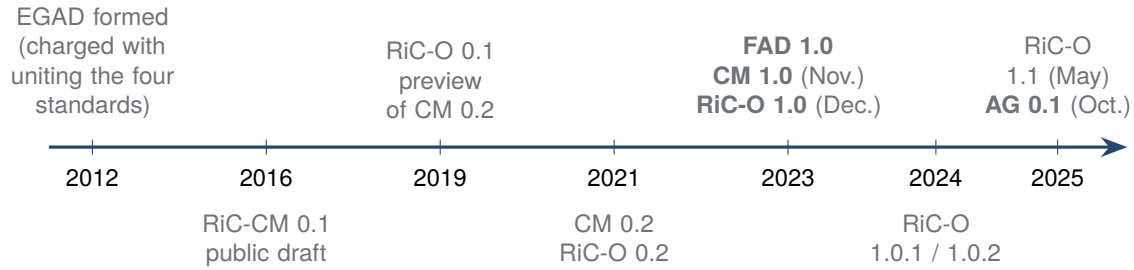


Figure 1.2: The development of RiC. The 2023 release of 1.0 is the first stable and complete version.

The main points are these (from RiC-CM §1.7 and the RiC-O history notes).

- In 2012 the ICA established EGAD and charged it with developing a single standard that would reconcile, integrate and build upon the four existing ones. Over the course of the work EGAD included 31 members from 15 countries.
- The first draft, RiC-CM 0.1, appeared in 2016 and drew comments from 62 individuals, groups

and institutions, representing 19 countries and two international organisations. The structure of the model was substantially reorganised in response.

- RiC-O 0.1 was published in December 2019. RiC-O was tried in the field early, notably in the PIAAF pilot at the Archives nationales de France.
- FAD 1.0, CM 1.0 and RiC-O 1.0 were published together in November and December 2023, giving the first stable and complete version.
- RiC-O has continued to be revised; 1.1, of May 2025, is current and conforms to CM 1.0. Its vocabulary has grown: 107 classes and 480 object properties, against 423 object properties at 0.2.
- In October 2025 the first draft of RiC-AG appeared at the ICA congress in Barcelona, and all four parts were in the world, one of them still in draft.

EGAD is candid about the limits of who took part. RiC-CM §1.7 records that while the members were "broadly representative of the global archival community, many areas with long and distinguished histories of administration and governance, and concomitant traditions of record creation, use, and management are not represented, such as much of Asia and eastern Europe"; Africa and South America were represented, but the representation "should be broader and more inclusive". EGAD invites wider participation on that ground.

The invitation should be taken literally, and it cuts both ways. Whether the model fits a tradition that was not at the table is not a question the model can settle on its own; it has to be tested by the communities concerned, and the results fed back. This book is written from one of those traditions. Where a claim in RiC seems to depend on assumptions that are local rather than universal, this book says so rather than passing over it.

1.5 Who the standard is for

RiC-CM §1.3 lists its audiences: archivists first, then records managers, then the neighbouring cultural heritage communities of libraries and museums, then system developers, then researchers. One line addressed to developers tells a beginner something important. RiC-CM concedes that the model is "detailed and complex", and continues: "The developers of systems, however, can ameliorate the intellectual, technological, and economic challenge of data creation and maintenance by designing and implementing systems with user interfaces that mask the complexity."

EGAD, in other words, never expected practitioners to wrestle with RiC in the raw. The standard is designed on the assumption that tools will stand between the model and the person describing. Learning RiC is therefore not learning to hand-write RDF. It is learning what the concepts mean, so that you can tell whether the tool in front of you is doing the right thing with them. Chapter 8 returns to this.

Exercises (hints in Appendix A.3)

1.1 Explain the role of each of the four parts of RiC (FAD, CM, O, AG) in one sentence each.

1.2 RiC-CM stands at 1.0 while RiC-O has moved to 1.1. Explain why a conceptual model and an ontology are versioned separately, from the difference in what each of them does.

1.3 Choose one purpose of your own (understanding the concepts, implementing a system, or guiding day-to-day practice) and work out from this chapter which parts to read, and in what order.

Chapter 2

From the World of ISAD(G) to RiC

The quickest way into RiC is through the standards it sets out to replace. This chapter goes over the four existing ICA standards, the view of the world they rested on, and where that view ran out, following the analysis RiC-CM makes of itself and its predecessors (§1.5–1.6).

2.1 The four ICA description standards

Table 2.1: The four existing ICA standards, and what RiC does with them

Standard	1st / 2nd ed.	What it describes
ISAD(G)	1994 / 2000	Records: the fonds and its components. 26 elements of description and the rules of multilevel description
ISAAR(CPF)	1996 / 2004	Authority records for creators: corporate bodies, persons, families
ISDF	2007	Functions: the activities and business of an organisation
ISDIAH	2008	Institutions with archival holdings

ISAD(G) (General International Standard Archival Description), first published in 1994, is the most widely adopted of the four. It formalised multilevel description working down from the fonds, and it has underpinned cataloguing practice, description systems and the encoding standard EAD ever since.

The other three came out of a plan to *separate* the main components of a description. ISAAR (CPF) made it possible to maintain the description of a creator apart from the description of the records (EAC-CPF is its encoding standard); ISDF took functions as an object of description in their own right, and ISDIAH the holding institution. The point of separating them was to let the same information about a creator serve the descriptions of several fonds, to make description more economical and more consistent, and to open up ways into the records that a fonds-based catalogue could not offer.

RiC-CM §1.6.1 does not soften its verdict on how that worked out. The four standards were developed independently of one another over an extended period, “without an overarching and persistent vision for how such separation would work in practice, for how the different components addressed would be related to one another to form a whole description”. The result is that they “do not represent a coherent, consistent model of archival description”. Uptake bears this out: ISAD(G) has been widely influential, ISAAR(CPF) “has some use”, and ISDF and ISDIAH “very little”. What the ICA asked EGAD to do was to design the whole thing again, from the start, as a single multi-entity model.

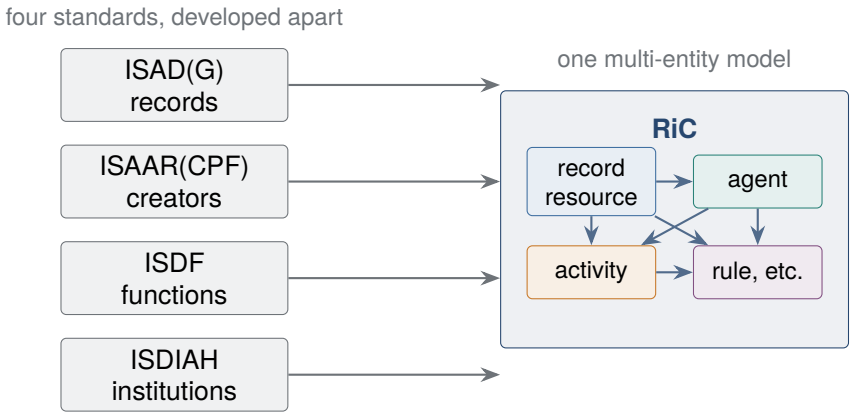


Figure 2.1: From four standards to RiC. What had been separated into four documents becomes entities in one model, joined by relations.

2.2 The assumptions behind traditional description

By the account in RiC-CM §1.5–1.6, description in the ISAD(G) manner rests on four assumptions.

(1) The traditional understanding of provenance Description proceeds from the Principle of Provenance as traditionally understood, which has two facets. *Respect des fonds* holds that the records created, accumulated and used by a person or group in the course of life and work are to be kept together and not intermixed with records from other sources. *Respect for original order* holds that the interrelations among the records established in the context of accumulation and use are to be preserved.

The two do different jobs. *Respect des fonds* fixes the *outer boundary* of a body of records, how far the one grouping extends. Respect for original order governs the *arrangement inside* it. What matters for later chapters is that the boundary is not drawn by the person describing: it is given in advance by the existence of the body that accumulated the records. That is why the RiC design in which one record can belong to several record sets (Chapter 3) is such a large change.

The two facets were not always one principle. The *respect des fonds* of nineteenth-century France said nothing about internal arrangement; internal order was treated separately, as *respect de l'ordre intérieur*. What joined them was the Prussian *Registraturprinzip* and, in 1898, the **Dutch Manual**[1]. The familiar statement that the principle of provenance has two aspects is the outcome of that joining. RiC reaffirms both facets as methodologically sound (RiC-CM §1.5.3). What it objects to is narrower: the traditional understanding cuts provenance down to a single body that accumulated a single fonds (Chapter 3).

(2) Self-contained hierarchical description of a single fonds Description begins with the fonds, then its components, then their components. RiC-CM calls it "largely self-contained, inward-looking" (§1.5.1): one fonds at a time, with almost no relations reaching outside.

(3) Treating input and output as the same thing What goes into a description system and what is presented to a reader have tacitly been assumed to be one and the same. ISAD(G) formally denies this (§I.6), but its linear sequence of elements follows the paper finding aid, and implementations have tended to produce screens that look like printed pages.

[1] *Handleiding voor het ordenen en beschrijven van archieven* by the Dutch archivists Muller, Feith and Fruin, translated as *Manual for the Arrangement and Description of Archives*. Written for the Netherlands Association of Archivists, it is generally taken as the first systematic codification of modern archival theory.

(4) Access points standing in for entities People, corporate bodies, places and subjects appear inside the description of records as "access points": names, and nothing more. RiC-CM makes a sharp observation about this. By recognising access points at all, ISAD(G) "implicitly recognized that records could only be understood in the context of their creation and use and as part of a wider network of relations with other entities" (§1.6.2). The insight was there; the machinery to act on it was not. ISAAR(CPF) and the standards after it were the response, and they did not get as far as an integrated model.

How RiC assesses ISAD(G)

RiC-CM thinks well of ISAD(G) and says so. "There is nothing inherently wrong with archival description that conforms with ISAD(G)" (§1.6.2). It expects fonds-down hierarchical description to remain the prevailing approach for the near future, and gives four reasons: it addresses the traditional understanding of the Principle of Provenance; it is well understood by the community; methods and systems already exist to create, maintain and publish it; and it is "a relatively economic approach to an exceptionally complex, labour-intensive challenge" (§1.6.1).

What RiC questions is the ceiling on what ISAD(G) can express. ISAD(G) "is limited in terms of what is possible using current and emerging communication technologies", and limited too in the face of what archivists and users have come to expect of those technologies. RiC-CM grants that it is itself far more complex than ISAD(G), and answers the objection head-on: "the world in which records are created and used is complex, and it is a fundamental responsibility of archivists to reflect that complex world to the best of their abilities" (§1.6.2). Whether the added complexity is worth it is a question this book returns to more than once.

2.3 Description and communication technologies

RiC-CM §1.5.2 makes a point of how far the form of archival description has depended on the communication technology available at the time. The argument deserves a close reading, because Chapters 5 and 8 build on it.

Two technologies moved archival description from paper to the computer in the last two decades of the twentieth century. One is markup: putting tags inside a document to show its structure, of which XML is the general case and EAD the archival application. The other is the relational database, which manages data as tables and is queried with SQL. What makes the RiC-CM account interesting is that it traces the failure of either technology to take over to the nature of fonds description itself. A fonds description mixes document-like parts (a narrative administrative history) with table-like parts (controlled element values), which makes it, in the phrase RiC-CM uses, "betwixt and between". It fits a tree of tagged text only partly, and a set of tables only partly. So implementations have used one or the other, or a carefully built combination.

XML turns up throughout this book, so its rules are collected here in one place.

XML: enough of the syntax to read it

XML (Extensible Markup Language) is a W3C standard for marking structure inside a document. It settles only how marks are made; which tags exist is decided by each application. EAD decides on tags for archival description, RDF/XML on tags for RDF.

Elements. The unit of structure is an **element**, whose content sits between a start tag `<unittitle>` and an end tag `</unittitle>`. An empty element can be written as one tag: `<dao/>`.

Attributes. A subsidiary value carried by an element is an **attribute**, written inside the start

tag as `name="value"`. The value is always quoted.
Nesting. Elements go inside elements. Start and end tags may not cross, so `<a><b></a></b>` is an error. Because of that rule the structure of an XML document is always a tree, with exactly one outermost element (the root).

```
<c level="series" id="ser01">
  <did>
    <unitid>P-83/1</unitid>
    <unittitle>General administration</unittitle>
    <unitdate normal="1948/1994">1948 to the 1994 reorganisation</unitdate>
  </did>
  <c level="file" id="fil12">
    <did><unittitle>Immunisation programme</unittitle></did>
  </c>
</c>
```

This fragment is EAD. A file-level `<c>` sits inside a series-level `<c>`, and the nesting is the hierarchy: nothing else states it. Note also the `normal` attribute, which carries the machine-readable form of a date whose displayed form is a phrase.
Whitespace and line breaks. Indentation outside tags is normally there for legibility and means nothing to the application, but the XML specification does preserve it as part of element content, so what counts as ignorable whitespace is settled application by application. Turtle (Chapter 5) is cleaner on this point.
Well-formed and valid. A document that obeys the syntax (tags nested properly, quotation marks closed) is **well-formed**. A document that also conforms to a schema saying which tags may appear where (for EAD, a DTD or an XML Schema) is **valid**. A document that is not well-formed cannot be parsed at all; one that is not valid fails what the application requires of it.
Other conventions. A declaration of the version and character encoding goes at the top of the document.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- a comment: not read as data -->
```

Comments run from `<!-` to `-->`. Since `<` and `&` are syntax, content that needs them is written `&lt;` and `&amp;`. Element and attribute names are case-sensitive.

Against these two, RiC-CM sets a third technology: graphs. Graph technologies, of which the W3C's RDF (Resource Description Framework) is the standard case, represent data as a web of nodes (things) and arcs (relations between them), and allow queries that follow the arcs.

The word **graph** here does not mean a bar chart or a line chart. It means a graph in the mathematical sense: *points, and lines joining the points*. The points are **nodes** or vertices, the lines **arcs** or edges. A transit map is a graph of stations and track; a family tree is a graph of people and descent. Applied to archival description, the points are records, people and activities, and the lines are the relations among them. The right-hand side of Figure 2.2 shows what that looks like. A tree is a special case of a graph, one in which each node has at most one line running to a parent, which is why hierarchical description fits inside a graph without any adjustment.

One point needs stating precisely, because it is often put too strongly. It is not that XML cannot express a graph. What the *syntax* of XML gives directly is a tree of nested tags; a graph can be laid on top of it by giving elements identifiers and having them refer to one another. RDF's own XML form (Chapter 5) does exactly that, and so does GraphML. But in that arrangement the graph lives in the convention rather than in the syntax, and only the tree part gets any help from the nesting. The asymmetry shows in EAD: the hierarchy is written naturally as nesting, while every

other relation is an identifier recorded in an attribute (Section 7.4). RDF is different in that *the data model itself is a graph*: every relation is written as a **triple** with equal standing. A triple is a sentence of three words, subject, predicate and object, and one triple is one arc (Section 5.1).

The reasoning behind RiC's choice of technology is stated in one sentence of RiC-CM: "given that the real world within which we live and work may be understood as a vast, dynamically interrelated network of people and objects situated in space and time, graph technologies offer new and more expressive forms of representation" (§1.5.2). RiC-CM supplies the conceptual ground for description as a graph; RiC-O implements it as an RDF graph.

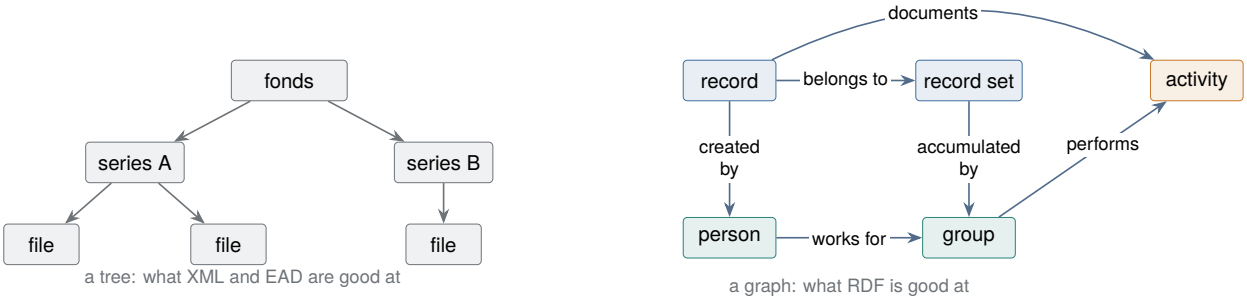


Figure 2.2: From tree to graph. A hierarchy can express one kind of relation, parent to child; in a graph the kinds and the directions can be multiplied freely. A tree is a special case of a graph, so hierarchical description fits inside one unchanged.

2.4 Reference and identifiers

What RiC-O data aims at is Linked Data: data that is joined up. That joining is **reference**, one description naming something else, and it underlies everything in this book. In daily practice reference shows up in unglamorous forms, a shelfmark or an authority number, and gets taken for administrative housekeeping. It deserves better. This section builds up from what reference is for, what it requires, and why it has a direction, and ends with a history of the means of reference. With that in hand, Chapter 5 and the conversion work in Chapter 7 become much easier to follow.

2.4.1 What reference is for

A description is a collection of fragments, each cutting out one piece of the world: the fonds, a series within it, the creator. Fragments lying side by side are of little use. Reference is what joins them.

The first thing reference buys is that *nothing has to be written twice*. Suppose a county council is the creator of thirty fonds. Without reference, the administrative history of the council is copied into all thirty descriptions, and a reorganisation means correcting it in thirty places (and missing some). With reference, the council is described once and the thirty descriptions *point* at it. The fact is recorded in one place and corrected in one place. This is what the principle called **normalisation** in database design is protecting.

What normalisation actually does

Normalisation means splitting one table that holds several kinds of thing into a table per kind. Take a catalogue kept as a single table.

Item ID	Title	Creator	Est.	Abol.	Parent body
S-1	Sanitary inspection reports	Public Health Dept	1948	1994	County Council
S-2	Vaccination registers	Public Health Dept	1948	1994	County Council
S-3	Port quarantine files	Public Health Dept	1948	1994	County Council

The three columns about the creator repeat once per item, and that causes three kinds of trouble. Correcting a wrong date of establishment means correcting as many rows as there are items (an *update anomaly*); miss one and the same body ends up with two dates. A body from which nothing has yet been accessioned cannot be recorded at all, because the only place to put information about a creator is a row about an item (an *insertion anomaly*). Destroy all of a body's records and delete the rows, and the body itself disappears from the catalogue (a *deletion anomaly*).
The cause is plain: facts about two different kinds of thing, items and creators, are sharing one table. So split the table, and have the item side point by identifier.

Items

Item ID	Title	Creator ID
S-1	Sanitary inspection reports	P-01
S-2	Vaccination registers	P-01
S-3	Port quarantine files	P-01

Creators

Creator ID	Name	Est.	Abol.	Parent body
P-01	Public Health Dept	1948	1994	County Council

Now the date of establishment appears once, so there is one place to correct. A body with no records yet can be entered in the creators table. Deleting every item leaves the body standing. All three anomalies go at once.
This handles only the case of one creator per item, though. As soon as the relation is many-to-many, as with joint creation, a single creator column will not take it. The answer is to *make the relation itself a table*.

Item–creator

Item ID	Creator ID	Role
S-1	P-01	drafted
S-1	P-02	approved
S-3	P-01	drafted

This third table is a **junction table**. One row says which item and which creator stand in which relation. Look at the role column: it is an attribute neither of the item nor of the creator, but *of the relation between them*. It can be written only because the relation has been given a row of its own.
Those three steps are the path this book is about to take. Splitting the creator into its own table is what ISAAR(CPF) did with authority records (Section 2.1); giving the relation a table of its own is what RiC does when it makes relations into entities (Section 4.4, and Chapter 5). Anyone who has designed a relational schema can read the entities, attributes and relations of RiC-CM as a continuation of it (Section 8.3).

The second thing reference buys goes deeper: *a finding aid is itself a mechanism of reference to the material*. A description does its job when the reader, or the system, can get from it to the thing described, and the means of getting there is the reference code. Archival description works as a web of references: description to holdings, description to description (up, down, sideways), description to authority record. Reference is what the description is for, not housekeeping around it.

2.4.2 Reference needs identifiers

For a reference to work, what it points at needs a name that picks it out without fail. That is an **identifier**. A name as a string of characters cannot serve: "Margaret Ellis" is not unique (other people have the name), not stable in form (spellings and initials vary), and not permanent (names change). Invert those three failures and you have the conditions a usable identifier has to meet.

1. **Unique.** Within the range in which it circulates, one identifier picks out one thing.
2. **Permanent.** Once assigned, it is not changed. Change it and every reference to it breaks.
3. **Free of meaning.** An identifier with meaning built into it (an organisation's name, a classification) invites you to change it when the facts change, which collides with the second condition. A number such as `PUBHEALTH-1948-005` becomes a lie the moment the department is reorganised.

Seen from here, a familiar feature of ISAD(G) reads differently. Of the 26 elements, ISAD(G) names six as essential for international exchange, and *first* on that list is the reference code (3.1.1). Before the title, before the dates. It even specifies the structure: country code (ISO 3166), repository code, and a local number, so that uniqueness survives outside the institution. There is a reason for that placement. *Having an identifier, being able to be the target of a reference, is the condition of a unit of description being a unit of description at all*, and the reference code is how the condition is met. Multilevel description works only if the levels can point at one another; exchange between institutions and citation by users both assume identifiers.

2.4.3 Reference has a direction

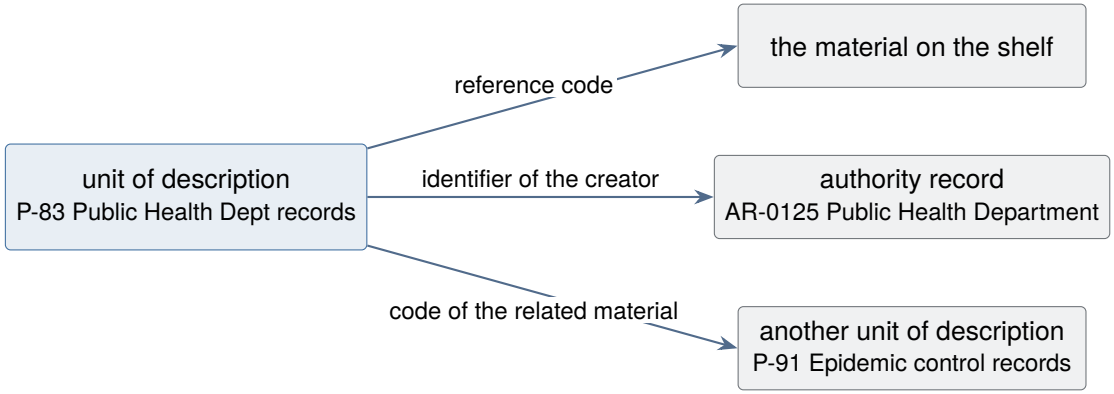
Reference is asymmetric: A points at B. There is a pointing side and a pointed-at side, and the record of the reference sits on the pointing side. Figure 2.3 shows the three kinds of reference leaving a single unit of description.

Which side to put the record on has a rule of thumb in the world of documents: *the many point at the one*. Thirty fonds descriptions point at one authority record, rather than the authority record listing thirty fonds. Put it on the pointing side and each description carries one line, and adding a fonds leaves the authority record untouched.

The rule exacts a price that cannot be avoided. *The reverse cannot be followed.* "List every fonds that points at this authority record" is a natural question for a reader to ask (where else are this person's records?), and a pile of documents cannot answer it directly. Either every document is read through, or a reverse index is built and maintained alongside. Cross-references on paper ("see also") and indexes in a database are ways of paying that price with labour. Making references followable in both directions, automatically, is what graph technology (Section 2.3) and inference (the inverse properties of Chapter 6) exist to do. Fix the rule and its price in mind here, and much of what comes later is recognisable as the same problem.

2.4.4 A history of reference: from names to IRIs

With those pieces in place, the history of archival description can be read again as a history of strengthening reference.



All three point outwards from the description. The reverse list (which descriptions point at this authority record?) cannot be had from documents without a separate index

Figure 2.3: Three kinds of reference leaving one description. A finding aid works as a web of references to holdings, to authority records and to other descriptions. The record of a reference sits on the pointing side, and so has a direction.

First period: paper finding aids. Names as strings, resolution by a person Reference by identifier was already there on paper: the shelfmark, pointing from the description to the material. What stayed a string of characters was the *mention of a person, a body or a subject*. Libraries developed **name authority control** to gather the variant forms of one name and separate the identical names of two people, and cross-references ("see") were built up alongside. All of it addressed a *human* reader. Actually getting from the name to the thing named, what may be called the **resolution** of the reference, was work done by a person in the search room.

Second period: ISAD(G) and access points. Controlled strings ISAD(G) asked that people, bodies, places and subjects be embedded in the description as **access points**, in as controlled a form as possible. As an entry to searching this is progress, but the substance of the reference is still a string, and the person or body named is not itself described. Homonyms, name changes and reorganisations all defeat it structurally. To be fair to ISAD(G), it did not neglect identifiers. As shown above, the reference code heads the list of essential elements, and where the target was a *record*, an identifier could be used (element 3.5.3, related units of description, takes a reference code). The string was the fallback where the target was a *person, body or subject*. Identifiers for records, names for actors: that asymmetry is what the period amounts to, and ISAAR(CPF) solved the second half of it.

Third period: ISAAR(CPF) and authority records. From strings to record identifiers ISAAR (CPF) took the description of the creator out into an **authority record** of its own, to be pointed at from the description of the records by *identifier*. Reference was promoted from a name to a record ID, which is a turning point in the history of description. The range in which the ID circulated stayed institutional, though, or at best national, and the procedure for getting from an ID to the record it names was left to whatever each system did internally.

Fourth period: EAD and EAC-CPF. References a machine can read The encoding standards turned references into attributes a machine could read: `authfilenumber` in EAD, the linking attributes on the relation elements of EAC-CPF. Three limits remained. Whether an ID or a URL still resolves in ten years depends on how it is operated. The *meaning* of a reference stays coarse,

carried by the name of the element. And because each reference is a statement inside one document, nothing keeps the two directions consistent. Section 7.4 takes this third point up with a diagram.

Fifth period: RiC and linked open data. Global names, and a data model made of references The RDF on which RiC-O rests is where this line arrives. Names become **IRIs** (Chapter 5), so uniqueness is guaranteed not locally but by the machinery of the web. The meaning of a reference is carried by a property drawn from a standard vocabulary, so that created, accumulated and holds are distinguishable by machine. And, the deepest of the three changes, in RDF *the data model is made of references*. Writing an IRI in the object position of a triple is a reference, and a collection of references simply is a graph. Reference stops being the plumbing and becomes the substance.

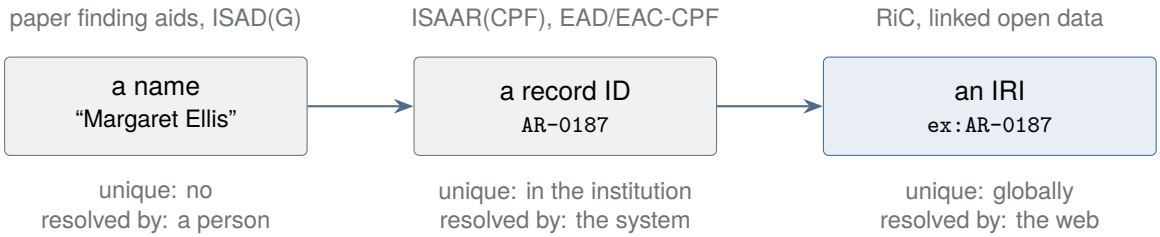


Figure 2.4: How the form of reference changed. Two things were strengthened by stages: the range within which a name is unique, and the means of getting to what it names.

This sketch gets used repeatedly from here. How to read arrows and the direction of reference is in Section 5.1.2; the comparison of three architectures (a statement inside a document, a foreign key, an arc in a graph) is Figure 7.2; who issues and maintains IRIs, which is an infrastructure problem rather than a modelling one, is Section 7.6.

2.5 RiC and ISO 23081

There are close relatives outside the archival standards. ISO 23081, on metadata for records management, is a framework for managing records as evidence from the moment they are made, and RiC-CM speaks of a “substantial commonality of purpose” across the two frameworks (§1.6.3). The multi-entity approach of RiC-CM was itself strongly influenced by the ISO 23081 framework, which relates entities such as records, agents, business and mandates. What this means in practice is that metadata created during the records management stage can be carried forward into archival description, and that the case for doing so keeps getting stronger. Differences between the two entity models remain, and convergence is stated as a goal.

2.6 The neighbours: RiC among the GLAM standards

Describing cultural material is work shared with neighbouring fields. Galleries, libraries, archives and museums collect, preserve and describe, and the shared acronym **GLAM** covers all four. Each community has developed its description standards in parallel with the others. RiC is easier to place when set beside them, and in practice it gets used *together* with them. One RiC term is needed in advance here. **Instantiation** is the appearance of a record as inscribed on a carrier, paper or a file, as distinct from the content of the record itself (Section 3.5). IIIF and PREMIS both connect to RiC at that point.

CIDOC CRM, the nearest in thinking The museum world’s CIDOC CRM describes cultural material not as a bundle of attributes attached to an object but as a *chain of events*: who made

Table 2.2: Neighbouring standards in the GLAM field, and where RiC touches them

Standard	Community	Central unit of description	Relation to RiC
CIDOC CRM (ISO 21127)	Museums	Events: production, acquisition, movement	Closest in outlook: relations first, a graph assumed. Semantic mappings have been studied
RDA / IFLA LRM, BIBFRAME	Libraries	Work–expression–manifestation–item	Shares the separation of content from carrier. Name authorities and identifiers can be borrowed
IIIF	Image and AV delivery	The manifest (a viewing unit)	Not a metadata standard but a delivery API. Connects at the instantiation; viewing is left to it
PREMIS	Digital preservation	Preservation events, formats	Intellectual description (RiC) and preservation management (PREMIS) divide the work at the instantiation
ISO 23081	Records management	Records in their business context	See the previous section. The direct predecessor of the multi-entity model

it when, who acquired it, where it moved. It is standardised as ISO 21127. Relations treated as first-class, a graph assumed, provenance followed through events: the museum community walked the road RiC took in the 2010s some years earlier. The difference is focus. CRM is a general event model for cultural material and pitched at a high level of abstraction; RiC is specific to records and provenance. Semantic mappings between the two have been worked on (RiC-CM to CIDOC CRM), which will matter for the borderline cases: documentary material in a museum, art in an archive.

Libraries: content, carrier, and authorities The library conceptual model IFLA LRM, which underlies the cataloguing code RDA, divides the bibliographic world into work (the content), expression (the realisation of that content), manifestation (the edition) and item (the copy in hand). BIBFRAME, from the Library of Congress, is its RDF implementation. That separation of content from carrier grew out of a world in which many copies of the same content exist, and it is the same thought as the RiC distinction between record resource and instantiation (Chapter 3), with the difference that archives, dealing mostly in unique material, need two layers rather than four. More useful still, in daily work, are the authorities. Libraries hold by far the deepest accumulation of controlled names for people, bodies and subjects, and VIAF (an international hub gathering the national name authority files of many countries; Section 7.6) and the national name authority files can be referred to by IRI. Link a RiC Agent to one of them with `owl:sameAs` (the OWL property declaring that two IRIs name the same thing) and the library infrastructure is available as it stands (Section 7.6).

IIIF: dividing the work with the viewer IIIF (International Image Interoperability Framework, pronounced “triple-I-F”) is not a metadata standard but a set of APIs for *delivering and displaying* images and audiovisual material: an image API, and a manifest in JSON-LD that gathers up a viewing unit. It meets RiC at the instantiation. Describe the digitised image as an instantiation on the RiC side, point from there to a IIIF manifest, and the two questions divide cleanly: what the thing is (the RiC graph) and how it is shown (the IIIF viewer).

Here is the first piece of RDF in this book. The notation is Turtle, used throughout from here on; its syntax is set out in Section 5.1. Until then only this is needed: three words, subject, predicate and object, make a statement, a full stop ends it, a semicolon carries the same subject on to the next statement, and `a` in the predicate position means “is a member of the class”.

```
ex:diary  rico:hasOrHadDigitalInstantiation  ex:diaryScan .
ex:diaryScan  a  rico:Instantiation ;
    rico:identifier  "https://archives.example.org/iiif/diary/manifest" .
```

PREMIS: dividing the work with preservation PREMIS is a data dictionary for digital preservation metadata (there is an OWL version), recording *preservation events*: formats, fixity checking (recomputing a checksum to confirm that a file has not changed), migration. The practical division is that RiC carries the intellectual and contextual description and PREMIS the preservation management, with the instantiation as the point they share.

RiC, then, is the archival instance of a shift from bundles of attributes to graphs that has been going on across the GLAM fields at the same time. Learning RiC is one way into that larger movement.

2.7 How the transition is envisaged

RiC-CM §1.6.4 is realistic about the move to RiC, and makes two points. "Descriptions based on ISAD(G) are accommodated by RiC-CM" (§1.6.2), so existing description need not be thrown away; it can be placed inside the RiC framework. And the transition "will be gradual". Institutions differ enormously in size, resources and political setting, and EGAD says openly that many "will simply not have the resources to immediately embrace RiC-CM". Those that do have the need and the means are expected to implement, to feed back, and "to pave the way" for the rest. Nobody expects everyone to move at once.

The forecast may be optimistic, but one thing follows from it plainly for anyone starting to learn. Knowledge of ISAD(G) and EAD is not wasted. Designing a conversion from existing data (Chapter 7) demands an exact understanding of the structure being converted from. Learning RiC does not replace learning the older standards. It is built on top of them.

Exercises (hints in Appendix A.3)

2.1 The reference code heads the list of six elements ISAD(G) considers essential for international exchange. Explain why, using the words *reference* and *identifier*.

2.2 Find one reference made by a string and one made by an identifier, in the EAD and EAC-CPF examples of Section 7.4 or in a catalogue you have to hand.

2.3 A reference made with `resourceRelation` in EAC-CPF runs in the opposite direction to one made with `authfilenumber` in EAD (both appear in Section 7.4). State, for each, which document points at what.

2.4 Explain why library authority files such as VIAF matter to archives, from the standpoint of Section 2.6.

Chapter 3

What RiC Changed

This chapter takes five changes that RiC makes to the older standards, and then faces the question that follows from all of them: what has been given up in exchange for the freedom gained.

3.1 Change 1: from a model of the finding aid to a model of the world

The most basic change fits in one sentence, quoted in Chapter 1 and worth having again: "The existing ICA standards model description, that is, they model a finding aid, whereas RiC-CM models the entities as such, as a basis for describing but without anticipating any particular end product" (RiC-CM §1.2).

The 26 elements of ISAD(G) are, in the end, a list of what a good finding aid contains. That is why they come in an order, why there is a unit of description, and why there is a hierarchy: the shape of the *output*, the finding aid on paper, is reproduced in the standard. RiC-CM turns this around. Describe the world first (records, people, activities, rules, dates, places) as a network of entities and relations, which is to say standardise the *input* to the system; then generate the outputs (finding aids, exhibitions, published datasets) from that network as they are wanted. The standard says nothing about the form of the output. RiC-CM is explicit that this is deliberate: it wants "user interfaces unconstrained by rules that might unwittingly hamper efforts aimed at innovation and experimentation" (§1.6.2).

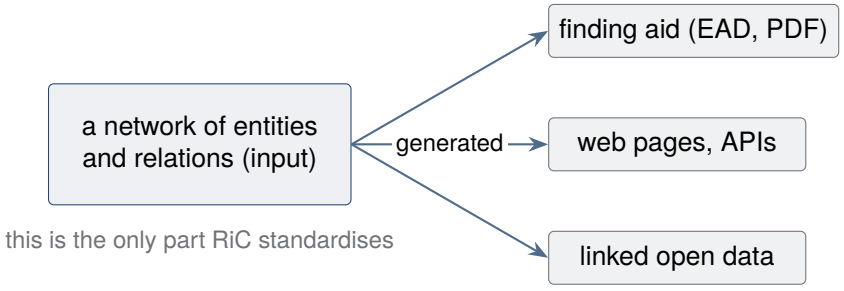


Figure 3.1: Input separated from output. RiC settles the conceptual structure of the input, the description data, and leaves the output to the implementation.

Separating the two is ordinary practice in system design, and it points the same way as the separation of data from presentation that relational database work has followed for decades. One way to read the change is that archival description standards have caught up.

Modelling the world: how close is this to a road already travelled?

"Modelling the world itself" may ring a bell. Through the 1970s and 1980s, the era of expert systems, frames and semantic networks, artificial intelligence pursued precisely that programme: write down knowledge of the world in formal terms and have a machine reason over it. It did not work out. The reasons are well documented. Formalising knowledge by hand cost more than it returned (the knowledge acquisition bottleneck); the knowledge, once written, was not maintained; and commonsense reasoning proved beyond formal logic. The long labour of the Cyc project, which set out to write down human common sense, is usually cited as the emblem of this. The **Semantic Web** proposed in 2001 (write information on the web so that machines as well as people can make sense of it, and have them follow links and reason) did not arrive in the form first drawn either, and RDF, OWL and SPARQL, the technologies this book covers, all descend from it. Whether RiC is heading down the same road deserves an answer.

The RiC sources nowhere discuss this history; there is no passage in the drafting documents about lessons from the AI winters. But look at what the design actually does, and RiC has taken a deliberately narrower road than the one that failed. Four things mark the difference.

The domain is bounded The "world" RiC models is records, creators and activities: *the range for which the record-keeping professions are answerable*. RiC-CM opens by listing what it is not (§1.2), and anything else that might be a subject of a record is merely *named*, through the catch-all entity Thing, with its description left to whatever community owns it. There was never a plan to write an encyclopaedia.

The weakness of the inference is deliberate As Chapter 6 shows, inference in RiC-O is limited to the structural and terminating kinds: class hierarchies, inverse relations, transitivity, shortcuts along a path. It does no commonsense reasoning and does not aim to. This reads as a domain-limited version of what Semantic Web research took from its own disappointments: a little semantics used widely gets further than a lot used narrowly.

That choice has a technical side to it. One of the hard parts of commonsense reasoning is *negation*. To conclude "no creator is recorded for this record, therefore it has no creator" requires the assumption that the data in hand is complete on that point. That assumption is the **closed world assumption** (CWA). Knowledge representation in the expert-system era, relational databases, and negation-as-failure in Prolog all stand on it. Only with a closed world can you write rules with exceptions bolted on afterwards ("birds fly, except penguins"), which is what non-monotonic reasoning is for. The machinery that handles common sense well cannot be built without it.

But the assumption is a strong one. In a domain where the data is not complete, it turns what merely has not been written into what is false. Archival description is exactly such a domain: records whose creator is unknown, records not yet examined, are commonplace. So OWL takes the other side, the **open world assumption**: what is not stated is not false but unknown (Chapter 5, and Section 6.9). On a web where data is pooled across institutions, no dataset can be more than partial, which makes the choice close to forced. Give up commonsense reasoning, keep only the inferences that survive being pooled: that trade is why inference in RiC-O looks so unambitious. In practice it means that a query about absence, such as a list of records with no creator recorded, is written as a check on the data rather than as a logical negation (Chapter 6).

There is an answer to the knowledge acquisition bottleneck The knowledge RiC formalises is not knowledge to be written from scratch. It is expertise that archivists have *been producing for more than a century*, in the form of finding aids. RiC re-formalises it; it does not ask for encyclopaedic new entries. A profession that writes this knowledge, and has a standing motive to maintain it, existed before the standard did. That is the decisive difference from most projects of the expert-system era.

Judgement is not handed to the machine Record or record set, the same record or a new one: RiC supplies the *form in which a judgement is expressed* and leaves the judgement itself with a person throughout, in the posture RiC-AG states as "there is no definitive right or wrong approach". RiC puts an expert's understanding into a form that can be shared with a machine; making a machine understand the world was never the plan.
So "a model of the world" in the heading of this section should be read in the narrow sense: modelling what is described *directly*, rather than through the output that a finding aid is. That narrowness is what may let RiC survive. It does not exempt RiC from every lesson, though. The cost of formalising and maintaining, and the absence of shared infrastructure (Section 3.6, Section 7.6), are the domain-specific form of the knowledge acquisition bottleneck. Whether RiC avoids the fate of the earlier programme will be settled not by the design of the vocabulary but by who carries that cost, and for how long.

3.2 Change 2: from multilevel to multidimensional description

Multilevel description in ISAD(G) is a single self-contained hierarchy with the fonds at its head. RiC-CM replaces it with multidimensional description (§1.6.2.2). Description takes the form of a graph, the traditional hierarchy sits inside that graph *unchanged*, and relations that a hierarchy cannot hold become expressible.

The example in the source makes it concrete. An activity is carried out by several organisations in succession, and one record series documents the whole span of it. Traditional description has to force that series into the fonds of one of the organisations. Multidimensional description relates the series to the activity and *at the same time* places it in the fonds of each organisation in turn. That one record or record set can belong to several record sets at once (the scope note of RiC-E03[1]) also opens the way to describing secondary groupings: a subject collection assembled by a researcher, a virtual exhibition, a project spanning several institutions.

RiC does not repudiate hierarchical description. A record set can have record sets as members (series, sub-series, file). RiC-CM §1.6.2.2 goes further and sets out to make the "inheritance of description" machine-processable: of the information written at the level of a set, those attributes and relations shared by every member (that all the records document the same activity, say) can properly be inherited as description of each member, while summary information (a range of dates) is inherited only as context, not as an attribute of any individual member. The distinction returns in the treatment of attributes in Section 4.3.

[1] RiC-CM gives every entity, attribute and relation an identifier: `RiC-E01` onwards for entities, `RiC-A01` for attributes, `RiC-R001` for relations. This book often drops the prefix and writes `E17` or `R026`. Each is described in a fixed set of fields; for an entity there are six, namely Identifier, Name, Definition, Scope Notes, Examples and Comments (§2.1.1). The scope note states the range of application and the rules of thumb for judging that a definition alone cannot convey, and when this book says "the scope note says", it means that field.

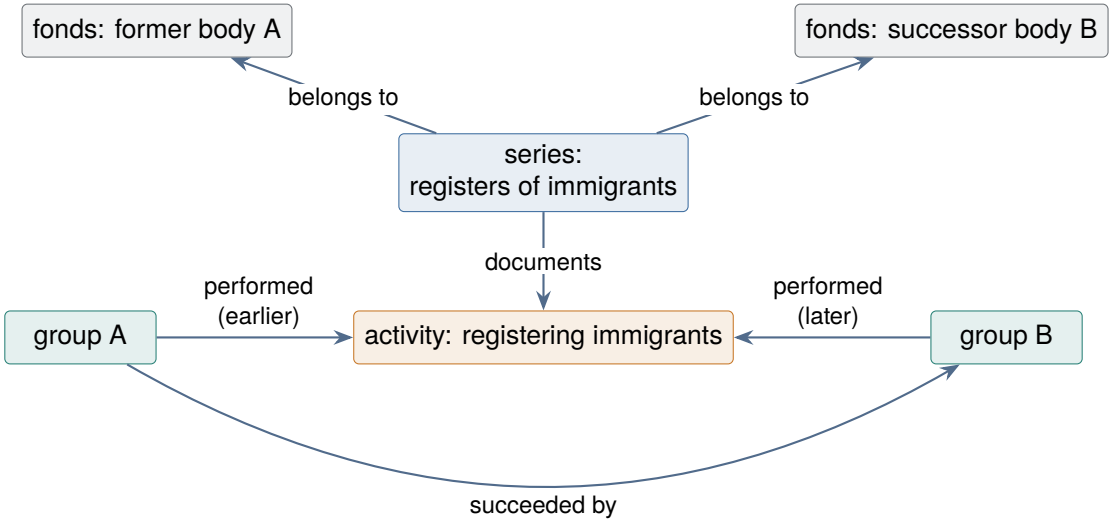


Figure 3.2: Multidimensional description. One series sits in the fonds of both organisations that carried out the activity in turn. A tree cannot state this situation accurately.

3.3 Change 3: a wider understanding of provenance

The change is concentrated in the plural in the title of the standard: Records in *Contexts*. The traditional principle of provenance privileges the single person or body that accumulated the fonds. Archival theory of recent decades has criticised that from two directions (RiC-CM §1.5.3, RiC-FAD §6).

The intellectual criticism is that the origin and history of a record involve more than the accumulator: other people and bodies took part in creating and using it, and so did a range of activities around it. Records are commonly made jointly, or one after another by different hands, and collaborative editing in a digital environment has made this more tangled, not less. The ethical criticism is that a description privileging the accumulator makes everyone else invisible: those who took part in creating and using the records, and those who are *the subjects of* the records (the indigenous people entered in the records of a colonial administration, the people held in the records of a camp).

RiC accepts both. It reaffirms the methodological soundness of the traditional principle, and at the same time widens provenance to mean that a record comes into being, and goes on existing, within a dynamic web of relations to other records, activities, persons and groups. The multi-entity model and the network of relations are the machinery for putting that wider understanding into description. Because the model does not force a choice of who stands at the centre, descriptions from several perspectives, and several routes of access to the same web of records, can coexist.

Provenance and custodial history: what separates them

Provenance sits next to custodial history and gets confused with it. The two need separating. **Provenance** is about the *context of creation*. In the wording of RiC-FAD §5 it is the relation of having been "created, accumulated, and used by a person or group in the course of life and work": it asks whose records these are, and out of what undertaking they came. The *Dictionary of Archives Terminology* of the Society of American Archivists[a] gives as its first sense "the origin or source of something", and its notes tie that origin to "the individual, family, or organization that created or received the items in a collection".

Custodial history is about what happened to the records *after* they left the creator. This is element 3.2.3 of ISAD(G), whose stated purpose is "to provide information on the history of

the unit of description that is significant for its authenticity, integrity and interpretation".
The English names need care, because two are in circulation and they do not cover the same ground. ISAD(G) 3.2.3 is headed *Archival history*; the element in the same position in the American standard DACS (5.1) is called *Custodial history*. ISAD(G) takes the wider view: alongside changes of ownership, responsibility and custody it lists the history of arrangement, the production of contemporary finding aids, re-use of the records for other purposes, and software migration, so the processing done by the holding repository itself falls inside it. DACS 5.1 cuts the period at the door: from leaving the creator to being acquired by the repository, with nothing about the repository's own work. On where the history begins the two agree: ISAD(G) directs that when the unit of description has been acquired directly from the creator, no archival history is recorded and the fact goes instead into the immediate source of acquisition (3.2.4).
Two axes hold provenance and custodial history apart. The first is *time*: provenance points to the making of the records, custodial history to the route they travelled afterwards. The second is *the direction of the question*: provenance answers "what are these records evidence of", custodial history answers "are they genuine, and what happened to them on the way".
What muddies this is that the word provenance has a wider sense too. The same dictionary gives as a second sense "information regarding the origins, custody, and ownership of an item or collection", and in that sense custodial history is part of provenance. The two senses have collided in practice. A body of photographs was arranged under the provenance of the donor, one point in the chain of custody, with the result that the provenance that mattered, the photographer, disappeared from view. The dictionary sums it up as an application of the principle of provenance obscuring provenance based on origin. When using the word in the wide sense, say which sense is meant.
RiC brings the distinction in as a *difference between types of relation*. The provenance relations (R026, has provenance, and those below it) have Agent as their range, and the definition is confined to the party that creates, accumulates, receives or sends. Custody does not go here; it is placed under management relations (R038 is or was manager of, R039 is or was holder of). For prose there is the History attribute (RiC-A21), which takes both, and whose specification covers "the history of origination, responsibility, property, custody, control, arrangement, description, and management of the record resource". So the design folds the ISAD(G) notion of archival history into a single attribute for narrative, and distributes it among relations of different types when the description is structured.
The word moves when it crosses into a neighbouring field. In the art world and the rare book trade, provenance usually means the *history of ownership* from the making of a work to its present holder, and it is used to establish authenticity and to ground claims for the restitution of looted art. That is close to what archives call custodial history, and its centre of gravity is elsewhere than archival provenance.
The two differ because the questions that produced them differ. Archival provenance was framed as an answer to a problem of arrangement. The French Revolution gathered into one repository, the Archives nationales, the documents of the royal archives, the Parlement of Paris, the surrounding monasteries, the ministries of the old government and the émigré nobility. The first two directors, Camus and Daunou, treated the whole as one accumulation, divided it into five subject sections, and arranged within them by place, by date or by reign. Much of it could no longer be traced to its source at all. The circular of the Ministry of the Interior of 24 April 1841, drafted by Natalis de Wailly, laid down *respect des fonds* against that state of affairs[b].
Provenance in art has older roots. Notes of previous owners appear in the European sale catalogues indexed by the Getty Research Institute's *Provenance Index* from 1650, and in

household inventories from 1520: for works that move one at a time through a market, the chain of ownership is the evidence of authenticity and of title. What the late twentieth century added was the duty to publish what the research found, with the Washington Conference Principles of 1998 on Nazi-confiscated art as the turning point[c]. Meanwhile provenance in the world of data (the W3C PROV standard) is defined as information about the entities, activities and people involved in producing a piece of data or a thing, which is close to the archival sense. RiC-O widening the range of provenance to agents *or activities* with `rico:hasOrganicOrFunctionalProvenance` puts it near the PROV way of thinking.

[a]The SAA *Dictionary of Archives Terminology* is a useful starting point but not a source to be trusted without checking. Its entry for *digital signature* states in a note that a digital signature is a message digest encoded with the *public* key, and that the recipient verifies the sender's identity with "the matching *private* key". *This is wrong.* In public-key cryptography the two keys have the opposite roles: a signature is made with the signer's private key and verified with the signer's public key (NIST SP 800-63-4 describes it as an asymmetric key operation in which "the private key is used to digitally sign data and the public key is used to verify the signature"). If a recipient could hold the sender's private key, there would be no signature to speak of. The author wrote to the SAA in September 2025 pointing this out; as of July 2026 the entry is unchanged. Check a definition against more than one source, and where the field has a standard, against that.

[b]Michel Duchein, "Theoretical Principles and Practical Problems of Respect des fonds in Archival Science", *Archivaria* 16 (1983): 64–82; originally in *La Gazette des Archives* 97 (1977): 71–96. Duchein calls the circular the birth certificate of the notion of the fonds, and compares the practice before it to the excavation of Pompeii, where objects were lifted out of their context and treated as a collection of works of art.

[c]The sequence runs from the AAMD (Association of Art Museum Directors) report of June 1998, to the Washington Conference Principles of December 1998, signed by 44 countries, to the recommended procedures of the AAM (American Alliance of Museums) in 2000. The Metropolitan Museum of Art, in its own account, treats provenance and ownership history as the same thing.

A question put to original order

RiC-FAD §6 turns an equally frank question on respect for original order. When is "original"? During accumulation the order of records is dynamic and fluid, and things are rearranged repeatedly. Is "order" physical arrangement or intellectual relation? There is no agreement. And plenty of accessions arrive with no discernible order at all. RiC's framework for describing order as a *relation*, that is, as intellectual information detached from physical arrangement, follows from that reflection. In RiC-O the work is done by the Proxy class and the ordering properties, taken up in Chapter 5.

3.4 Change 4: the unit of description comes apart

The "unit of description" of ISAD(G) is a convenient abstraction: a fonds, a series and a single letter can all be described with the same 26 elements. RiC-CM breaks it into three distinct entities (§1.6.2.1, §2.2.2).

Record A body of information content, produced in the course of life or work and inscribed on a carrier.

Record Set One or more records gathered together *by some party* on the basis of shared attributes or relations. Fonds, series, file and collection are all Record Set Types.

Record Part A component of a record that has information content of its own and contributes to the record's intellectual completeness: an enclosure in a letter, the seal on a charter.

A caution about the names first. "Record Set" invites the reading that a record set is a kind of collection of records, and from there the reading that a record is a kind of record set, but there is no such subordination between them: both sit under Record Resource as distinct kinds of entity. That

a record *belongs to* a record set is described at a different level, as a relation, not as a classification. Chapter 4 returns to this in a box.

The reason for separating the two is that a record and a record set are *products of different activities*. Making an individual record, and gathering records into a set (classifying, arranging, accumulating), are commonly done by different parties, at different times, for different ends. The accumulator of a fonds and the creator of an individual record inside it are often not the same, and mixed provenance within a fonds is common. Treat the two as one and the description turns vague. Keeping the attributes of the set itself (who gathered it, when, and for what) apart from summaries of its members (a range of dates, a distribution of languages) makes the description mean something definite.

One more point matters: a record set is stated to be an *intellectual* grouping, not a physical one (RiC-E03). Its members need not sit in the same place. And what counts as a record and what as a part of one is said to depend on context and judgement (the scope note of RiC-E02). Is a photograph attached to an email a record or a record part? RiC supports either, and by designing records and record parts so that all the attributes and relations used to describe them are shared, it makes sure the difference is not fatal. This posture, supporting judgement rather than prescribing, runs through the whole standard. RiC-AG devotes an FAQ to judgements of this kind, and its guidance is practical. Alongside the principle that identity deriving from the thing itself points to a record while identity deriving from the members points to a record set, it observes that if the attributes and relations you need exist on only one of the two, that is a clue to the answer; and that *the same object may be judged differently as time passes*. A statistical table treated internally as a record set whose members are individual rows may, once published as a half-yearly extract, be treated as a single record. The examples are drawn from British import statistics and from parish registers.

3.5 Change 5: separating the record from the instantiation

The other distinction RiC-CM introduces is between the record (the information content, the message conveyed) and its instantiation, the inscription of that content on a carrier (§2.2.2). When the same minutes exist as a Word file, as a PDF and as a paper file, RiC can describe this as one Record with three Instantiations.

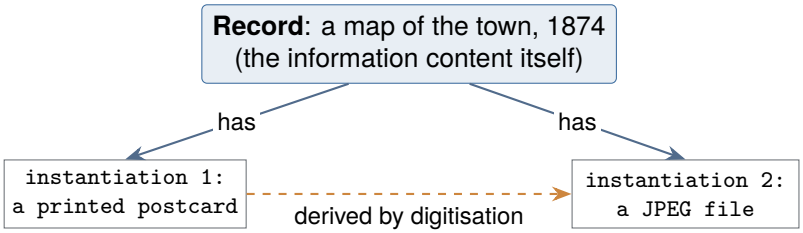


Figure 3.3: Record and instantiation, following the scope note of RiC-E06. A JPEG made by digitising the postcard can be treated as a new instantiation of the same record or as a new record. Which of the two is a judgement made by whoever is describing.

The rules run as follows. A record and a record part do not exist without at least one instantiation, though that instantiation need not survive at the time of description, which is what makes it possible to describe a lost original. A record set *may* have an instantiation (a set of registers bound into one volume) but need not. One record may have several instantiations, at the same time or at different times. And one instantiation may be derived from another, by copying, digitisation or format migration.

Here too RiC leaves the judgement with the person describing. Whether a derived instantiation

counts as an instantiation of the same record or as a new record depends on perspective and on the context of use (RiC-CM §2.2.2). The RiC-AG FAQ offers a test. If what you want to describe is a property *of the physical thing* (it is printed on paper, it was scanned at a given resolution), the description belongs to the instantiation; if it is a matter *independent of how the thing was realised* (it is in the queen's hand, it is the product of this administrative process), the description belongs to the record resource. A second, time-dependent test is offered too: a mechanical copy stays a new instantiation of the original record, but once it acquires a history of its own, through annotation for instance, it can be treated as a new record. To a diplomatist for whom the materiality of the carrier is inseparable from the meaning, a digital image may well be a different record. To a reader interested in the information conveyed, it is the same one. The source is explicit that RiC is a framework in which two or more equally defensible options can stand. RiC-CM also notes that the distinction corresponds to that between Intellectual Entity and Representation in PREMIS (footnote 15), which makes the design easy to join up with digital preservation practice.

Not the same as FRBR / IFLA LRM

Anyone with a library science background will think of the four layers of work, expression, manifestation and item (FRBR, now IFLA LRM). Record and Instantiation in RiC are two layers, and the two models were designed independently. The reason RiC-O gives for building its own ontology is that in 2013 "no generic domain ontology existed for the specific needs of the archival community". As a domain ontology it also "at this stage" "does not borrow from other existing ontologies (such as the cultural heritage models — IFLA-LRM and CIDOC-CRM, PREMIS, or PROV-O)", which should make it easier for an institution to understand, implement and maintain; alignment is left to a future revision cycle, "where possible". RiC-CM adds the underlying reason: "different cultural heritage communities have fundamentally different understandings of the nature of the objects for which they have curatorial responsibility" (§1.3). The safer course is to use the two models as separate tools rather than to try to map one onto the other.

3.6 Freedom of description, and what it costs

Gather the five changes under the heading of the describer's freedom, and you can see what RiC opens up and what it demands in return.

3.6.1 What became free

1. **Freedom of structure.** Description is not confined to one hierarchy. Multiple membership, relations across institutions, relations across time (a body and its successor, an original and a copy) can be stated directly.
2. **Freedom of perspective.** The model does not force a choice about whose point of view frames the records. A description centred on the accumulator and one centred on the people recorded can live in the same framework.
3. **Freedom of granularity.** A series you know little about can be described coarsely, as one node and two or three relations, while an important record is described down to its instantiations, events and rules, and both can sit in the same dataset. RiC-O supplies coarse classes and properties above fine ones in a hierarchy, so that description at any level is legitimate (RiC-O design principle 3, a flexible framework).
4. **Freedom of expression.** The same fact can be written the short way or the exact way (Chapter 5). The short way writes the value as a string or a number. Such a plain value is a **literal**: "Margaret Ellis", "1868", values that do their work as soon as they are read and

have nowhere further to lead. The exact way makes the same value into a thing described separately, and points at it. A creator written as a literal gives you a name to read; a creator made an entity gives you dates of birth and death, and relations to other records, to follow. The contrast between design A and design B in Section 4.1.2 is this same contrast. Dates work the same way: a string, or a Date entity related to the record.

5. **Freedom of product.** With input separated from output, one body of data yields finding aids, linked open data and exhibitions alike.

3.6.2 What it costs

The price comes in six parts.

(1) More judgement is required ISAD(G) handed you 26 elements to fill in. RiC hands you a vocabulary of 19 entities, 42 attributes, 85 relations, 107 classes and 555 properties, and *which of them to use, and how far* is for the describer, or the institution, to settle. Record or record part; a new instantiation of the same record or a new record; a date as a literal or as an entity. Everywhere RiC says "either will do", an institution has to make policy. Those decisions, accumulated and written down, are its application profile (Chapter 7).

(2) Consistency is harder to hold Where the form is fixed, the form guarantees consistency. With a free vocabulary there is always room for two people to write the same fact differently. That RiC-O admits several ways of expressing the same information is deliberate, and it eases migration, but the other face of it is inconsistency within a dataset. Search and inference depend on consistency, so inconsistency comes back as a loss of function.

(3) Managing identifiers becomes a new job Nodes in a graph need stable identifiers (IRIs). Making creators, activities, places and rules into entities in their own right means minting a unique and permanent identifier for every one of them, and keeping it alive. What used to be an access point, a string, becomes a managed resource with an identifier. That is an organisational and long-term commitment rather than a technical problem.

(4) The tools are immature ISAD(G) and EAD come with catalogue systems, editors and a track record of exchange. Systems that claim RiC support are still few. The RiC-AG draft of October 2025 is a collection of examples, and its refusal to give a step-by-step guide (Chapter 7) means that no form to fill in has appeared. Institutions are still writing their own policies, guided by the experience of early implementers and the examples in the AG.

(5) There is more to learn The multi-entity model, graphs and ontologies all take getting used to. How much technical grounding is needed varies a great deal by role, though (Chapter 8), and not everyone needs to be able to write OWL.

(6) There is no canonical form, and there cannot be The last point deepens the second. However carefully an application profile is drawn, a RiC-conformant expression of a given reality is not unique. What to make into an entity, how finely to cut, which relations to assert, whether to use a direct property or to make the relation an entity: the number of combinations grows with the expressive power of the model. The more that can be described, the more ways there are to describe the same thing. Freedom of expression and ambiguity of expression are one thing seen from two sides, and no amount of refinement in the design guidance removes the ambiguity. RiC

defines no normal form, no single form that counts as the standard way of writing something, and defining one would mean giving up the freedom.

This weighs on practice. “It can be written any way at all” amounts in the working day to “I do not know how to write it”, and if practice diverges institution by institution and cataloguer by cataloguer, the cross-institutional search and exchange that RiC exists for are damaged. From the position that fixing the form was precisely what a standard was good for, it is arguable that RiC has stopped being one.

What RiC offers against this is partial. Inference in RiC-O (Chapter 6) restores equivalence at query time for the variations that were *anticipated*. A long path and a shortcut are made to answer the same query by property chains (declaring in the vocabulary that if the long route holds, the shortcut property holds too); coarse and fine descriptions are made to answer the same query by property hierarchies (declaring the fine property to be subordinate to the coarse one). But variation that was *not* anticipated (whether to cut at the record or the record part, whether this is a new record or a new instantiation of the same one, how far to raise things into entities) is not absorbed by any of that. These are differences of judgement at the level of meaning, and inference does not rescue them. What is left is convergence through practice: application profiles shared within a community, conversion tools and description systems that establish a de facto normal form, and official collections of examples that set the going rate for a judgement.

The RiC-AG draft of 2025 bears this analysis out. Having refused a step-by-step guide, it writes that on the question of record or record set “there is no definitive right or wrong approach”, and instead of laying down a normal form it shows the situations where judgement divides and supplies material for judging (if the attributes and relations you need exist on only one side, that is a clue; a judgement may change over time). EGAD, in other words, has chosen to raise the quality of judgement rather than to recover uniqueness.

Having stopped being a standard of form and become a standard of vocabulary and meaning, RiC has put convergence beyond the reach of the standard documents; it can now happen only on the implementing side. Whether that transfer succeeds is not yet known, and this book records it as an unresolved structural risk. A second structural problem, that the *shared infrastructure* which convergence would depend on (a registry of extensions, identifier authorities that cross institutions) does not exist, is taken up in Section 7.6.

Table 3.1: Each freedom and its price

Freedom gained	Burden incurred
Release from the hierarchy (multiple membership, relations across institutions)	Choosing which relations to assert, and justifying the choice. Every relation is one more thing to maintain
Several perspectives	Writing down, as institutional policy, who and what will be described
Coarse and fine in one dataset	Setting criteria for granularity, and managing the mixture
Several ways to express one fact	Standardising practice within the institution (an application profile). Inconsistency degrades search and inference
Freedom of output	Needing the implementation skill to build the output; the standard does not help with it
Entities in their own right (creators, activities, places)	Minting and maintaining IRIs, permanently
Greater expressive power in general	No canonical form. The expression of a given reality is not unique, and convergence is left to practice in the implementing community

RiC has turned filling in a form into designing a model. That can be judged as raising cataloguing from routine to intellectual design, or as offloading an unreasonable design burden onto people who already have enough to do. What EGAD assumes is that systems and shared guidelines will

absorb most of the burden (RiC-CM §1.3, §1.6.2). The AG draft and the official crosswalks are a first step; whether the assumption holds depends on the tools that get built and on how far the AG matures.

Exercises (hints in Appendix A.3)

3.1 From the five changes in this chapter, pick the one of greatest value and the one of greatest burden for you or for an institution you have in mind, and give your reasons.

3.2 What does it mean to say there is no canonical form? Construct two ways of writing the same fact, and say how the difference would affect a search.

3.3 What misgivings about the move from a model of the finding aid to a model of the world are available to someone who remembers the failure of the expert-system programme? And what limits has RiC placed on itself in response?

Chapter 4

Reading RiC-CM: Entities, Attributes, Relations

This chapter is a reading of the body of RiC-CM 1.0, its chapters 2 to 6. It starts with the entity-relationship model that the document assumes, and then goes through entities, attributes and relations in turn.

4.1 Starting with the word “entity”

The word entity calls for a pause before RiC-CM is opened. Different fields put the weight in different places, and knowing which usage RiC-CM has taken makes the rest of the reading go more smoothly.

What RiC-CM takes is the usage of the entity-relationship model (ERM), a tradition of information system design (RiC-CM §1.4). In the ERM proposed by P. Chen in 1976, the things of interest are entities, their characteristics are attributes, and the connections between things are relations (or relationships). None of the weight of substance in metaphysics is carried over. An entity is closer to “something we want to manage as a record in a database”. In a library circulation system the entities are borrower, item and staff member, the attributes are name and shelfmark, and the relations are borrows and holds.

Strictly, ERM distinguishes an *entity type* from an *entity instance*: the type Person, and the particular instance Nelson Mandela. That instance is borrowed from the source, where RiC-CM offers exactly this person among the Examples for E08 Person. All 19 entities defined in RiC-CM are entity *types*, and the Examples attached to each definition are instances. Ordinary language does not separate the two, so the list of entities in RiC-CM should be read not as a list of 19 things but as a classification into 19 *kinds* of thing.

To make matters worse, the RDF and OWL world of later chapters has different names for nearly the same ideas. What answers to an entity type is a class; what answers to an entity instance is an individual, or a resource. In a relational database an entity type is a table and an instance is a row. This book gives the correspondence each time it matters; for now it is enough to see that one way of thinking has a dialect in each field.

4.1.1 How to read an ER diagram

The ERM comes with a diagram notation. In the classical notation of Chen’s 1976 paper, entities are rectangles, relations are diamonds and attributes are ellipses, joined by lines (Figure 4.2). Practitioners often use a more compact notation that lists attributes inside the rectangle (so-called IE notation, or the UML class diagram), but the principle of reading is the same. RiC-CM itself

Table 4.1: Words for the same idea across fields. The concepts are not identical, but the thinking is shared

	Type	Instance	Connection
ERM (RiC-CM)	entity	instance of an entity	relation
RDF/OWL (RiC-O)	class	individual, resource	property
Relational database	table	row	foreign key, join
Object-oriented design	class	object	reference

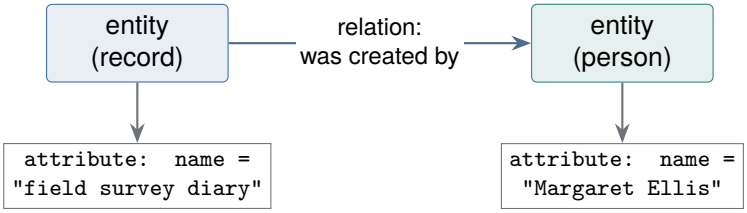


Figure 4.1: The elements of the ERM, corresponding to Figure 1 of RiC-CM. Boxes are entities, a line between boxes is a relation, a value hanging from a box is an attribute.

defines entities, attributes and relations in prose and tables, tied to no particular notation, and the diagrams in this book are drawings of the RiC-CM content made for legibility.

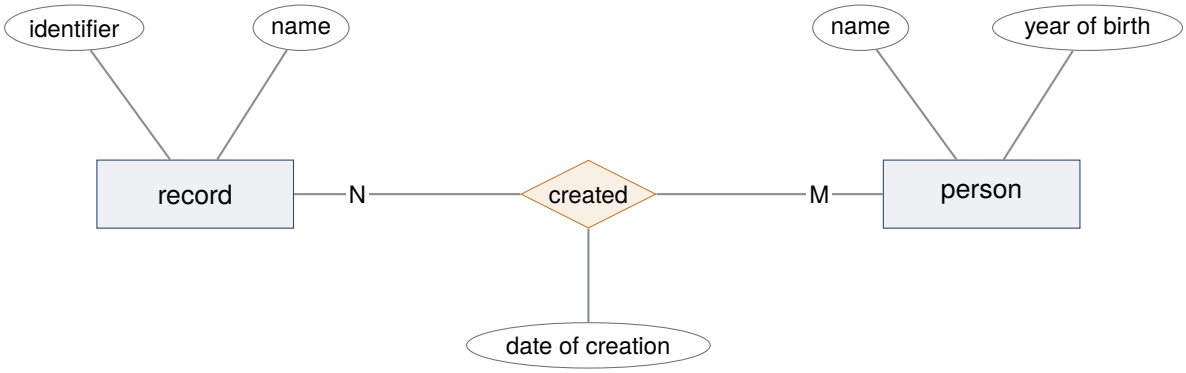


Figure 4.2: An ER diagram in Chen notation. Note that a relation can carry attributes of its own (the date of creation). RiC-CM defines attributes of relations (Section 4.4) in this tradition.

Four things are to be read off Figure 4.2. Rectangles are entity types. Diamonds are relations joining two entity types. Ellipses are attributes, and they attach not only to entities but *to relations*: when the record was created is an attribute neither of the record nor of the person, but of the relation of creating. The letters on the lines are the **cardinality**, saying how many of one side may be joined to one of the other. There are three cases, one-to-one, one-to-many and many-to-many, and this diagram is many-to-many.

RiC-CM states the cardinality of every relation (the Cardinality field of §5.4; R026 has provenance is M to M, R002 has or had part is 1 to M). It also inherits from the ERM the property that relations can carry attributes (Section 4.4).

4.1.2 One reality does not fix one model

The most important thing to take from the ERM is that *several correct models of the same reality are possible*. What is cut out as an entity, and what is left as an attribute, depends on the purpose of the description. Figure 4.3 sets two designs of the same fact side by side: that the field survey diary was created by Margaret Ellis.

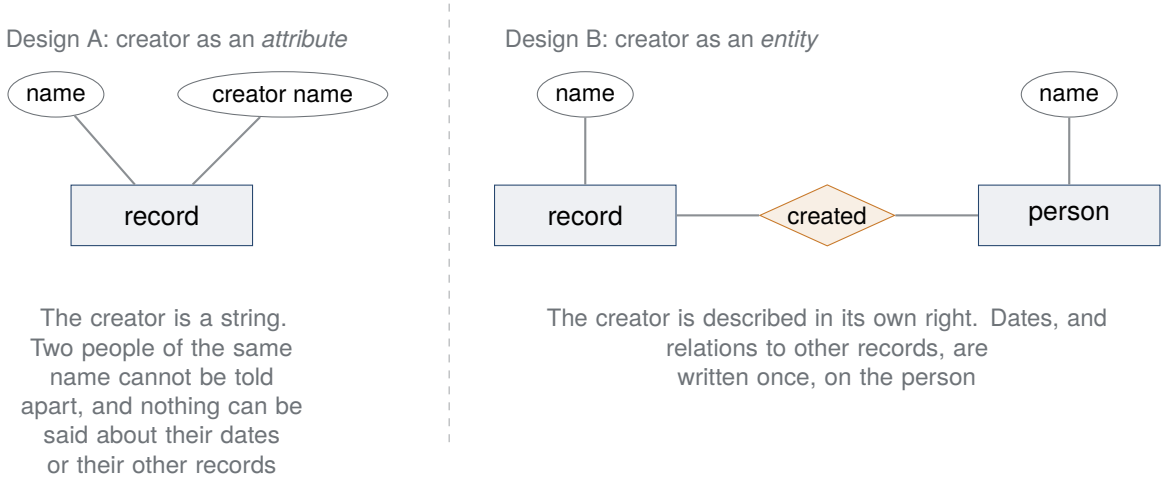


Figure 4.3: Two models of one fact. Neither is wrong; the purpose of the description settles the choice.

Design A is quick, and adequate if the record is all you care about. Design B costs more, but promoting the creator to something described in its own right makes it possible to distinguish people of the same name, to record dates and career, and to ask for a list of the records that person created. Neither is right in the abstract. The purpose chooses.

This choice runs straight into the subject of this book. When ISAAR(CPF) took the description of the creator out into an authority record, it moved from design A to design B (Chapter 2). RiC-CM raising 19 entities is the same move carried further, and the note in RiC-CM that an attribute which conceptually takes a controlled value is often better implemented as an entity (Section 4.3) is a statement that moving back and forth between the two designs is allowed. The point made in Section 3.6, that freedom of expression and ambiguity of expression are the same thing seen twice, has its root here. A RiC-conformant expression of a given reality is not unique because the ERM is not that kind of framework to begin with.

4.2 The hierarchy of entities

RiC-CM 1.0 defines 19 entities. They are not a flat list: they form a hierarchy under the relation "A is a kind of B", with Thing, which encloses everything, at the top.

Four of these are the core entities: Record Resource, the closely bound Instantiation, Agent, and Activity (RiC-CM §2.1). Agents perform activities in the world; in the course of those activities records are made and used; the records are evidence of the activities having been performed. That sentence is the RiC view of the world in miniature, and the core entities are its cast. The rest (Event, Rule, Date, Place) are there to describe the core entities fully.

Reading the global overview diagram with care

RiC-CM §2.1 contains a single page-wide diagram titled "RiC-CM v1.0: a global overview" (Figure 3 in the source), showing entities as nested boxes with relation arrows between them.

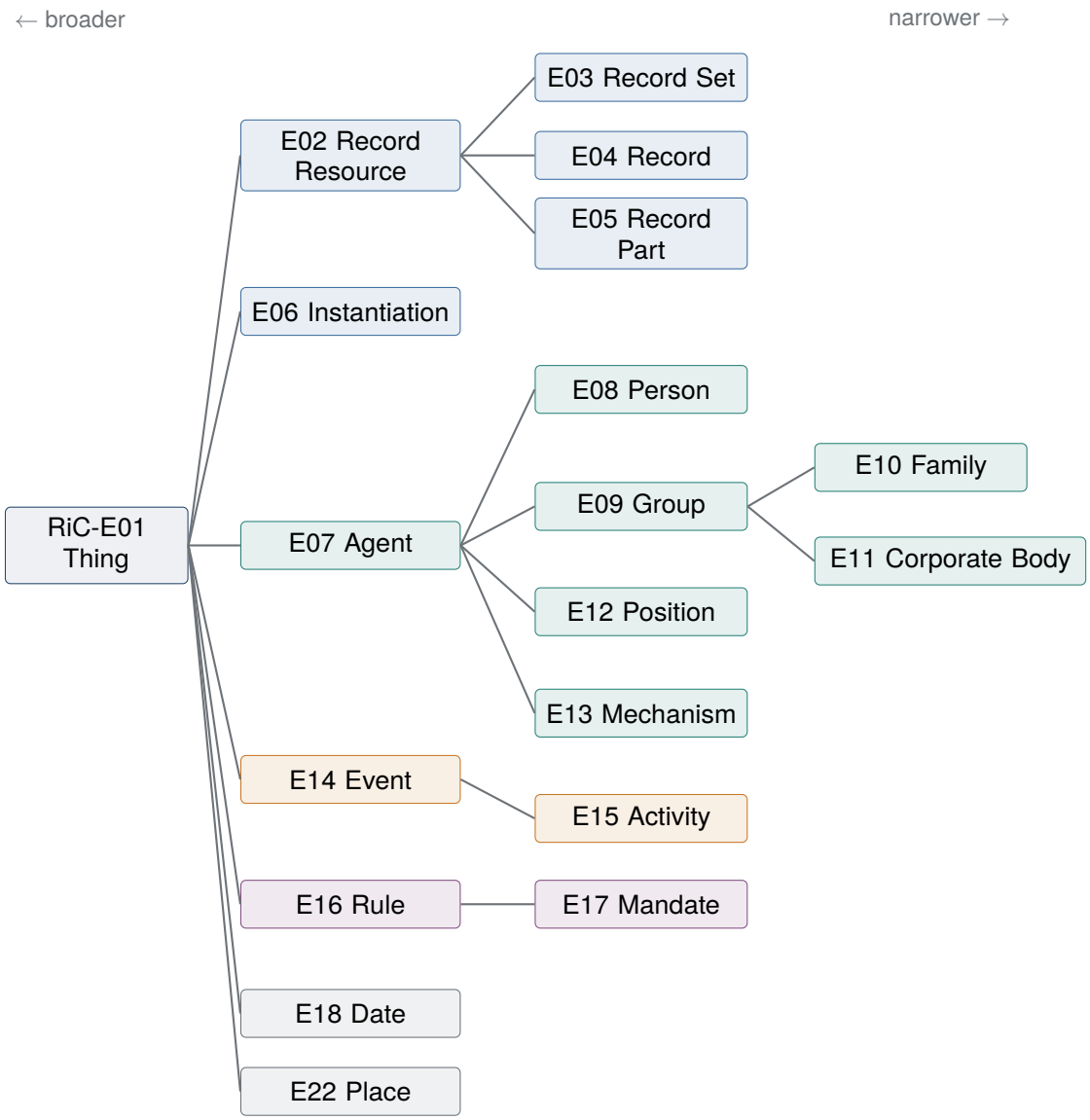


Figure 4.4: The entity hierarchy of RiC-CM 1.0, from the table in §2.1. Every line means "is a kind of". Note that the relation of a record set *including* records does not appear here at all (Section 4.4). E19 to E21 are vacant because the subentities of Date in version 0.2 (Single Date, Date Range, Date Set) were dropped in 1.0.

It is much reproduced, and it is a good diagram, but four things in it mislead if taken at face value.

- *Nesting is not inclusion.* The nesting of boxes shows only "is a kind of". Record Set (E03) and Record (E04) sitting inside the Record Resource (E02) box means no more than that both are kinds of record resource. The relation of a record set including records (R024) is not drawn in that diagram at all. Figure 4.4 is kept as a hierarchy-only diagram to stop the two being confused.
- *Thing (E01) is missing.* The root of every entity, and the domain or range of many relations, is absent from the overview. Relations that take Thing are drawn separately here, in Figure 4.8.
- *Only seven pairs of relations are drawn*: R019, R025, R026, R033, R060, R063, R080 and their inverses. That is a sample of the 85, not a map of them. For which entity can point to which, the place to look is the hierarchical chart in §5.3.
- *The core entities are not on one level.* The core entities are Record Resource, Instantiation, Agent and Activity, but Activity alone is not in the second tier, because it is a subentity of Event (E14). The source says so immediately before the diagram: the core entities other than Activity are in the second tier of the hierarchy. Activity looks tucked away in the drawing, but nothing about its importance follows from that.

What follows takes the entities in order. The definitions are summarised from RiC-CM §2.2.

4.2.1 Thing (E01)

"Any idea, material thing, or event within the realm of human experience." It is the root of every entity, and anything RiC-CM does not define explicitly is also a Thing. The Examples given for E01 are deliberately wide: the concept of slavery; Impressionism as an artistic movement; Puck, a fictional character from Shakespeare; the French Revolution as an event; and the Flatiron Building. That the definition begins with "idea" shows the point: having a physical form is not required. Records can be about *anything at all*, so a catch-all entity is needed as somewhere for subjects to point. The correspondence with the OWL root class `owl:Thing` is noted in the source.

4.2.2 Record Resource (E02) and what falls under it

"Information produced or acquired and retained by an agent in the course of life or work activity." Below it stand Record Set (E03), Record (E04) and Record Part (E05); Section 3.4 covered the distinction and its point. *Produced* covers more ground than *created*: producing a record resource, the scope note says, may imply either its initial creation or a reuse of information that already existed, by combination, rearrangement, selection or reformatting. The definitional core is an asymmetry: the identity of a record derives from the record itself, the identity of a record set derives from its members, and the identity of a record part derives from the record it belongs to.

The examples are not confined to paper. Among the official examples of a record, alongside a seventeenth-century letter of appointment under the royal seal, is a digitally signed email with two attachments; the attachments appear as examples of record parts (RiC-E04, E05). The scope note of RiC-E02 takes on the question of how to count born-digital information objects. Is a bitstream of an image embedded in a document file (a stretch of bits forming a unit of its own inside the file) a record or a record part? Are several bitstreams compressed into one ZIP file a record or a record set? Where digital information has no settled documentary form, the boundaries get harder to judge, and that recognition is the premise of the entity design.

How to read the word "Set": Record and Record Set are siblings in the classification

The name Record Set invites two misreadings: that a record set is *merely* a lot of records gathered up, and that a record is a special case of a record set with one member. Both come of confusing classification with membership, so classification comes first.
The lines in the entity hierarchy (Figure 4.4) mean only "is a kind of", which is *classification*. Record and Record Set are both kinds of Record Resource, but a record is not a kind of record set, nor the reverse. They are siblings in the classification, different kinds of thing. That a record is *included in, or belongs to* a record set is described quite separately, as an ordinary *relation* (RiC-R024; in RiC-O, `rico:includesOrIncluded` and its inverse `rico:isOrWasIncludedIn`). Keeping "is a kind of" apart from "includes" is the first step in reading the hierarchy.
Nor is a record set just its contents. An anthology is the useful analogy. Take the same ten pieces of writing: one editor gathers them as a memorial volume, another as a collection of local historical sources. The contents are identical and the two books are not the same book, because who compiled them, when, and for what, differ. So with record sets. When the source says that the identity of a record set derives from its members (§2.2.2), it means that a record set does not exist without members, not that the roll of members *alone* settles its identity. Gathering is an activity with its own purpose, distinct from the activity of making the records gathered (footnote 14 to the same section: even where the same agent is responsible for both the record set and its members, "it is the performance of two different activities with distinct purposes that brings each into existence"). Hence several record sets may be made from the same body of records, and a record set survives an accrual that adds members to it.
And a record set with only one record in it is perfectly possible (the definition says "one *or more*"), and it is a different entity from that record. An anthology containing one piece is not that piece. The anthology has an editor and a reason for existing, described apart from the author of the piece and the reason it was written.

4.2.3 Instantiation (E06)

"The inscription of information made by an agent on a carrier in any persistent, recoverable form as a means of communicating information through time and space." Section 3.5 covered it. Inscription on a carrier suggests paper or film, but the weight of the scope note examples falls on the digital side: the same record with three concurrent instantiations as DOCX, PDF/A and a paper printout; a large record set (a fonds or a series) maintained as a single database; a whole case file scanned into one PDF. The observation that some instantiations need a mediating device to communicate their information (a vinyl record, a video cassette, a digital file) leads directly into the problems of digital preservation.

4.2.4 Agent (E07) and what falls under it

"A thing that performs activities in the world." Not only human beings and their groupings: a role with no physical form (a position) and something existing as software (a mechanism) can be agents too. There are four subentities.

Person (E08) An individual human being. Most of the scope note is given over to *personas*, socially constructed identities. A pen name that eclipses the given name (Mark Twain), a persona shared by several people, a person who changes identity in the course of a life: the note is explicit that "one person, one identity" does not always hold.

Group (E09) "Two or more agents that act together as an agent." Family (E10) and Corporate Body (E11) fall under it. The three divisions of ISAAR(CPF), corporate body, person and family,

are absorbed here, with family and corporate body placed under Group so that unformalised groupings such as an electorate can also be handled.

Position (E12) "The functional role of a person within a group." A vice-chancellor, a county surveyor, a head of records. What is new is making the *intersection* of a person and a group an entity of its own. A record set produced by a position can then be described without pinning it to whichever individual held the post, which is a common practical need.

Mechanism (E13) "A process or system created by a person or group that performs an activity." Software, bots, probes. It acts under rules laid down by its designers and can create and alter records. A web crawler and a Mars rover are among the examples. This is the model's answer to an age in which automated processes generate records in bulk.

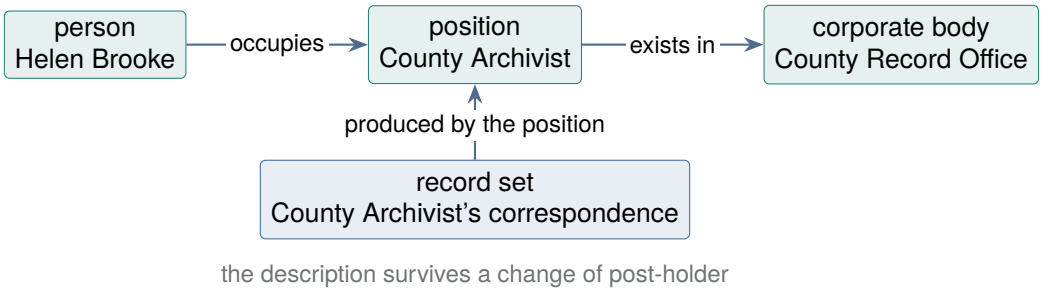


Figure 4.5: What Position is for. Tying records to a position rather than to a person or to a body gives a description that is not disturbed by staff changes. Based on the scope note of RiC-E12.

4.2.5 Event (E14) and Activity (E15)

An event is "something that happens in time and space". Natural occurrences (an earthquake, a flood), things brought about by agents (an election, a format migration), and combinations of the two (the salvage of water-damaged records) all count. An activity is a subentity of event, defined as "the doing of something for an agent designed purpose" and glossed in §2.2.4 as "an agent designed and performed event that has an intended purpose or purposes". In the context of a corporate body it covers the same ground as the older word "function", but function sits badly on what a person or a family does (writing poetry), so the more general word was chosen (RiC-CM §2.2.4). What ISDF dealt with is absorbed here.

The scope note observes that an activity can be viewed in two ways: by purpose (what it is for) and by process (how it is done). A functional classification of an organisation (a hierarchy of purposes) and a workflow (a sequence of steps) are both expressible as descriptions of activities.

4.2.6 Rule (E16) and Mandate (E17)

A rule is "conditions that govern the existence, responsibility, or authority of an agent; or the performance of an activity by an agent; or that contribute to the distinct characteristics of things created or managed by an agent". It is explicit that this covers not only written law, regulation and standards but *unwritten* social norms and custom. A mandate is a subentity of rule: one agent conferring on another the responsibility or authority to carry out an activity. The scope note narrows the activity to a specified one, and adds that limits of place (jurisdiction) and time normally come with it.

Two things trip up beginners. A rule or mandate is a *different entity* from the document that evidences it: the source is blunt, "a documentary source is a record". The statute is a record; the authority it confers is a rule. And description is itself an activity governed by rules (RiC-CM itself,

national cataloguing codes), so the Rule entity turns up again when description describes itself (Section 4.5).

4.2.7 Date (E18) and Place (E22)

A date is "chronological information associated with an entity that contributes to its identification and contextualization". To the reasonable question why this is an entity rather than an attribute, the answer given is that describing time is inherently complex (RiC-CM §2.2.6). A natural-language form ("Michaelmas 1694") alongside a machine-readable standard form (ISO 8601, and EDTF, its extension for uncertain dates and ranges; Section 7.7); degrees of certainty and precision. A single-valued attribute will not carry that. A place is a "bounded, named geographic area or region", covering jurisdictions, built structures and natural features, and treated as a historical entity whose beginning, end and boundaries change. References to coordinates are possible too.

4.3 Attributes

An attribute is a characteristic of an entity. RiC-CM 1.0 defines 42 of them (RiC-A01 to A45, with gaps) in alphabetical order (§3.6), and its next chapter tabulates which attributes apply to which entities. Name, Identifier, General Description, History, Language, Legal Status, Record Resource Extent, Scope and Content: many of the ISAD(G) elements find their counterpart here. Attributes come up less often in this book than the 19 entities and the 85 relations, but the list in RiC-CM §3.6 is the first thing to open when actually designing a description.

Three things help when reading them.

(1) The value schema is indicative, not prescriptive Each attribute is given a Value Schema, one of four: free text, model-based text, controlled value, or rule-based value (§3.5). But the source states that the value schemas are "indicative" and neither "prescriptive" nor "proscriptive". Here too RiC declines to lay down content rules. What to write, and out of which controlled vocabulary, is for the implementation to settle.

(2) Attribute and entity swap places in implementation An attribute that conceptually takes a controlled value (activity type, record set type, occupation type) is often better handled as an entity, that is, a class, in implementation, and RiC-CM says so itself (§3.3). Made into a shared vocabulary, such a value can be referred to from many descriptions, and in a linked data setting it can be connected to vocabularies elsewhere. RiC-O did exactly this, turning them into `*Type` classes (Chapter 5). "What was an attribute in the CM is a class in the O" is the single most important rule of translation when moving between the two documents.

(3) An attribute on a record set means one of two things The distinction drawn in Chapter 3 takes concrete form here. When "language: English" is written on a record set, it is not an attribute of the set itself but a summary of an attribute of *the records that are its members*. RiC-CM §3.4 faces this directly. Strictly, the relation should be split into "the set has all members with attribute X" and "the set has some members with attribute X"; but doubling the attributes inside an ERM would be unwieldy. So the CM settles for marking them, an asterisk for an attribute that describes the members, a plus for one that may describe both the set and its members. RiC-O implemented the strict version as families of properties such as `rico:hasOrHadAllMembersWithLanguage` and `rico:hasOrHadSomeMembersWithLanguage`. It is machinery for resolving, at a level a machine can act on, an ambiguity that everyday usage lives with ("this series is restricted").

4.3.1 Attributes in use

All three points show up in one example. Figure 4.6 puts attributes on the series “Communicable disease prevention” and on one record inside it, a typhoid notification.

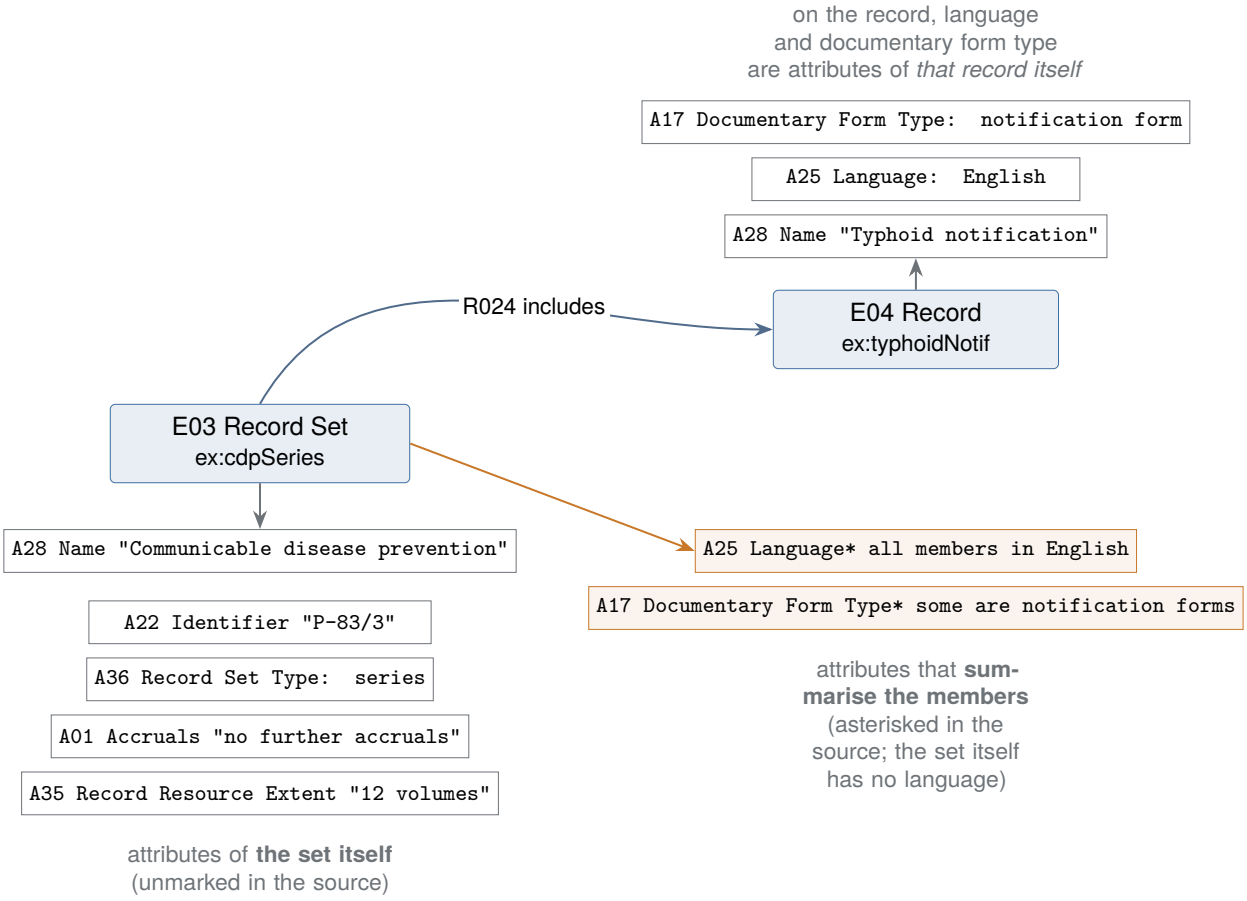


Figure 4.6: How attributes attach, following the list of record set attributes in RiC-CM §4.2.1. The same “language” is an attribute of the record when written on a record, and a summary of the members when written on a record set. The source marks the latter with an asterisk; RiC-O implements them as differently named properties. Note that Extent (A35) is unmarked: twelve volumes is not a summary of the members but the size of the set itself.

Written in RiC-O, the distinction appears as a difference of property name. The record set first.

```
ex:cdpSeries  a  rico:RecordSet ;
    rico:name        "Communicable disease prevention" ;   # name of the set
    rico:identifier  "P-83/3" ;
    rico:hasRecordSetType  rst:Series ;
    rico:accruals    "No further accruals expected" ;
    rico:recordResourceExtent  "12 volumes" ;      # size of the set itself
    rico:hasOrHadAllMembersWithLanguage   ex:eng ; # every member in English
    rico:hasOrHadSomeMembersWithDocumentaryFormType  ex:notification ;
    rico:includesOrIncluded  ex:typhoidNotif .
```

The same language and documentary form type, written on the record, look like this. Note that `ex:eng` and `ex:notification` are not literals but individuals in their own right.

```
ex:typhoidNotif  a  rico:Record ;
    rico:name  "Typhoid notification" ;
```

```
    rico:hasOrHadLanguage  ex:eng ;                  # language of this record
    rico:hasDocumentaryFormType  ex:notification .

ex:eng           a  rico:Language ;              rico:name "English" .
ex:notification  a  rico:DocumentaryFormType ;  rico:name "notification form" .
```

What deserves attention is that `rico:hasOrHadLanguage` *cannot be written* on the record set. The domain of that property — the kinds of entity that may stand at the start of it, in the sense set out in Section 4.4 — is agent, record and record part; record set is not in it. The domain of `rico:hasDocumentaryFormType` is likewise confined to record and record part. What the CM marked with a discreet asterisk, RiC-O enforces *through the domain itself*. Try to write "this series is in English" and there is nowhere to put it, so the writer has to decide whether all the members are in English or only some. The route that lets the ambiguity through has been closed.

Points (1) and (2) are in the example too. The values of `rico:accruals` and `rico:recordResourceExtent` are free text, and RiC has no rules about how to word them (the value schema is indicative). Meanwhile `rst:Series` and `ex:notification` are *references to individuals* rather than literals, because record set type and documentary form type, attributes taking controlled values in the CM, are made into instances of classes in RiC-O. Any attribute whose value schema is "controlled value" in RiC-CM §3.6 can be expected to take this form in implementation.

4.4 Relations

Relations are the connections between entities, and they are what actually carries the "Contexts" of the title. RiC-CM 1.0 defines 85 of them (§5.4), numbered RiC-R001 to RiC-R086 with R043 vacant. As with entity identifiers, this book drops the prefix and writes R026. Most relations come with an inverse (R*nnn*i), so counting the inverses separately raises the total further.

The entity hierarchy (Figure 4.4) says what kinds of thing there are and nothing about how they join up. The system of relations does that. What follows takes the classification into 13 types (Section 4.4.1), then the actual wiring between entities (Section 4.4.2), then the hierarchy among the relations themselves (Section 4.4.3).

Counting relations in the source

The table in §5.6, "List of Relations", ends at R079 and omits the date relations R080 to R086 ("is creation date of" and the rest). Those appear both in the hierarchical chart of §5.3 and in the individual definitions of §5.4. Count from the list alone and you will be seven short.

4.4.1 The 13 types

RiC-CM sorts every relation into 13 conceptual types (§5.2); one relation can belong to more than one type. Table 4.2 gives the 13, with the broadest relation in each, its domain (what can be the starting point) and its range (what can be the destination).

Read the range column downwards and eight of the 13 end at Thing. Thing is above every entity (Section 4.2), so these relations can take any entity as their destination; two of them, whole–part and sequential, leave the starting end unconstrained as well. The remaining five are confined to particular entities at both ends. The system of relations is therefore made of two kinds: relations that wire specified entities together, and general-purpose relations that do not constrain the far end. The first kind builds the skeleton along which the making of a record can be traced. The second attaches context (time, place, rule, subject) to any entity at all.

Table 4.2: The 13 types of relation and the broadest relation in each, from RiC-CM §5.2, §5.3 and §5.4. “Has or had” and “is or was” carry the sense “now or in the past”

Type	Broadest relation	Domain	Range
Whole–part	R002 has or had part	Thing	Thing
Sequential	R008 precedes or preceded	Thing	Thing
Subject	R019 has or had subject	Record Resource	Thing
Record resource to record resource	R022 is record resource associated with record resource	Record Resource	Record Resource
Record resource to instantiation	R025 has or had instantiation	Record Resource	Instantiation
Provenance	R026 has provenance R033 documents	Record Resource, Instantiation	Agent Activity
Instantiation to instantiation	R034 is instantiation associated with instantiation	Instantiation	Instantiation
Management	R036 has or had authority over	Agent	Thing
Agent to agent	R044 is agent associated with agent	Agent	Agent
Event	R057 is event associated with	Event	Thing
Rule	R062 is rule associated with	Rule	Thing
Date	R068 is date associated with	Date	Thing
Spatial	R074 is place associated with	Place	Thing

4.4.2 A map of the relations between entities

A list of types does not show which entity an arrow can run from and to. Figure 4.7 draws the relations that join specific entities, and Figure 4.8 those that do not constrain the far end.

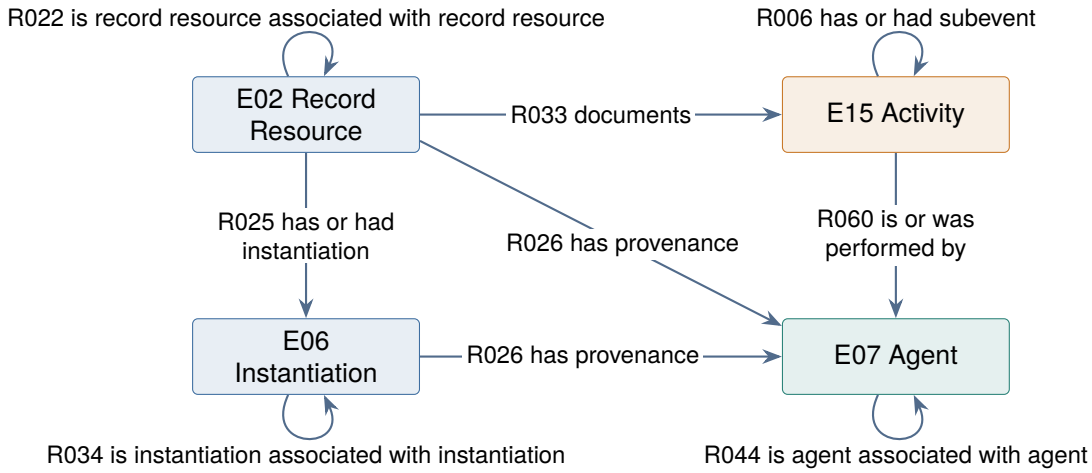


Figure 4.7: Relations joining the core entities, from RiC-CM §5.3 and §5.4. Arrows run from domain to range. An arrow returning to its own box is a relation between two entities of that kind. Every relation has an inverse defined alongside it, so an implementation can write from either end.

Figure 4.7 is the RiC view of the world from Chapter 3 drawn as a wiring diagram. An agent performs an activity (R060); the activity is documented in a record resource (R033); the record resource has an agent as its provenance (R026); on a carrier it exists as an instantiation (R025). Those five lines give the route along which the making of a record can be traced. The remaining arrows join entities of the same kind, carrying the sideways connections: this record set is related to that one (R022), this member of staff reports to that one (below R044).

The thing to watch in Figure 4.8 is which way the arrows point. In ordinary speech one puts a date on a record, but R068 takes Date as its domain and Thing as its range, and is defined as the date being associated with the thing. Event, rule and place are the same: the entity carrying the

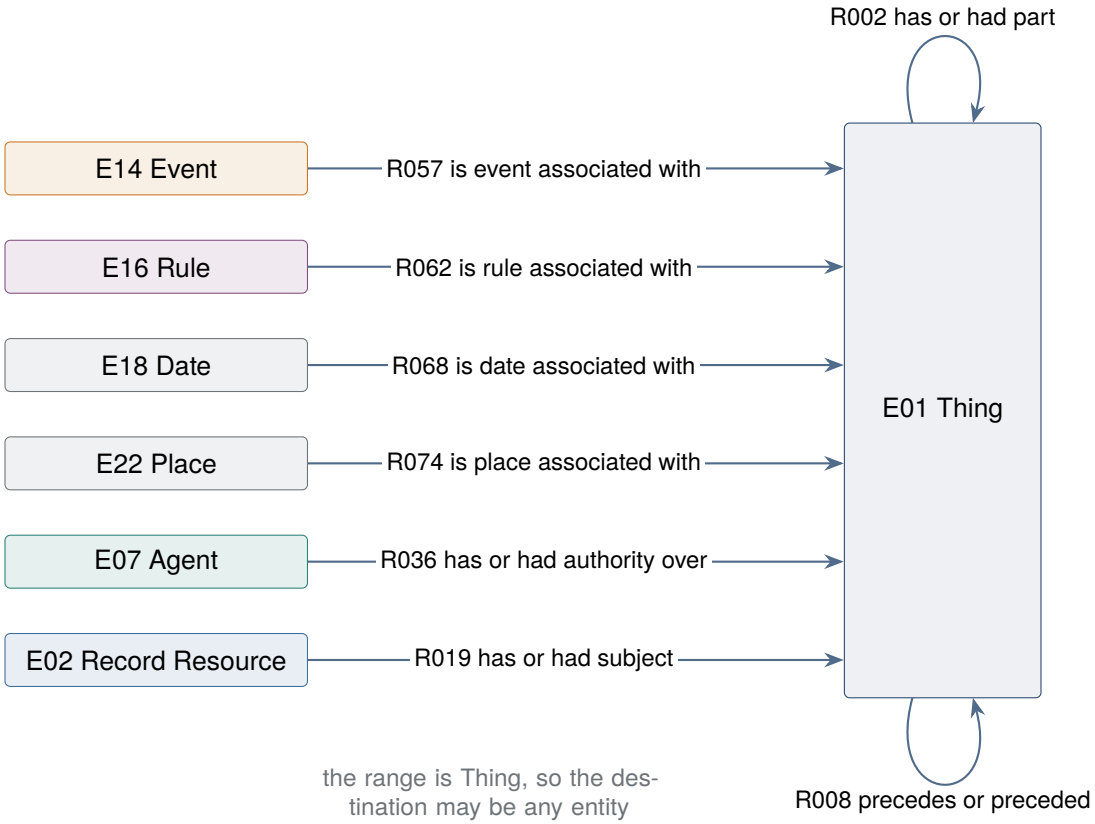


Figure 4.8: Relations whose range is Thing, from RiC-CM §5.3. Event, rule, date and place can be attached to a record resource, to an agent, to an activity, and to one another. The two arrows returning to Thing show that whole–part and sequence hold regardless of the kind of entity.

context is the starting point. RiC-CM does define an inverse for each (R068i), and RiC-O has both `rico:isDateAssociatedWith` (date to thing) and `rico:isAssociatedWithDate` (thing to date) as properties. In practice you write whichever direction is convenient, and the reasoner supplies the other (Section 6.3). Take the directions in the figure as following the domain and range as the source states them.

4.4.3 The hierarchy among relations

Relations are not a flat list either. They form a hierarchy from broad to narrow, headed by "a thing is related to a thing" (R001), which branches into the broadest relation of each of the 13 types (the second column of Table 4.2) and subdivides from there. The chart in the source (§5.3) has columns for five tiers. Most branches stop at the third or fourth; only the deepest reaches a sixth (R018, has child; with no column left, the chart pushes it into the fifth column with a note naming "the sixth level"). The hierarchy is polyhierarchical: one relation can have several parents and appear at several places in the chart, which is why some relations are annotated "(see also above/below)".

Taking the provenance type as the example, the hierarchy looks like Figure 4.9.

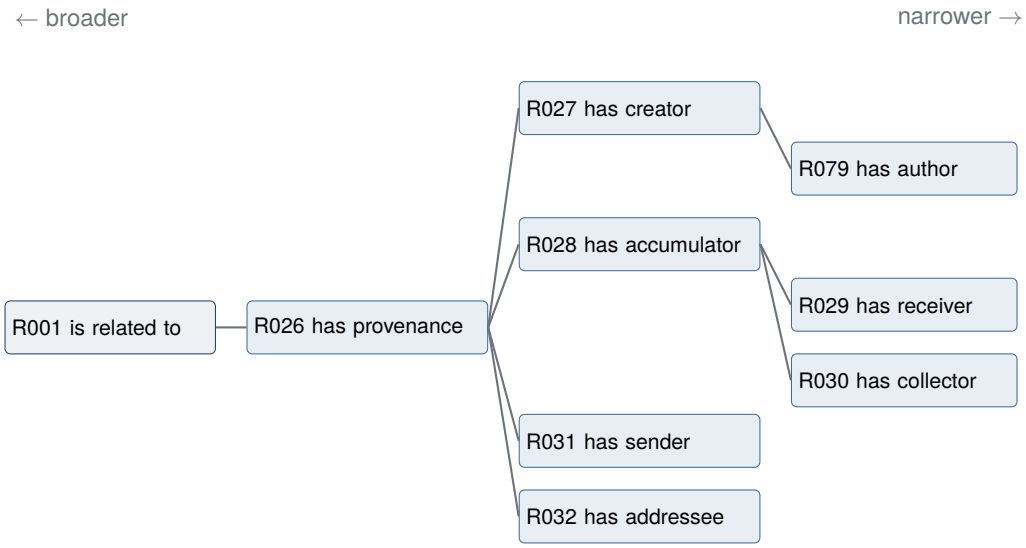


Figure 4.9: The hierarchy of relations in the provenance type, from the Broader and Narrower relations fields of RiC-CM §5.4. A line means "is a narrower relation than". R079 alone has its domain confined to Record and its range to Person, Group and Position; the others take Record Resource or Instantiation as domain and Agent generally as range.

This structure supports both description and retrieval. If a narrow relation holds ("this letter has Margaret Ellis as sender", R031), the broader relations above it hold automatically (has provenance, R026; is related to, R001). The describer chooses the narrowest relation the evidence supports; the searcher queries across the broad one. Freedom of granularity (Chapter 3) rests on this design, and RiC-O formalises it as subproperty declarations, leaving the reasoner to supply the broader relation (Section 6.4).

4.4.4 Relations have attributes of their own

RiC-CM §5.5 defines attributes that attach to a relation itself: six of them, RA01 Certainty, RA02 Date, RA03 Description, RA04 Identifier, RA05 Source, RA06 Place (each full name ends "of Relation": Certainty of Relation, and so on). Of the relation "A created B" one can then record when, how certain it is, and on what authority. A relation of creation might carry "date: about

1873–1875", "certainty: uncertain", "source: an inscription on the cover". The idea generalises what ISAAR(CPF) already had for describing relations. Since RDF cannot express an attribute on a relation directly, RiC-O implements this by *reifying* the relation: raising the relation itself into a node that can carry attributes (Section 5.5.3).

4.4.5 Reading a relation definition

Each of the 85 definitions (§5.4) consists of the identifier, name, domain (the entity a relation can start from), range (the entity it can reach), cardinality, definition, scope notes, examples, type of relation, broader relations and narrower relations. R026, at the head of the provenance type, has the name *has provenance*, the domain Record Resource or Instantiation, the range Agent, the type provenance, R001 as its broader relation, and R027, R028, R031 and R032 as narrower. Domain and range are used in exactly this sense in RDF and OWL, so getting used to them here pays off later.

When moving between the CM and the O, watch for names that do not match. The RiC-O property corresponding to R026 is `rico:hasOrganicProvenance`, with *organic* added. RiC-O records the corresponding CM relation identifier on every property in a `rico:RiCCMCorrespondingComponent` annotation, so matching by identifier rather than by name is the reliable method.

4.5 Describing description

Chapter 6 of RiC-CM is unusual among description standards. Its subject is the description of the act of describing. A description record is a record too: an agent (a repository, an archivist, a conversion program) performed an activity (describing) and produced it. So it can be described with the entities and relations of RiC *as they stand*, with no special vocabulary.

The source likens the four layers to zooming in on a map (§6.1): the repository (under what mandate and rules it preserves and describes), the position (which post does the work), the description record (when and by whom this catalogue data was created and revised), and the verification of individual statements inside the description (on what source does this date of death rest).

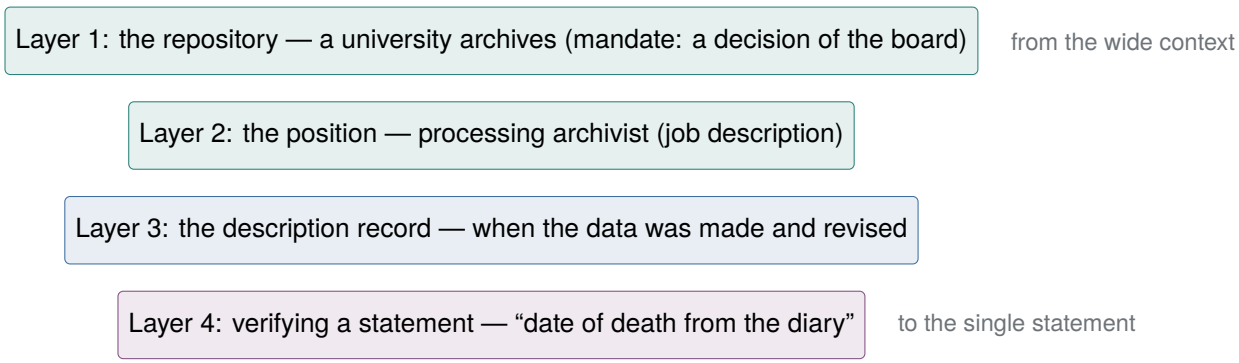


Figure 4.10: The four layers of documenting description (RiC-CM §6.1). Each layer is the context of the more specific one below it.

The chapter is also RiC's answer to a traditional division in metadata. Practice has distinguished **descriptive metadata** (describing the content and context of the records, for users to find them) from **administrative metadata** (who made that description, when, under which rules, and when it was revised), and has given the latter a place of its own: area 7 of ISAD(G) (description control: archivist's note, rules or conventions, date of description), `eadheader` and `control` in EAD, `control` in EAC-CPF. The two kinds of metadata lived in *separate compartments of the same document*.

RiC treats the distinction not as a difference of compartment but as a difference of *storey*. Administrative metadata is descriptive metadata about a record that happens to be a description record. No special vocabulary is needed: the same entities and relations are applied one storey up. The archivist who described is a `rico:Person`, exactly the class used for historical figures; the cataloguing code or application profile followed is a `rico:Rule`; describing is an `rico:Activity`; the dates of creation and revision are `rico:Date`. Written out (Figure 4.11), it looks like this. The square brackets are the Turtle way of creating an individual on the spot without giving it a name (Section 5.1).

```
# the description record: the catalogue data itself, as one record
ex:desc83  a  rico:Record ;
    rico:describesOrDescribed  ex:fondsP83 ;   # what it describes
    rico:hasCreator  ex:prescott ;             # the archivist who wrote it
    rico:isOrWasRegulatedBy  ex:profile2026 ;  # the rule followed
    rico:hasOrHadLanguage  [ a rico:Language ;
                             rico:name "English" ] ;
    rico:isAssociatedWithDate  [ a rico:Date ;
                             rico:normalizedDateValue "2026-07" ] .

ex:prescott  a  rico:Person ;                  # the describer is a Person too
    rico:name  "Alan Prescott" ;
    rico:occupiesOrOccupied  ex:pos1 .         # up to the position (layer 2)

ex:pos1  a  rico:Position ;
    rico:name  "Processing archivist" ;
    rico:existsOrExistedIn  ex:archivesOffice .      # up to the body (layer 1)

ex:archivesOffice  a  rico:CorporateBody ;
    rico:name  "Westmere County Record Office" .

ex:profile2026  a  rico:Rule ;
    rico:name  "Westmere description application profile, 2026" .
```

Two things matter here. One relation, `rico:describesOrDescribed`, joins *two records on different storeys*: the description record (desc83) and what it describes (fondsP83). Everything that went into area 7 of ISAD(G) is expressed as attributes and relations on the desc83 side. And *the describer gets no special treatment*. Alan Prescott the archivist is the same `rico:Person` as the historical actors who made the records, connected through a `rico:Position` to the institution of layer 1, a `rico:CorporateBody`. That is economical with vocabulary, and it is also a statement of position. Describing is itself a historical act, and the describer will in time be a figure in the history of the records: the contemporary archival argument that the archivist's intervention is part of the provenance is implemented here as model structure.

The diagram in the source is finer-grained than the example above in two respects.

- The source raises *describing as an activity* to an entity. A person performs an activity (R060i), and the activity results in the description record (R061, results or resulted in): three steps rather than two. The example above goes straight to the person with `rico:hasCreator`, which is a shortcut for the sake of an introduction. Writing a process as an activity is covered in Section 7.9.
- The layer 1 diagram in the source (Figure 4) contains a *retention schedule*. A rule, the records management policy, is expressed by a record, the retention schedule (R064). A rule and the record that evidences it are raised as separate entities and joined. Section 7.9 uses this pattern as it stands.

In practice there is a cheaper route than raising the description record to an entity: record

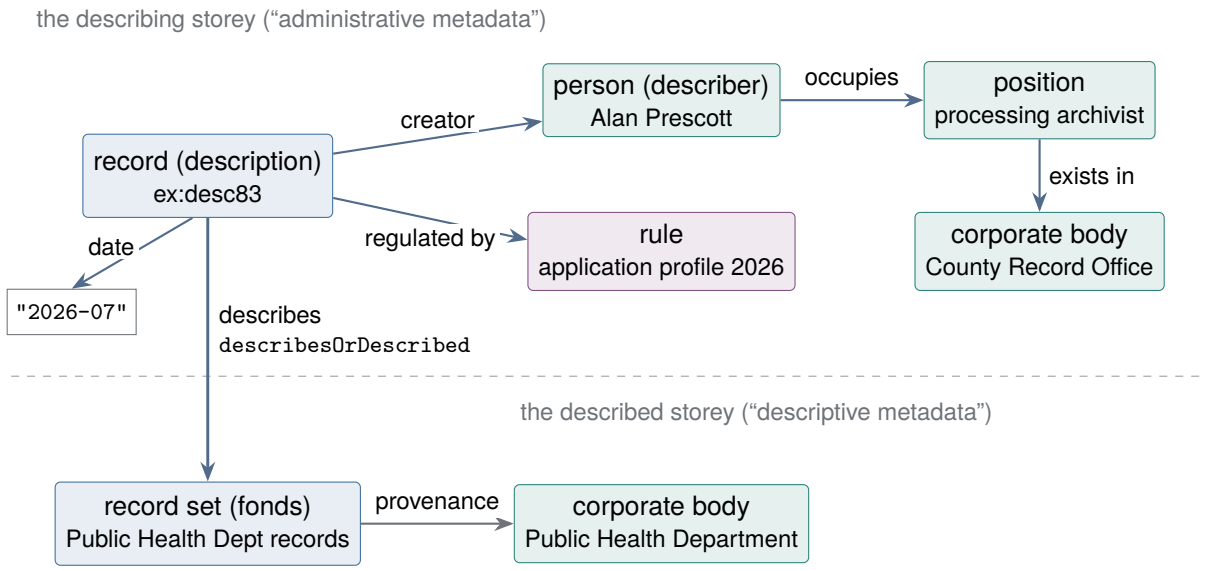


Figure 4.11: How description describes itself. The catalogue data becomes a record one storey up, with the describer, the rule and the date as its attributes and relations. The two storeys are joined by `rico:describesOrDescribed`, an ordinary relation. The describer, the position and the institution are of exactly the same classes as entities on the described storey.

the creator, date and governing rule per *named graph*, that is, per labelled subset of the store (Section 7.4.4). The granularity is coarse but the cost is low, and the two are not mutually exclusive. Starting with management information at graph level and refining just the parts that need it into description records is a realistic way to proceed.

What this chapter of the source amounts to is considerable. If the author, the basis and the revision history of a description survive in structured form, a catalogue stops being an unsigned authority and becomes a collection of statements that can be checked. Being able to see who described something, when, and on what evidence, is a practical answer to the ethical criticism in Chapter 3, that description has a point of view and involves choices. Attaching a source to every statement is expensive, though, and how far to go is once again a decision for the implementation.

Exercises (hints in Appendix A.3)

4.1 Explain the difference between a record *belonging to* a record set and a record *being a kind of* record resource, using the anthology analogy from this chapter.

4.2 Take something close at hand (submitting an essay for a course, say) and draw it with RiC-CM entities (record, agent, activity, date) and relations.

4.3 Name two things you can do once a body of catalogue data has been raised to an entity as a description record.

4.4 In Figure 4.11 the archivist and the historical figures belong to the same class, Person. What position does that express?

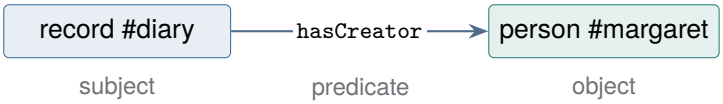
Chapter 5

RiC-O: The Ontology as Implementation

This chapter is about RiC-O. The first half is an introduction to what it presupposes, RDF and ontologies; the second half reads the design of RiC-O itself.

5.1 RDF: a world made of three-word sentences

RDF (Resource Description Framework) is a framework for representing data, standardised by the W3C. The idea is simple: express any information as a collection of three-part statements, subject, predicate and object, called triples.



Collect enough triples and you have a graph in which nodes (subjects and objects) are joined by arcs (predicates). The right-hand side of Figure 2.2 was a bundle of triples all along. RDF asks you to accept only three conventions.

1. **Things are named with IRIs.** An IRI (Internationalized Resource Identifier) is a name that is unique the world over. It looks like a web address, and there is a reason for that. Domain names (`ica.org`, `example.org`) are registered, and no two are alike, so a name beginning with a domain you control cannot collide with anyone else's name anywhere. An IRI borrows that *machinery of uniqueness* as a way of naming. `https://archives.example.org/diary` is "the name given to that record by the institution that holds example.org", and it will not clash with a name minted by any other institution. Note that an IRI is a name and nothing more: it promises no page that a browser can open. (Opening a RiC-O IRI does bring up documentation, but that is good practice, not a condition of the name working as a name.) Because the names are globally unique, data from different institutions can be joined by machine.
2. **Values can be written as literals.** In the object position, besides a reference to another thing (an IRI), you can put a **literal**: a raw value such as "field survey diary", "1868" or "2026-07-31", with nowhere further to go. You write "#diary — name → 'field survey diary' ". The difference from an IRI is whether it can be followed. An IRI points at the thing called by that name, and description of that thing may continue from there. A literal is the end of the road: read it as a string or a number and that is all. In a drawing of the graph, an IRI is a circle that arrows can leave, a literal a terminal box that can only receive them. Write the creator as an IRI and you can follow through to dates of birth and death; write it as a string and you have a name to read. This choice is the "quick way or the exact way" of Section 3.6, and it is design A against design B in Section 4.1.2.

3. **Predicates are IRIs too.** The relation of having been created also gets a globally unique name. Supplying the vocabulary of such predicates is what an ontology does, and it is the job of RiC-O.

A triple store is a stacked table

If spreadsheets or data analysis are familiar ground, triples can be understood as a change in *how a table is held*. Take marks in a school. The obvious shape is the one on the left, a row per pupil and a column per subject. The same information can be held as on the right.

Unstacked (wide form)

Name	Maths	History
Hart	40	88
Ward	98	70

Stacked (long form)

Name	Subject	Mark
Hart	Maths	40
Hart	History	88
Ward	Maths	98
Ward	History	70

Each row on the right is a triple: whose, what, how much. That is the whole of it. *A collection of triples is a stacked table whose three columns have been renamed subject, predicate and object.* The name is the subject, the school subject the predicate, the mark the object. A database that stores RDF and answers SPARQL queries over it is a **triplestore**, and that its insides really are a huge three-column table of this kind is shown in Section 6.8.

The advantages of the stacked form are the advantages of RDF. A new subject needs no new column, only new rows, which is why RDF can accept new predicates without a schema change. There are no empty cells: if Ward did not sit French, the row is simply absent, and no NULL appears. And different individuals can carry different sets of properties, so records can have record predicates and people can have person predicates, with no need to design columns common to everything. For archival description, where entities of many kinds sit side by side, that is a good fit.

The reverse direction matters too. The familiar wide table can be *generated* from the stacked one by pivoting. A conventional tabular list is one of the outputs made from a collection of triples, which is the smallest instance of the separation of input from output in Chapter 3. This way of holding data has a respectable history: wide and long form in data analysis, and the Entity–Attribute–Value pattern in database design.

RDF is a model rather than a syntax, and it can be written several ways. This book uses Turtle, which is the most readable. A fonds and one record inside it, in Turtle:

```
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX rst:  <https://www.ica.org/standards/RiC/vocabularies/recordSetTypes#>
PREFIX ex:   <https://archives.example.org/>

ex:fondsEllis  a  rico:RecordSet ;                  # "a" means "is an instance of"
    rico:hasRecordSetType  rst:Fonds ;            # type: fonds (a supplied individual)
    rico:name  "The Ellis family papers" ;
    rico:includesOrIncluded  ex:diary .           # contains the diary

ex:diary  a  rico:Record ;
    rico:name  "Field survey diary" ;
    rico:hasCreator  ex:margaret ;
    rico:hasOrHadDigitalInstantiation  ex:diaryScan .

ex:diaryScan  a  rico:Instantiation .              # the scanned image (TIFF)
```

```
ex:margaret  a  rico:Person ;
    rico:name  "Margaret Ellis" .
```

The record `ex:diary` is the diary as information content, and its scan appears as a separate individual, the instantiation. The distinction drawn in Chapter 3 has been carried straight into the shape of the data. Turtle has few rules, and the box below has them all. With nothing more than these you can read the examples in this book and most of the official RiC-O samples.

Turtle: enough of the syntax to read it

Statements and full stops. A Turtle statement is three words, subject, predicate and object, ended by a full stop. `ex:diary rico:hasCreator ex:margaret .` is one statement: the creator of the diary is Margaret Ellis. The full stop is the only thing that ends a statement; a line break does not.

Whitespace. Words are separated by any number of spaces, tabs or newlines; to the parser these are all one separator. So the following are the same single statement.

```
ex:diary rico:hasCreator ex:margaret .
ex:diary    rico:hasCreator    ex:margaret .
ex:diary
    rico:hasCreator
        ex:margaret .
```

Indentation and alignment are decoration for human readers. Unlike Python, indentation carries no meaning; unlike XML, no closing tag marks an extent. Extent is settled only by punctuation: full stop, semicolon, comma, square bracket. The predicates are aligned in the examples in this book for legibility, nothing more.

What you may *not* do is put a separator inside a word. Break the line after a prefix, as in `rico:  hasCreator`, and the parser sees two words where there should be one name, which is normally a syntax error. When a long IRI runs off the edge, do not wrap it; lengthen the line, or shorten the IRI with a prefix.

A full stop may be written tight against the preceding word, `ex:margaret.`, but the convention is a space, `ex:margaret .`, and it leads to fewer misreadings and fewer typing errors.

Namespaces. An IRI such as `https://www.ica.org/standards/RiC/ontology#Record` falls into two parts. The tail, `Record`, is the local name; the head, `https://www.ica.org/standards/RiC/ontology#`, is the **namespace**, saying which institution's vocabulary the name belongs to.

Namespaces exist to stop names colliding. The word `Record` means a record in archives, a row of a table in databases and a disc in music. With bare short names, one field's `Record` could not be told from another's. Put a namespace in front and "the `Record` of the ICA's RiC ontology" and "the `Record` of some other vocabulary" are different IRIs, safe to mix in one dataset. It is the way two people called Smith are told apart by the organisation each works for. Namespaces use domain names because, as Section 5.1 said, the registration system guarantees uniqueness.

The # at the end of a namespace. The RiC-O namespace ends `.../ontology#`, with a hash. In a web URL the hash separates off a **fragment**, a position within a document, and RDF has borrowed it as the boundary between the namespace and the local name. Expand `rico:Record` and you get `.../ontology#Record`: vocabulary before the hash, one term of the vocabulary after it. There is a reason for the choice. HTTP does not send anything after the fragment to the server, so fetching `.../ontology#Record` actually retrieves the single file `.../ontology`, and *the whole vocabulary arrives in one request*. For something small and updated as a unit, which is what an ontology is, that arrangement suits it well (see the W3C note *Cool URIs for the Semantic Web*).

The other convention uses `/` as the separator. An institution publishing records at `https:`

`//archives.example.org/records/1234` and the like is using that convention, and each name can be fetched separately. An institution publishing tens of thousands of records cannot hand the whole set to a reader who wants one, so `/` is the better choice there. Hash for vocabularies, slash for data: that is where the split comes from.

Prefixes. Namespaces are long and repeat endlessly within one file. So Turtle opens with `PREFIX` declarations that abbreviate them, in the way a dictionary's front matter declares that OED stands for the *Oxford English Dictionary*. After the declaration above, writing `rico:Record` means exactly what writing `<https://www.ica.org/standards/RiC/ontology#Record>` would mean. So `rico:` announces membership of the ICA's RiC ontology, and `ex:` is a namespace invented for the fictional institution in the examples of this book. Written out in full, an IRI goes in angle brackets <>. There are two forms of declaration: `PREFIX rico: <...>` (no full stop) and `@prefix rico: <...> .` (full stop required). The first was added in Turtle 1.1 to match SPARQL; the second is the older form. They mean the same thing and both are common in published data. Only the full stop differs, so watch for it when copying.

The predicate `a`. In a statement such as `ex:prov83 a rico:OrganicProvenanceRelation`, the `a` is itself a predicate, an abbreviation of `rdf:type` meaning "is an instance of the class" (from the English "is *a*"). So the statement reads: prov83 is an instance of the class organic provenance relation. Trying to read `a` like other predicates, as a relation between two things, involves a slight twist, because the object is not a thing but a *class*. Section 5.2 comes back to that twist when the notation is set against logic.

Semicolons and commas. To write several statements about one subject, a semicolon carries the same subject on to the next predicate. Where the predicate is the same as well and only the object differs, a comma lists the objects. These three are identical in meaning.

```
# (1) every statement written out
ex:f  rico:includesTransitive  ex:a .
ex:f  rico:includesTransitive  ex:b .
ex:f  rico:name  "Fonds F" .

# (2) sharing the subject with a semicolon
ex:f  rico:includesTransitive  ex:a ;
      rico:includesTransitive  ex:b ;
      rico:name  "Fonds F" .

# (3) sharing the predicate too, with a comma
ex:f  rico:includesTransitive  ex:a , ex:b ;
      rico:name  "Fonds F" .
```

Literals. Strings go in `"..."`. A language tag can be attached, as in `"registre paroissial"@fr`, and a datatype, as in `"1868"^^xsd:gYear`. Inside the quotation marks, spaces and line breaks are *significant characters*, which is the large difference from outside them. A raw line break cannot appear inside `"..."`, so text that contains one goes in triple quotes, `"""..."""`. A quotation mark in the value itself is written `\"`.

Blank nodes. Square brackets `[ ... ]` create an individual on the spot with no name (no IRI). What is inside the brackets is a list of predicates and objects taking that individual as subject.

```
ex:diary  rico:isAssociatedWithDate  [ a  rico:Date ;
                                       rico:expressedDate  "Michaelmas 1921" ] .
```

Read: the diary is associated with a date; that date is an instance of Date, expressed as "Michaelmas 1921". This is convenient for subsidiary individuals not worth a global name,

such as dates and quantities, and the official RiC-O samples use it freely. But a blank node has no IRI, so nothing outside can refer to it. In data meant to be kept and shared, the rule is that the significant entities get IRIs.
Comments. From `#` to the end of the line is a comment, ignored as data. It ends at the line break, so a comment over several lines needs a `#` on each.
Encoding and case. Files are written in UTF-8. Non-ASCII characters are legal in both IRIs and literals, but non-ASCII IRIs are handled inconsistently by some tools and publishing platforms, so the safe practice is to keep IRIs in ASCII and put the non-ASCII text in literals. IRIs and prefix names are case-sensitive (`rico:Record` and `rico:record` are different), and so are the keywords `a`, `true` and `false`, which must stay lowercase. The one exception is the `PREFIX` (and `BASE`) form of declaration, which may be written in either case.

5.1.1 RDF has more than one syntax: Turtle, JSON-LD, RDF/XML

RDF is an abstract data model, and the same set of triples can be written in several syntaxes (serialisations). The meaning is identical whichever is used, and tools convert mechanically between them. The main ones are these.

Turtle The format used above. The name stands for Terse RDF Triple Language. It is the most concise and the best suited to being read and written by people. The examples in this book, and most of the official RiC-O samples, are Turtle. For learning, illustration and writing by hand, use it.

JSON-LD RDF in the shape of **JSON** (JavaScript Object Notation). JSON writes pairs of key and value inside braces and allows the same shape to nest in the value position[1]. The LD is Linked Data. What separates it from ordinary JSON is a mechanism (the `@context` in the example below) that maps the names appearing as keys and values onto IRIs, so that the whole can be read as RDF triples. Since it can be handled with the standard tooling of web development (libraries in every language, web APIs), it is often the first choice when talking to developers or joining systems together. RiC-AG lists JSON-LD alongside RDF as a target format for conversion.

RDF/XML The oldest format. The RiC-O ontology itself (`RiC-O_1-1.rdf`) is distributed this way. It can be processed with XML tools, but it is verbose to read and there is little reason to choose it for something new.

N-Triples One triple per line, with no abbreviations. Verbose, but the structure is self-evident, which suits bulk exchange and line-by-line processing.

One look at the difference is enough. Part of `ex:diary` above, in JSON-LD:

```
{ "@context": { "rico": "https://www.ica.org/standards/RiC/ontology#" },
  "@id":   "https://archives.example.org/diary",
  "@type": "rico:Record",
  "rico:name": "Field survey diary" }
```

[1] It began as the way of writing data in JavaScript, the language of web browsers, and is now the common format in which web services exchange data, whatever the language. Its role is close to that of XML (Chapter 2); using braces and keys rather than paired start and end tags makes it shorter to write. It is present in archival practice too. ArchivesSpace, the management system for description following the American standard DACS, states that it represents records as JSONModel objects and that these are the unit of exchange with the system, and the name of an agent carries a `"rules": "dacs"` field naming the rules applied (API documentation, version 4.2.0). What is used there is JSON, not JSON-LD; DACS itself specifies no output format, and its text is maintained in Markdown.

So the question is not which format is correct but who, or what, is going to read it. Turtle for people, JSON-LD for talking to web applications, RDF/XML to sit on existing XML infrastructure, N-Triples for bulk transfer between machines. This book stays with Turtle.

5.1.2 Which way do the arrows point?

This book has been drawing relations as arrows since the first chapter. Stop for a moment over what the *direction* of an arrow means. The question looks like one about drawing conventions and turns out to be about data design. An arrow is, in substance, a **reference**: one description naming something else. Reference is the skeleton of data design (its history was traced in Section 2.4). Which side names the other; by what means (a string, a key, an IRI); and whether the reverse can be followed. Those three settle how well a system can search and how easily it can be maintained. Yet in diagrams the direction tends to be drawn by feel and read by feel. Figure 5.1 draws one fact, that Margaret Ellis created the diary, in three conventions.

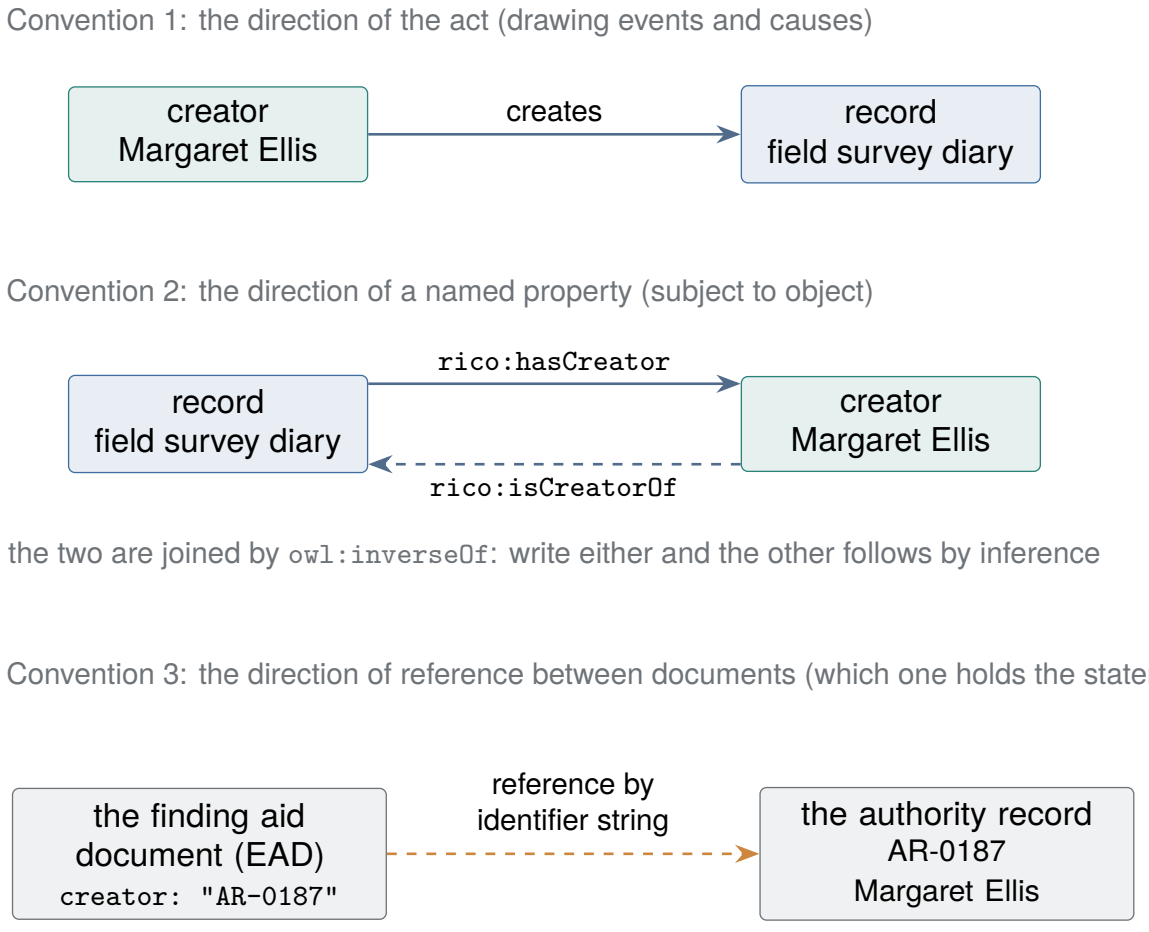


Figure 5.1: Three arrow conventions for one fact, that Margaret Ellis created the diary. Each direction is settled for a different reason, and two of them pointing opposite ways is no contradiction.

Convention 1: the direction of the act The creator *creates* the record. Conceptual diagrams explaining cause or sequence normally take this direction, and it agrees with intuition. But this arrow corresponds to no statement in the data. What is drawn is an event in the world, not a structure in the description.

Convention 2: the direction of a named property An RDF triple always runs from subject to object, so an arrow for a property is drawn in the direction *its name specifies*. `rico:hasCreator` says the record *has* a creator, so it runs record to creator, the opposite of convention 1. Remember that RiC-CM supplies almost every relation with *names in both directions* ("Record has creator Agent" and "Agent is creator of Record"). RiC-O implements this as 416 `owl:inverseOf` declarations, and whichever direction you write, the other follows by inference (Section 6.3). So there is no correct direction as such. What decides right from wrong is only whether *the label and the direction agree*. An arrow labelled "creates" should leave the creator; an arrow labelled `rico:hasCreator` should leave the record. Some relations have no meaningful direction at all: `rico:hasSibling`, `rico:hasOrHadSpouse` and 14 other RiC-O properties are declared *symmetric* (`owl:SymmetricProperty`) and mean the same written either way.

Convention 3: the direction of reference between documents When the identifier of an authority record is written into a finding aid, the arrow of reference runs finding aid to authority record. That is not the direction of the meaning of the relation but of *where the statement is kept*. In document-based description the writer does not choose this direction; the standard and the document structure fix it. Since that is a decisive difference between the older standards and RiC, Section 7.4 takes it up with a diagram.

What follows in practice One practical warning about convention 2. The inverse "follows by inference" only where inference is switched on. In a store without it, only the triples *actually written* turn up in a query. SPARQL can absorb this at the query end, since a `^` before a predicate follows it backwards, but the party receiving the data has to know that. Otherwise the result is a report that the creator cannot be found. The reliable course is to settle in the institution's application profile which direction a given relation is written in, or that both are (Chapter 7).

5.2 What an ontology is

The word ontology comes from metaphysics, but in computer science it means a vocabulary that defines the concepts (classes) and relations (properties) of a domain in a form a machine can process. The languages for writing an ontology on top of RDF are RDFS (RDF Schema) and OWL (Web Ontology Language); RiC-O is written in OWL 2.

An ontology defines four kinds of thing.

Classes Kinds of thing, answering to the entities of RiC-CM: `rico:Record`, `rico:Person`. Classes can be declared subclasses of one another (`rdfs:subClassOf`), so "`rico:Person` is a subclass of `rico:Agent`" carries the entity hierarchy of Figure 4.4 over unchanged.

Object properties Predicates joining things, answering to the relations of RiC-CM: `rico:hasCreator`, `rico:isOrWasIncludedIn`. Logical characteristics can be declared for them: domain and range, an inverse (`owl:inverseOf`), a superproperty (`rdfs:subPropertyOf`), transitivity.

Datatype properties Predicates joining a thing to a literal value, answering to the free-text attributes of RiC-CM: `rico:name`, `rico:scopeAndContent`.

Individuals Instances of classes. RiC-O ships seven named individuals of its own (below).

5.2.1 Underneath it all: description logic

So far, declarations such as "`rico:Person` is a subclass of `rico:Agent`" have been written in English. To have a machine reason with them, the English has to be replaced by a formal language that no two readers can interpret differently. That language is **description logic** (DL), and the semantics of

OWL rest on it. Each declaration written down this way is an **axiom**: not a fact about a particular piece of data but a commitment that holds wherever the vocabulary is used, and inference proceeds from these (Chapter 6).

What follows builds the notation up one piece at a time, for readers who have not met logical formulae before. The notation is actually needed only in a supplement to Chapter 6, so skipping ahead does no harm. The RiC-O specification does write its axioms this way, though, so being able to read it lets you go to the source directly.

Only three kinds of thing Description logic deals with three things and no more.

Individuals Named particular things: "Samuel Hart", "fonds 01". In the vocabulary of Chapter 4, entity instances.

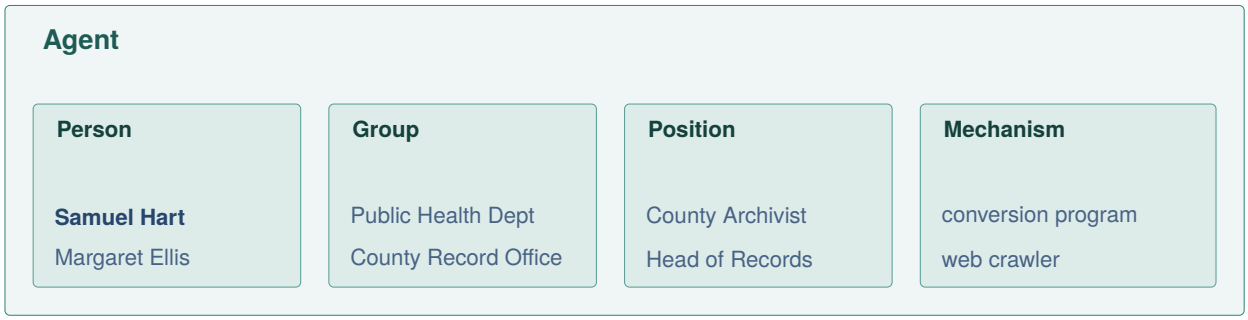
Classes *Sets* of individuals. The class `Person` is the set whose members are Samuel Hart, Margaret Ellis and the rest.

Properties *Sets* of pairs of individuals. The property `hasCreator` is the set of pairs such as (the diary, Margaret Ellis).

Seeing classes as sets is the key. "`rico:Person` is a subclass of `rico:Agent`" becomes, in the language of sets, "the set `Person` is wholly contained in the set `Agent`", and description logic writes it

$$\texttt{Person} \sqsubseteq \texttt{Agent}$$

where $\sqsubseteq$ answers to the mathematical $\subseteq$ and reads "the left is contained in the right". As Figure 5.2 shows, if the containment holds then anything that is a person is automatically an agent. The inference "Samuel Hart is a person, therefore Samuel Hart is an agent" is one step along that containment.



The four inner boxes all sit wholly inside Agent. So writing "Samuel Hart belongs to Person" makes "Samuel Hart belongs to Agent" hold without being written

Figure 5.2: Classes as sets. The subclass relation $\sqsubseteq$ is containment of sets. RiC-CM placing four entities under Agent (Figure 4.4) amounts, in the language of sets, to saying that `Person`, `Group`, `Position` and `Mechanism` are subsets of `Agent`.

Translating into first-order logic to check the meaning The symbols of description logic are terse, and the sure way to check what one means is to translate it into **first-order logic**. First-order logic is a notation for writing "for every" and "there is some" (**quantification**), together with "and", "or" and "if ... then", in a form that no two readers interpret differently. Description logic is a *fragment* of it: predicates are limited to one and two arguments, and quantification is confined to a few patterns, so that every axiom translates mechanically into a first-order formula. What follows reads the symbols one at a time with that translation and an actual RiC-O axiom alongside. Readers not at home with the notation should read the box first.

Preliminary: reading a formula

A **predicate** is a pattern for a claim about things, and comes with one argument or two. A one-place predicate such as $\texttt{Person}(x)$ expresses a property, "x is a person", and answers to a class. A two-place predicate such as $\texttt{hasCreator}(x, y)$ expresses a relation, "x has y as creator", and answers to a property. Put an actual name in the argument and $\texttt{Person}(\texttt{hart})$ becomes a claim with a truth value.
x and y are **variables**. Just as x in school algebra stood for a number not yet fixed, here it is a temporary name for an individual not yet fixed. The letter does not matter; rewriting x as u changes nothing.
A formula with free variables has no truth value on its own. "x is a person" is neither true nor false until we say who x is. **Quantifiers** fix the range.

- $\forall x\,(\ldots)$ — "for *every* x, …" (universal quantification)
- $\exists x\,(\ldots)$ — "there is *at least one* x such that …" (existential quantification)

Claims are joined by **connectives**. $\wedge$ is "and", $\vee$ is "or", $\rightarrow$ is "if … then", $\leftrightarrow$ is "if and only if", $=$ is "is the same thing as". Only $\rightarrow$ departs a little from ordinary speech. $A \rightarrow B$ says only that wherever A holds, B holds too; about the cases where A fails it says nothing.
That is enough to read formulae. It helps to read them aloud.

Formula	Reading
$\forall x\,(\texttt{Person}(x) \rightarrow \texttt{Agent}(x))$	for every x, if x is a person then x is an agent
$\exists y\,\texttt{hasCreator}(\texttt{diary}, y)$	there is at least one y that diary has as creator
$\forall x \forall y\,(\texttt{hasCreator}(x, y) \rightarrow \texttt{Agent}(y))$	for every x and y, if x has y as creator then y is an agent

The pattern of reading is fixed. A formula opening with $\forall$ says "every one of them…"; one opening with $\exists$ says "there is one such that…". Read what is left of $\rightarrow$ as the condition and what is right of it as the consequence.
"First-order" means that only *individuals* can be quantified, not the predicates themselves. "For every class C…" cannot be written. What that restriction buys is the completeness discussed next, and with it the half-guarantee that goes by the name of semi-decidability; full decidability comes only from restricting the syntax further, as description logic does.

The return on limiting expressive power is **decidability**: a guarantee that some procedure will produce an answer in finitely many steps for any input. Full first-order logic carries no such guarantee. There is no procedure that always decides whether one sentence follows from another (the unsolvability of the decision problem, shown independently by Church and by Turing in 1936). One side is guaranteed: where the conclusion does follow, generating proofs in order will eventually reach it and answer yes (this follows from Gödel's completeness theorem of 1930). What is not guaranteed is concluding that it does *not* follow; there the procedure may run on without stopping. A property that returns only one of the two answers reliably is called **semi-decidable**. Description logic restricts the syntax available and thereby recovers the state where both answers come in finitely many steps. Put in working terms, the possibility of a reasoner running for ever without an answer has been excluded at the level of the design of the system. The OWL 2 profiles seen in Section 6.8.4 strengthen that further, bounding the computation time as well.

The notation has one other feature that should be met in advance: *no variables appear* in a description logic formula. There is no x in $\texttt{Person} \sqsubseteq \texttt{Agent}$. That is design rather than abbreviation. Description logic grew out of work making the semantic networks and frames of the 1970s (the

knowledge representation of the expert-system era, mentioned in Chapter 3) rigorous as logic, and in doing so it took an algebraic notation that *operates on classes as sets* instead of putting individuals first. The notation pays off twice over: an axiom fits on one variable-free line, and expressive power and computational cost can be tuned finely by choosing which operators to admit[2]. The main symbols follow, each with its translation and a real RiC-O axiom. Read a class as a set of individuals and a property as a binary relation between individuals.

Subsumption, $\sqsubseteq$ The basic symbol, read "the left is contained in the right". A class axiom $C \sqsubseteq D$ translates to

$$\forall x\,(C(x) \rightarrow D(x))$$

"for every individual x, if C then D too". The RiC-O instance is `Person` $\sqsubseteq$ `Agent`. A property axiom $R \sqsubseteq S$ takes two variables,

$$\forall x \forall y\,(R(x,y) \rightarrow S(x,y))$$

with the instance `directlyIncludes` $\sqsubseteq$ `includesTransitive`: if it directly includes, it includes.

Inverse, $^-$, and equivalence, $\equiv$ The superscript $^-$ turns a relation round.

$$R^-(x,y) \;\leftrightarrow\; R(y,x)$$

Since `hasCreator` runs record to creator, `hasCreator`$^-$ runs creator to record. $\equiv$ says the two sides mean exactly the same, and is used to give a name to a relation's inverse. The RiC-O axiom `isCreatorOf` $\equiv$ `hasCreator`$^-$ translates to

$$\forall x \forall y\,(\texttt{isCreatorOf}(x,y) \leftrightarrow \texttt{hasCreator}(y,x)).$$

Composition, $\circ$ $R \circ S \sqsubseteq T$ reads "if you can go by R and then by S, you can go by T in one step", translating to

$$\forall x \forall y \forall z\,(R(x,y) \wedge S(y,z) \rightarrow T(x,z)).$$

Transitivity is the special case $R \circ R \sqsubseteq R$, and translating the RiC-O instance gives

$$\texttt{includesTransitive}(x,y) \wedge \texttt{includesTransitive}(y,z) \rightarrow \texttt{includesTransitive}(x,z)$$

what is inside something included is itself included.

Building classes: $\exists R.C$, $\forall R.C$, $\sqcup$ Here description logic starts to look like itself. These symbols are not statements but *operations that build a new class*, that is, a new set of individuals. They let you write not only classes with names given in advance, such as `Person` and `Agent`, but sets *defined on the spot*, such as "things having a person as creator".

$\exists R.C$ denotes all individuals having at least one R-partner belonging to C. So

$$\exists \texttt{hasCreator}.\texttt{Person}$$

is the class of things having a person as creator. $\forall R.C$ denotes all individuals whose R-partners, if any, all belong to C, and $\sqcup$ is union ("or"). All three become clear when the condition for an individual x to belong is written in first-order logic.

[2]The initials of the operators admitted give the systems their names, $\mathcal{ALC}$, $\mathcal{SROIQ}$ and the rest. The semantics of OWL 2 DL rests on $\mathcal{SROIQ}$.

$$x \in \exists R.C \iff \exists y\,(R(x,y) \land C(y))$$
$$x \in \forall R.C \iff \forall y\,(R(x,y) \to C(y))$$
$$x \in C \sqcup D \iff C(x) \lor D(x)$$

The $\exists$ and $\forall$ of the symbols reappear in the translation as quantifiers over the partner y. That is the join between the variable-free notation of description logic and the explicit variables of first-order logic.

$\forall R.C$ needs one caution in reading. The right-hand side of its translation has the form of $\to$, so it is true of an individual with *no* R-partner at all: where the condition never holds, the claim that all such partners are C cannot be broken. So $\forall$`hasCreator.Agent` does not mean "has one or more creators, all of them agents". It means "anything recorded as a creator is an agent". To say that at least one exists, combine it with $\exists$.

$\top$, the class of all individuals $\top$ (read "top") is *the class every individual belongs to unconditionally*: the class whose membership condition is always true. That sounds vacuous, but in a logic without variables $\top$ does duty in two set phrases.

The first is $\exists R.\top$, "*the partner can be anything*". Putting the class that everything belongs to in the C position of $\exists R.C$ builds the class of individuals that have an R-partner of any kind whatever. $\exists$`hasCreator`.$\top$ is "things that have a creator, whoever it may be".

The second is $\top \sqsubseteq X$, "X *holds of every individual*". Since $\sqsubseteq$ means the left is contained in the right, the formula says the class of all individuals is contained in X, that is, every individual is in X. It is the set phrase for saying, without variables, what first-order logic says with $\forall x\,(\ldots)$.

With those two, domain and range axioms can be written. The RiC-O domain axiom

$$\exists \texttt{hasCreator}.\top \sqsubseteq \texttt{RecordResource} \sqcup \texttt{Instantiation}$$

reads "an individual that has a creator, whoever it may be, is a record resource or an instantiation", and in first-order logic

$$\forall x\,(\exists y\,\texttt{hasCreator}(x,y) \to \texttt{RecordResource}(x) \lor \texttt{Instantiation}(x)).$$

The range axiom $\top \sqsubseteq \forall$`hasCreator.Agent` is read from the outside in with the second set phrase and from the inside out with $\forall$: for every individual ($\top \sqsubseteq$), its creators, if any, are all agents ($\forall$`hasCreator.Agent`). The translation is

$$\forall x \forall y\,(\texttt{hasCreator}(x,y) \to \texttt{Agent}(y))$$

whatever is on the creator side is an agent. The notation looks cryptic, but $\exists R.\top \sqsubseteq C$ (the domain is C) and $\top \sqsubseteq \forall R.C$ (the range is C) can be memorised whole, as idioms.

Counting: $=1$ $=1\,R.C$ denotes all individuals having *exactly one* R-partner belonging to C. The RiC-O instance `Proxy` $\sqsubseteq =1$ `proxyFor.RecordResource` says a proxy stands for exactly one record resource. Translating it needs a formula with equality: at least one exists, and any two are in fact the same.

$$\begin{aligned}\forall x\,(\texttt{Proxy}(x) \to \exists y\,(&\texttt{proxyFor}(x,y) \land \texttt{RecordResource}(y)) \\ \land\, \forall y \forall z\,(&\texttt{proxyFor}(x,y) \land \texttt{RecordResource}(y) \\ &\land \texttt{proxyFor}(x,z) \land \texttt{RecordResource}(z) \to y = z))\end{aligned}$$

Note that the `RecordResource` condition is needed in the uniqueness half as well. Drop it and the claim becomes a different and stronger one, that there is only one `proxyFor` partner of any kind at

all. Axioms that limit a count in this way are *cardinality restrictions* generally; the form that limits what is counted by a class is a *qualified number restriction*, and it is what the $\mathcal{Q}$ of $\mathcal{SROIQ}$ names. Using equality makes it more expressive, and that bears on the complexity of the system.

The $\sqsubseteq$ notation is good for listing axioms compactly without variables; translation into first-order logic, the shape of a rule, is good for checking what one means. Both are used below as the occasion suits. Table 5.1 collects the symbols so far.

Table 5.1: Description logic symbols at a glance (C and D are classes, R and S properties, x, y, z variables over individuals)

Symbol	Reading	Translation into first-order logic
$C \sqsubseteq D$	C is contained in D	$\forall x\,(C(x) \rightarrow D(x))$
$R \sqsubseteq S$	R is contained in S	$\forall x \forall y\,(R(x,y) \rightarrow S(x,y))$
$C \equiv D$	C and D are the same	$\forall x\,(C(x) \leftrightarrow D(x))$
R^-	the inverse of R	$R^-(x,y) \leftrightarrow R(y,x)$
$R \circ S \sqsubseteq T$	R then S implies T	$\forall x \forall y \forall z\,(R(x,y) \wedge S(y,z) \rightarrow T(x,z))$
$\exists R.C$	has at least one partner in C	$\exists y\,(R(x,y) \wedge C(y))$
$\forall R.C$	partners, if any, are all in C	$\forall y\,(R(x,y) \rightarrow C(y))$
$C \sqcup D$	C or D	$C(x) \vee D(x)$
$C \sqcap D$	C and D	$C(x) \wedge D(x)$
$\top$	the class of all individuals	always true
$=1\,R.C$	has exactly one partner in C	one in C exists, and any two in C are equal

How this relates to RDF triples The correspondence with the triples of Section 5.1 is exact if stated in two stages. First, *triples of data correspond to atomic formulae*, statements of fact that cannot be broken down further.

Turtle	Description logic	First-order logic
`ex:diary rico:hasCreator ex:margaret .` `ex:margaret a rico:Person .`	$\texttt{hasCreator}(\texttt{diary},\texttt{margaret})$ $\texttt{Person}(\texttt{margaret})$	$\texttt{hasCreator}(\texttt{diary},\texttt{margaret})$ $\texttt{Person}(\texttt{margaret})$
`rico:Person rdfs:subClassOf rico:Agent .`	$\texttt{Person} \sqsubseteq \texttt{Agent}$	$\forall x\,(\texttt{Person}(x) \rightarrow \texttt{Agent}(x))$

The top two rows are *facts*. An ordinary triple like the first becomes a two-place atomic formula without fuss. The second, the `a` triple, contains a *twist* that should not be passed over. Treated like the first, it would become $\texttt{a}(\texttt{margaret},\texttt{Person})$, that is $\texttt{type}(\texttt{margaret},\texttt{Person})$, a two-place atomic formula. The table instead reads it as $\texttt{Person}(\texttt{margaret})$, an application of a one-place predicate. `Person` stands differently in the two readings: as an *argument*, the name of a thing, in the first; in the *predicate* position in the second. So `a` alone is asymmetric with the other predicates, in that the *name* of a predicate occupies the object position. The standard logical reading is the second, and on that reading the two notations are *isomorphic at the level of fact* (some description logic literature writes $\texttt{margaret} : \texttt{Person}$). The first reading, `type` as a two-place predicate, is not wrong either, and it is the subject of the box below. Only at the third row, an *axiom*, do the three columns take on different faces: first-order logic shows the quantified variables, description logic folds them into a variable-free line, and Turtle encodes that as one triple. So what is distinctive about description logic appears at the level of axioms, and an RDF graph at the level of fact is nothing but a set of atomic formulae. Description logic calls that set the **ABox** (assertion box).

The axioms are triples too. In the RiC-O file these axioms are themselves written as RDF triples; an ontology is a kind of RDF data. Some of the axioms read above, in Turtle (the RiC-O file itself is the same content as RDF/XML):

```
# subclass: a person is a kind of agent
rico:Person  rdfs:subClassOf  rico:Agent .

# subproperty: if it directly includes, it includes
rico:directlyIncludes  rdfs:subPropertyOf  rico:includesTransitive .

# inverse: "is creator of" is the inverse of "has creator"
rico:isCreatorOf  owl:inverseOf  rico:hasCreator .

# transitivity (the same as the composition axiom)
rico:includesTransitive  a  owl:TransitiveProperty .

# a shortcut over a chain of three steps (the brackets list the path)
rico:hasOrganicProvenance  owl:propertyChainAxiom
    ( rico:thingIsSourceOfRelation
      rico:organicProvenanceRelation_role
      rico:relationHasTarget ) .
```

The symbols and the Turtle are two notations for the same thing: the symbols for thinking with, the Turtle (or RDF/XML) for handing to a machine. The set of axioms is called the **TBox** (terminology box).

When a class becomes a subject: two ways to read it

Look hard at the listing above and something odd appears. Read `rico:Person rdfs:subClassOf rico:Agent` straightforwardly as a predicate and you get $\texttt{subClassOf}(\texttt{Person}, \texttt{Agent})$, in which `Person` occupies a *term* position, the position of a name of a thing. But `Person` was supposed to be a *predicate*, used as $\texttt{Person}(x)$. Making a predicate into a term, so that it can be quantified over or asserted about, looks like a step outside first-order logic into second-order. The twist noticed a moment ago in the `a` triple has become general. There are two standard answers.

Reading 1: the triple merely encodes an axiom (the OWL 2 DL position). On this view a `subClassOf` triple is not a formula but *syntax* for carrying an axiom. On loading, the reasoner translates it into the first-order sentence $\forall x\,(\texttt{Person}(x) \rightarrow \texttt{Agent}(x))$. The `Person` in term position is merely *quoting the name* of a predicate, and no quantification over classes occurs anywhere. So it stays inside first-order logic.

Reading 2: treat everything as a thing (the RDF semantics and rule-engine position). The other direction works too. Regard classes as individuals of a kind, and write membership with an ordinary two-place predicate, `type`, which is what `a` really is. Then $\texttt{subClassOf}(C, D)$ is a first-order atomic formula as it stands, and adding this bridging rule as an axiom yields the same inferences as reading 1:

$$\texttt{type}(x, C) \wedge \texttt{subClassOf}(C, D) \rightarrow \texttt{type}(x, D)$$

The quantified variables C and D here range over *individuals* that happen to be called classes, not over predicates, so this too stays first-order. Triplestore implementations really do work this way. They treat the whole dataset as one huge three-place predicate $\texttt{triple}(s, p, o)$ and run rules of this kind (the form of the rule language known in databases as Datalog) to accumulate derived triples. That is what is going on inside the process described in Chapter 6 as "the reasoner adds the dashed triples". On this reading, taking `a` as the two-place predicate `type` is also justified: $\texttt{Person}(\texttt{margaret})$ and $\texttt{type}(\texttt{margaret}, \texttt{Person})$ are two notations for one fact, and the bridging rule carries you between them.

Neither reading, then, leaves first-order logic. And using one name as both a class and an individual is sanctioned practice in OWL 2, under the name *punning*. An instance has already

appeared in this book: `rst:Fonds` is an *individual*, the value of `rico:hasRecordSetType`, and at the same time a SKOS concept (SKOS itself is explained in Section 7.5). Switch roles according to context and read on.

Inference (Chapter 6) is nothing but applying the TBox (axioms) to the ABox (facts) to derive new atomic formulae, which is to say new triples. The simplest form is the syllogism: `Person` $\sqsubseteq$ `Agent`, and Samuel Hart is a `Person`, therefore Samuel Hart is an `Agent`. Each scenario in Chapter 6 comes with the axioms it uses, written in this notation.

An ontology on the map of relational databases

For anyone at home with relational databases, the following correspondence gives the shape of it. A class is roughly a table definition, an individual a row, a datatype property a column, an object property a foreign key (a join), an IRI a primary key unique across the world. Three differences are decisive.

A row in a relational database belongs to exactly one table; an individual in RDF can belong to several classes at once (the official samples contain an individual that is both a `rico:Instantiation` and a `premis:Representation`).

A relational schema is a constraint, and data violating it is rejected; an OWL axiom is a warrant for inference. "The range of `rico:hasCreator` is `rico:Agent`" means that whatever is written as a creator *may be inferred* to be an agent. It is not an input check.

A relational database works under the closed world assumption (what is not written is false); OWL is interpreted under the open world assumption (what is not written is *unknown*). The absence of a creator triple does not mean there was no creator. That difference bears directly on how query results are to be read (Chapter 6).

5.3 Why these all sit together: the layers of RDF, OWL and SKOS

RDF, RDFS and OWL have appeared so far, and SKOS and Dublin Core arrive in later chapters. With so many similar-looking acronyms about, it helps to set out how they relate and why they can be mixed in one body of data. The short answer is that they are *one set of building blocks arranged in layers*, and that the only convention holding them together is that everything is written as triples (Figure 5.3).

One base: RDF The convention at the bottom is naming with IRIs (Section 5.1). RDF above it is the *data model*: write data as triples of subject, predicate and object. What matters is that RDF has almost no terms of its own (`rdf:type` is one of the few exceptions). RDF does not settle what to say, only how to say it. It is syntax and nothing else.

Vocabularies are all equal Above that sits the layer of **vocabularies**. A vocabulary, reduced to essentials, is no more than *a set of IRIs with agreed meanings*. `rico:`, `skos:`, `dcterms:` (Dublin Core), and `rdfs:` and `owl:` themselves, are all vocabularies in that sense and all of equal standing. Any vocabulary's IRI may appear in the predicate or the type position. A triplestore does not care which vocabulary a predicate belongs to, so `rico:`, `skos:` and a locally minted `myo:` can live in one dataset (the example in Section 7.5 does exactly that). It is the way legal and medical terms can be mixed in one English sentence: the grammar does not object to a mixture of terminologies.

Only RDFS and OWL are special One pair of vocabularies has a special role. `rdfs:` and `owl:` are "vocabulary for *defining* vocabularies" (`subClassOf`, `inverseOf` and the rest), and they carry a

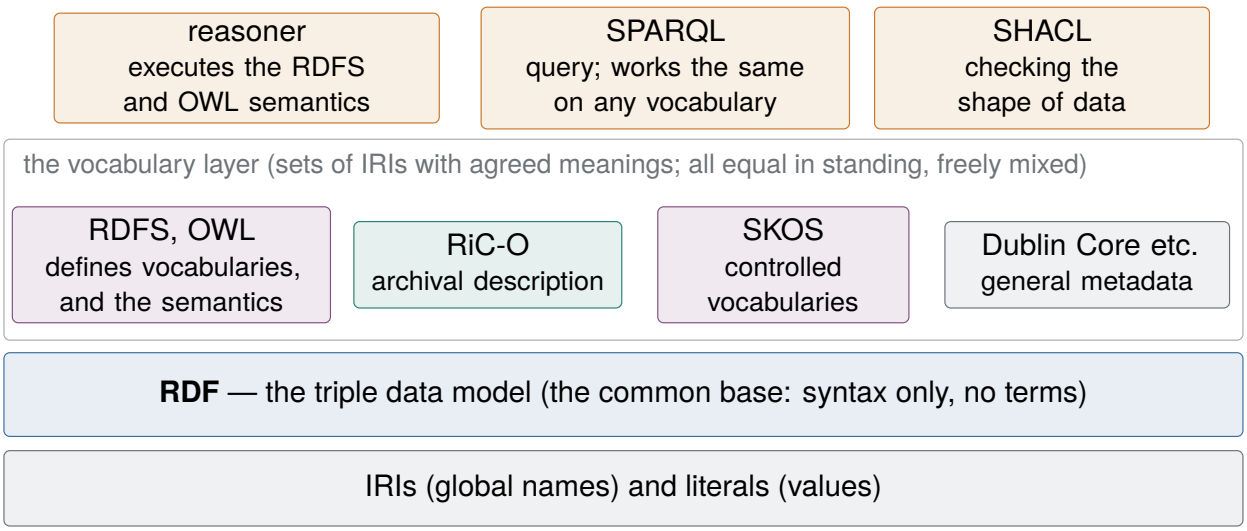


Figure 5.3: The layers. There is one base, RDF, and every vocabulary sitting on it has the same standing. RDFS and OWL alone have the special role of defining vocabularies and supplying the semantics for inference; SPARQL and SHACL act identically on triples from any vocabulary.

mathematical *semantics*, a specification of what follows from triples of a given shape. A reasoner is an execution engine for that semantics (Chapter 6). RiC-O, being a vocabulary defined in OWL, inherits the OWL semantics entire. SKOS goes the other way and keeps its semantics deliberately light (almost nothing follows logically from `skos:broader`; see the box in Section 7.5). OWL and SKOS are not on different layers, then. They differ in the *weight of meaning* carried within the same layer.

The tools work the same on everything The top row is languages and tools. SPARQL is a query language, SHACL a validation language, and neither is a vocabulary. They act identically on triples from any vocabulary. Querying SKOS data with SPARQL, or validating RiC-O data with SHACL, needs no additional machinery of any kind.

Who interprets the meaning The uniformity of syntax is as described. Who interprets the *meaning* of `skos:broader` or `rico:hasCreator` needs saying separately. There is no dedicated engine per vocabulary. What a reasoner knows built in is the semantics of RDFS and OWL and nothing else; the meaning of RiC-O or SKOS is written *on the data side*, in the vocabulary's own definition file, as a set of OWL axioms. Load that definition alongside the data and executing the OWL semantics yields the consequences of each vocabulary. Meanings that formal semantics does not carry (which label to show on screen, whether a convention has been observed) are the business of the application and of SHACL, and reading the prose specification is the business of a person.

Put in programming terms: a vocabulary is a library

For anyone who has written code, this correspondence clears the view.

The RDF world	The programming world
IRI	a namespaced identifier
RDF (triples)	the syntax and data model of the language
a vocabulary (`rico:`, `skos:`)	a library
the axioms of a vocabulary (an OWL file)	the implementation of the library
a reasoner	the language runtime
the prose specification	the documentation

Using NumPy does not call for a NumPy interpreter. The behaviour is written in NumPy's own code, and the runtime merely executes it under the semantics of the language. For the same reason, there is no such thing as a RiC-O reasoner. SKOS is the library whose implementation is nearly empty: names and conventions, little else. The analogy has a limit, in that code is procedural where axioms are declarative (see the box on axioms and procedures in Chapter 6). The arrangement holds all the same: the meaning ships with the thing being used, and one general-purpose runtime suffices.

Seeing the layers makes "using RiC-O" a precise statement. Choose RDF as the data model; use RiC-O, and the OWL semantics it inherits, as the principal vocabulary; mix in terms from SKOS or Dublin Core where they are wanted. That is all it comes to. The standards divide the work between them.

5.4 RiC-O in outline

Some figures for RiC-O 1.1, from the official documentation. The namespace IRI is `https://www.ica.org/standards/RiC/ontology#` and the recommended prefix is `rico:`.

- Classes: 107, against 19 entities in RiC-CM. Where the difference comes from is the subject of the next section.
- Datatype properties: 75.
- Object properties: 480, including those answering to the 85 relations of RiC-CM.
- Named individuals: 7. Fonds, Series, File and Collection (record set types, defined from the ISAD(G) glossary), and FindingAid, AuthorityRecord and IIIFManifest (documentary form types).

The naming of properties follows a rule. A relation covering both the present and the past is named *hasOrHad…* or *isOrWas…* (`rico:hasOrHadMember` joins a group to someone who is or was a member of it). Archival description is historical description by nature, and this is the device for handling relations that once held. Every class and property carries labels in English, French and Spanish, with German labels added to the classes in 1.1, an English definition, and, where a counterpart exists, a note of the RiC-CM component it answers to. With the CSV lists alongside it works as a dictionary.

Five design principles are stated (RiC-O documentation). It is a domain and reference ontology: "at this stage" it does not borrow from other existing ontologies (IFLA-LRM, CIDOC-CRM, PREMIS, PROV-O), which should make it easier to understand, implement and maintain, with alignment left to a future revision cycle "where possible". It is "immediately usable": conversion from existing metadata loses no information. It is a flexible framework: description may be coarse or fine. It opens new potential for description, through SPARQL and inference. And it is extensible, by

adding subclasses and subproperties and by connecting to SKOS, the standard for expressing controlled vocabularies in RDF (Section 7.5). Note in particular that "immediately usable" has meant deliberately keeping *several ways of expressing the same information*. The gain is easy migration; the cost is that keeping the data consistent falls to the implementation, which connects back to the trade-offs of Chapter 3.

5.5 From RiC-CM to RiC-O

5.5.1 Entities become classes

Each entity of RiC-CM has a class of the same name, one to one, and the hierarchy is carried across. Nothing surprising here.

5.5.2 Attributes become datatype properties, or classes

Free-text attributes (general description, history, scope and content) become datatype properties. Attributes taking controlled values (carrier type, record set type, activity type) are promoted to *classes*: `rico:CarrierType`, `rico:RecordSetType` and the rest, all subclasses of `rico:Type`. Names and identifiers are raised to entities too, as subclasses of `rico:Appellation`. This is RiC-CM §3.3 carrying out what it announced: in implementation, treat an attribute as an entity.

The "usable at once" principle shows its face here. Nearly everything that has been made a class also has a *shortcut that stays with a literal*. Instead of writing a type as a reference to an instance of a class, you may write a string with the `rico:type` property; instead of making a `rico:Date` entity, you may write a string with `rico:date` (and its many subproperties); instead of making a `rico:Name` instance, you may write `rico:name`. These shortcut properties carry a note (`skos:scopeNote`) saying that they are to be used only where the corresponding class is not, and that they may be withdrawn in future. The compromise is labelled as one.

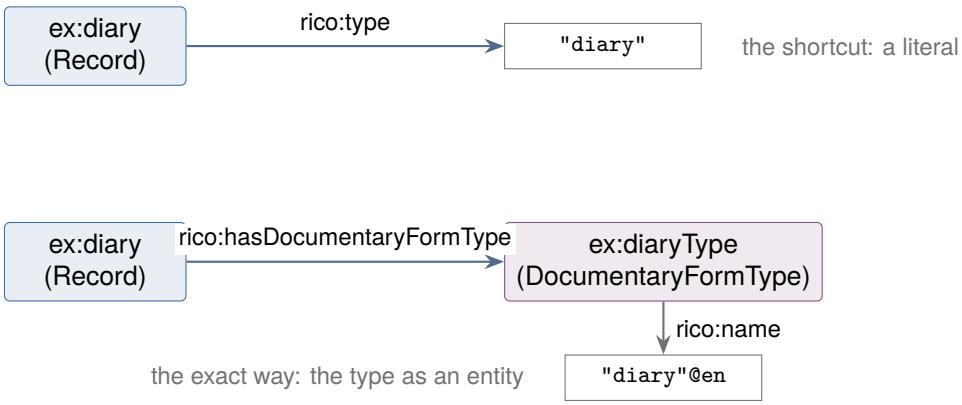


Figure 5.4: Two ways of writing the same information: the shortcut above, the exact way below. Written the exact way, the type itself can be developed as a SKOS vocabulary, given labels in several languages and broader and narrower relations, and shared between institutions.

5.5.3 Relations become object properties, and relation classes

RiC-CM relations are implemented twice over.

There is a *direct object property* for each relation. For R026, has provenance, it is `rico:hasOrganicProvenance` (the mismatch of names was noted at the end of Section 4.4). One triple does it, which is simple, but the relation itself can then carry no attributes of its own (date, certainty,

source; RiC-CM §5.5). An RDF triple is complete in three words, and a triple about a triple cannot be written directly.

And there is a family of relation classes, the relation *reified* as a class (`rico:Relation` at the head, with `rico:CreationRelation`, `rico:OrganicProvenanceRelation` and 46 others below). Make the relation a node and dates and certainty can hang off that node. It also handles the case where several parties are involved in one relation (an n-ary relation).

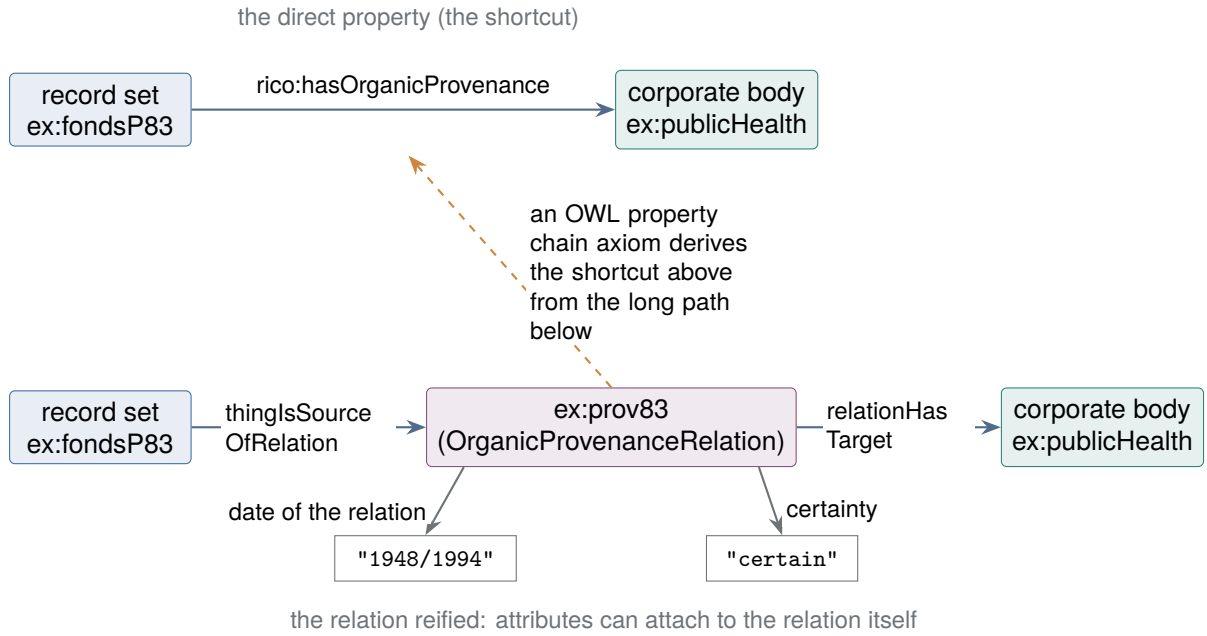


Figure 5.5: Two expressions of the same provenance relation. In RiC-O many direct properties are formally defined as shortcuts over a long path through a reified relation, so writing the long path yields the shortcut by inference (Chapter 6).

Which to write is again a decision for the implementation. A workable rule is to reify when there is something to say about the relation (a date, a basis) and to use the direct property when there is not. What matters is that the two are not unrelated duplicates: they are formally joined by an OWL 2 property chain axiom. `rico:hasOrganicProvenance` is defined as the shortcut over the path `rico:thingIsSourceOfRelation` → `rico:organicProvenanceRelation_role` → `rico:relationHasTarget`, so where a reasoner is enabled, writing the long path yields the shortcut automatically.

5.5.4 What RiC-O has and RiC-CM does not: Proxy

The main class with no counterpart in RiC-CM is `rico:Proxy`, defined as something that stands for a record resource *as it appears within one particular other record resource*. Its use is to let a record carry different information in different contexts.

Recall that one record can belong to several record sets. Suppose a letter is part of the Ellis family papers and also belongs to a collection on a coastal flood, assembled by a researcher. Its running number and position within the family papers, and its position within the collection, are two different pieces of information. Give the ordering to the record itself and the two contexts collide. So a proxy is raised as a stand-in per context, and the ordering and other context-specific information attach *to the proxy*.

Order itself is expressed with the sequence properties. At their head stand `rico:precedesOrPreceded` and its inverse. Below them sit a property for immediate succession, one for the transitive case, and variants for proxies. RiC-O 1.1 added `rico:rankInSequence`,

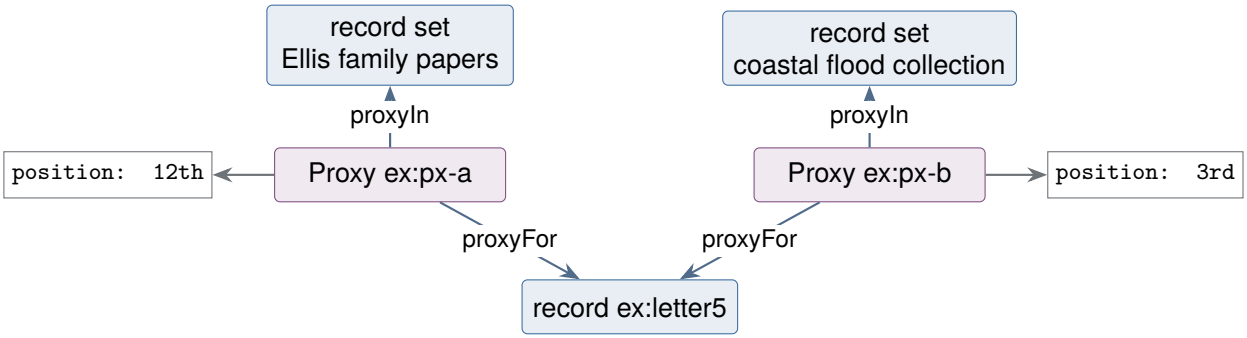


Figure 5.6: How Proxy works. One record has a separate stand-in per context, that is, per record set it belongs to, and context-specific information such as position in the sequence is recorded on the stand-in. Original order (Chapter 3) can be preserved without colliding with the order imposed by another context.

carrying a position as a number, with a note that it is to be used only alongside the sequence properties. Anyone who has worked with EAD can read Proxy as the device for carrying the document order of EAD components into a graph. A concept of the same name exists in another standard, OAI-ORE, and the idea is borrowed from there.

5.6 Reading a real example

Here is a simplified version of one of the official samples, a digital preservation package from docuteam in Switzerland. It shows the PREMIS vocabulary in use alongside RiC-O, as digital preservation work requires.

Before reading, a word about the odd-looking names. The <_20201118115821577> and <700> below are *IRIs, not literals* (as the syntax box said: quotation marks mean a literal, angle brackets an IRI). Unlike the earlier examples they use no prefix, and are written as *relative IRIs*, names relative to a base IRI that are completed when the data is loaded. The content of the name is an internal identifier the system generated mechanically from a timestamp (18 November 2020, 11:58:21.577), and it means nothing in itself. It is a good illustration of what an IRI is: as long as it is unique, a name may be a meaningless string of characters. They are long, so they are abbreviated below as <_2020...577>.

```
PREFIX rico:   <https://www.ica.org/standards/RiC/ontology#>
PREFIX premis: <http://www.loc.gov/premis/rdf/v3/>

<_2020...577>  a  rico:Record ;
    rico:title  "Logo docuteam" ;
    rico:scopeAndContent  "Enthaelt eine Logo-Datei" ;
    rico:isOrWasRegulatedBy  <700> ;              # rules such as access restriction
    rico:thingIsSourceOfRelation  [               # the creation relation, reified
        a  rico:CreationRelation ;
        rico:type  "archivist" ;
        rico:relationHasTarget  <101> ] ;
    rico:hasOrHadDigitalInstantiation  <_2020...159> .

<_2020...159>  a  rico:Instantiation, premis:Representation .

<700>  a  rico:Rule ;
    rico:type  "accessRestrictionsClosureYear" ;
    rico:endDate  "2025" .
```

```
<101>  a  rico:Person ;
    rico:name  "Tobias Wildi" .
```

Notice three things. The same individual belongs to `rico:Instantiation` and also to `premis:Representation`: RiC-O does not handle the details of the physical management and preservation of an instantiation, and connects to PREMIS instead, and that design (RiC-CM §1.2, and the RiC-O design principles) shows up directly in the data. The access restriction is expressed as a `rico:Rule` entity: what was a text element, "conditions governing access", in ISAD(G) has become an entity in its own right with an end date, in a form a machine can act on (has 2025 passed?). And the creation relation is reified, with the kind of involvement ("archivist") attached to the relation. Everything set out in this chapter is in use in twenty lines.

Exercises (hints in Appendix A.3)

5.1 Read the following Turtle aloud in English.

```
ex:s2  a  rico:RecordSet ;
    rico:name  "Communicable disease statistics" ;
    rico:isOrWasIncludedIn  ex:fondsP83 .
```

5.2 Rewrite exercise 5.1 as three independent statements, without semicolons or commas.

5.3 What relation is $\texttt{hasCreator}^{-}$? Write it in the form of a rule, using $\rightarrow$.

5.4 Take one statement from this chapter in which a class is the subject, and explain how punning lets it be read.

Chapter 6

Inference: What a Graph Makes Possible

The RiC-O documentation names three things it supports: publishing RDF datasets as linked data, querying them with SPARQL, and making inferences using the logic of the ontology. This chapter is about the third: it explains what inference is and then works through, one at a time, the situations in archival description where it actually earns its keep.

6.1 What inference is

Inference is a machine deriving, from the triples explicitly written and the axioms (definitions) of the ontology, *triples that were not written but follow logically*.

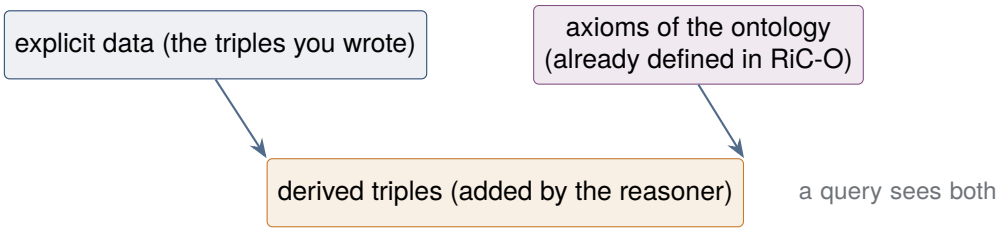


Figure 6.1: How inference works. The describer writes the minimum and the reasoner supplies the rest.

The axioms that inference works from are built into RiC-O already. Five kinds do most of the work, and all appeared in Chapter 5: subclass (`subClassOf`), subproperty (`subPropertyOf`), inverse (`inverseOf`; RiC-O carries 416 inverse declarations), transitivity (`TransitiveProperty`) and property chains (`propertyChainAxiom`, 121 of them). RiC-O is thus a list of terms and a set of inference rules at the same time. To put the rules to work, load the data into a triplestore with a reasoner (GraphDB, Fuseki; Chapter 7) and turn inference on.

Expressions in SPARQL, the query language for RDF, appear throughout this chapter as the way of checking what inference produced. This much is enough to read them at first sight.

The minimum SPARQL needed here

The basic form is `SELECT variables WHERE { patterns }`, and three things make it readable.
A word beginning with `?`, such as `?r`, is a *variable*, a hole not yet filled. The variables listed after `SELECT` are what you want returned.
What goes inside the braces after `WHERE` is a list of subject-predicate-object patterns written exactly as in Turtle, with the same rules for full stops and semicolons. The one difference is that a variable may stand in any position. So `?r a rico:Record` is the condition "`?r` is a

record".
Where the same variable appears in several patterns, all of them refer to *the same thing*. The engine looks through the graph for assignments to the variables that satisfy every pattern at once, and returns the list. That is all. Real queries begin with `PREFIX` declarations as in Turtle; the examples here omit them to save space.
Those three points cover every example in this chapter. The further machinery (`FILTER` for narrowing, `FILTER NOT EXISTS` for checking that something is absent, `CONSTRUCT` for building new triples out of results) is explained where it appears.

What follows takes the uses of inference one situation at a time.

6.2 Situation 1: searching across the hierarchy, by transitivity

Suppose an arrangement four levels deep, fonds to series to file to record, described only with "directly includes" between adjacent levels.

```
ex:fonds01   rico:directlyIncludes  ex:series02 .
ex:series02  rico:directlyIncludes  ex:file07 .
ex:file07    rico:directlyIncludes  ex:record31 .
```

What a user usually wants is every record contained in fonds 01, directly or otherwise. In RiC-O, `rico:directlyIncludes` is defined as a subproperty of the transitive property `rico:includesTransitive`, so the reasoner derives the dashed triples of Figure 6.2 by itself. In the notation of Section 5.2.1, two axioms are at work:

$$\texttt{directlyIncludes} \sqsubseteq \texttt{includesTransitive}$$

$$\texttt{includesTransitive} \circ \texttt{includesTransitive} \sqsubseteq \texttt{includesTransitive}$$

The first reads "if it directly includes, it includes"; the second is transitivity, "what an including thing includes is included". The whole "includes" relation that follows from the two is the **transitive closure**: the set of relations obtained by following the direct relation as far as it goes. If fonds to series and series to file can both be followed, fonds to file is in the closure. Written as a rule with variables (Section 5.2.1), the second axiom is

$$\texttt{includesTransitive}(x, y) \wedge \texttt{includesTransitive}(y, z) \rightarrow \texttt{includesTransitive}(x, z).$$

```
# derived by inference (you do not write these)
ex:fonds01  rico:includesTransitive  ex:series02 , ex:file07 , ex:record31 .
ex:series02 rico:includesTransitive  ex:file07 , ex:record31 .
```

However many levels deep the arrangement runs, then, the SPARQL is one line.

```
SELECT ?r WHERE {
  ex:fonds01  rico:includesTransitive  ?r .
  ?r  a  rico:Record .
}
```

Read: list every `?r` that fonds 01 includes in the transitive sense and that is also a record. The same variable `?r` occurs in both patterns, so only things satisfying both conditions come back. The answer is a list of records, `ex:record31` among them.

The equivalent in EAD and XML meant writing a program to walk the tree recursively. In a relational database it takes a recursive common table expression, which is fairly advanced SQL. With a graph and inference the model itself knows that inclusion is transitive, so the query stays

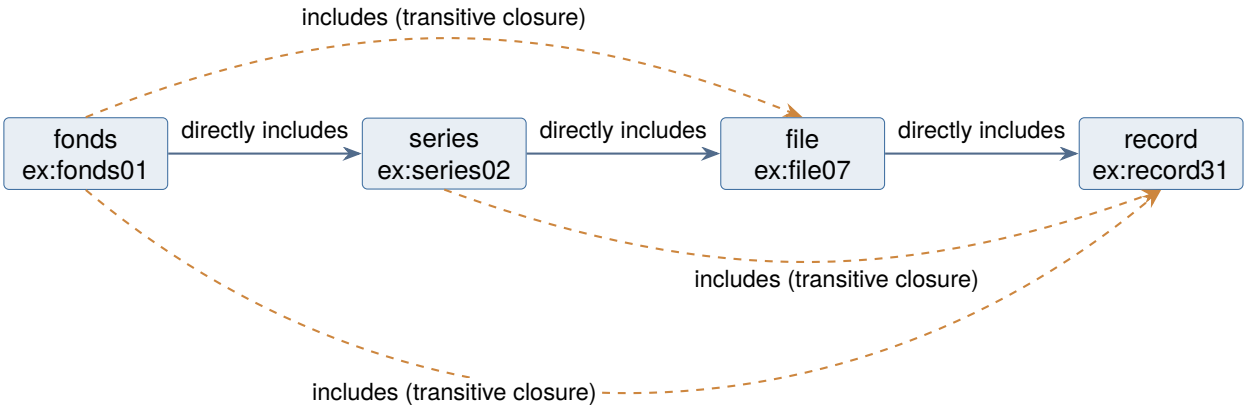


Figure 6.2: Derivation by transitivity. Solid lines are the triples you write (`rico:directlyIncludes`), dashed lines the triples inference supplies (`rico:includesTransitive`). Write the three between adjacent levels, and the reasoner fills in inclusion across levels.

simple. Transitive properties of the same kind exist for whole and part (`rico:hasPartTransitive`), subevents, containment of places, ancestry (`rico:hasAncestor`) and temporal precedence (`rico:precedesInTime`).

Axioms and procedures: between declaring and computing

Anyone who has written programs may want to state transitivity as two rules instead, in the familiar recursion that builds ancestry out of parenthood:

$$\mathit{parent}(x, y) \rightarrow \mathit{ancestor}(x, y)$$

$$\mathit{parent}(x, y) \wedge \mathit{ancestor}(y, z) \rightarrow \mathit{ancestor}(x, z)$$

The procedure shows through this formulation: build the closure by adding one step at a time to the parent data. Because the target, *ancestor*, appears at most once on the left of each rule, the shape is called *linear recursion*. The axioms in the main text, $\mathit{parent} \sqsubseteq \mathit{ancestor}$ and $\mathit{ancestor} \circ \mathit{ancestor} \sqsubseteq \mathit{ancestor}$, state only what is true of the result.

In principle the writer need not think about procedure at all. DL and OWL are *declarative*: an axiom settles what follows, and in what order to compute it is left to the reasoner. Decidability (Chapter 5) is the guarantee that any way of computing it terminates, and this division of labour, meaning to the writer and procedure to the engine, is the point of a declarative language. The two rules above can be written in OWL 2 as well (the first as a subproperty, the second as the chain axiom $\mathit{parent} \circ \mathit{ancestor} \sqsubseteq \mathit{ancestor}$).

Two qualifications. The two formulations *differ slightly in meaning*. Starting from parenthood alone the consequences are the same, but in RiC data one can assert directly, as a coarse fact, that A is an ancestor of B while the intervening parenthood is unknown (freedom of granularity). The transitivity axiom composes such directly asserted ancestries with each other ($\mathit{ancestor}(a, b)$ and $\mathit{ancestor}(b, c)$ give $\mathit{ancestor}(a, c)$); the linear recursion does not compose anything that does not pass through parenthood. RiC-O defining `rico:hasAncestor` as transitive is thus a choice about *meaning*, made on the assumption that coarse assertions will be mixed in, and not a matter of procedural convenience.

And even with the same meaning, the *cost of computing* varies with the shape of the axiom. Linear recursion builds the closure by joining derived facts to the original data, whereas applying the transitivity axiom naively joins derived facts to each other and can multiply redundant derivations. That is absorbed by the engine, though; a triplestore normally computes

transitive closure with a dedicated routine. OWL 2 goes further and defines *subsets with guarantees about computation* (the profiles RL, EL and QL), saying which range of axioms a rule engine can implement and which range runs in polynomial time. The principal RiC-O axioms (subclass, subproperty, inverse, transitivity, chains) fall broadly inside the range suited to a rule engine. The convention in this field is that the describer writes only meaning, and performance is separated off as a matter of store configuration and of whether derived triples are computed in advance (Section 6.8).

6.3 Situation 2: found whichever way it was written, by inverse properties

Some institutions write the creator on the record; others write the created works on the creator. In RiC-O the paired properties are declared with `owl:inverseOf`[1], so the reasoner supplies the other direction whichever was written, and the data can be searched from either end. When data is being pooled across institutions, which is what linked data is for, absorbing differences in the direction of writing is of real practical value.

6.4 Situation 3: no penalty for describing in detail, by rolling up subproperties

One institution describes personal connections in detail, as spouse (`rico:hasOrHadSpouse`) and teacher (`rico:hasOrHadTeacher`); another writes only that some connection existed. In RiC-O these properties form a hierarchy: `hasOrHadSpouse` $\sqsubseteq$ `hasFamilyAssociationWith` $\sqsubseteq$ `isAgentAssociatedWithAgent`. The reasoner derives the broader statement from the narrower one, so a query for "people connected in any way with Margaret Ellis", written with the one broad property, catches the finely described data and the coarsely described data *in the same net*. Narrow statements being drawn up into the broader property in this way is called **roll-up**, and the word is used again in later chapters.

This is the technical warrant for the freedom of granularity described in Chapter 3. Describe in detail and detailed searches are answered; describe coarsely and a search across everything still finds you. The hierarchy of the ontology stops differences of descriptive depth from turning into fractures in retrieval. Classes work the same way: an instance of `rico:Person` is automatically an instance of `rico:Agent`, so a query for all agents is answered by persons, corporate bodies and mechanisms alike.

6.5 Situation 4: shortcuts generated automatically, by property chains

As Section 5.5.3 showed, a relation that needs a date or a degree of certainty is written by reifying it. The `a` on the second line is the predicate meaning "is an instance of" (the Turtle box in Chapter 5).

```
ex:fondsP83  rico:thingIsSourceOfRelation  ex:prov83 .
ex:prov83  a  rico:OrganicProvenanceRelation ;
    rico:relationHasTarget  ex:publicHealth ;
    rico:relationHasDate    ex:date1948_1994 ;
    rico:relationCertainty  "certain" .
```

[1] `rico:hasCreator` and `rico:isCreatorOf`, for instance. In description logic, `isCreatorOf` $\equiv$ `hasCreator`$^{-}$.

Against that long path, RiC-O defines `rico:hasOrganicProvenance` as a shortcut by property chain. In description logic,

$$\texttt{thingIsSourceOfRelation} \circ \texttt{organicProvenanceRelation\_role} \circ \texttt{relationHasTarget}$$

$$\sqsubseteq \texttt{hasOrganicProvenance}$$

and as a rule with variables,

$$\texttt{thingIsSourceOfRelation}(x, y) \wedge \texttt{organicProvenanceRelation\_role}(y, z)$$

$$\wedge\, \texttt{relationHasTarget}(z, w) \rightarrow \texttt{hasOrganicProvenance}(x, w)$$

which is to say: if you can get from the entrance of the relation node to its exit in three steps, you may join the ends directly. The reasoner derives this:

```
# derived by inference
ex:fondsP83  rico:hasOrganicProvenance  ex:publicHealth .
```

So *data written carefully and data written quickly come out in the same shape at query time*. The searcher can always query with the short direct property and need not know which style the data was written in. RiC-O 1.1 defined shortcuts of the same kind for dates (`rico:hasCreationDate` and others) as chains. Without this machinery, the flexibility of having several ways to express one thing would have been a nightmare for retrieval. Property chain axioms are what let flexibility and searchability coexist.

6.6 Situation 5: following a succession of organisations

The example queries in the RiC-O documentation ring true to anyone who has used archives: which body took over the activities of an institution after it was dissolved; what instantiations exist of this photograph; what surviving copies there are of this original. Take the last of the situations through in full, using a succession of organisations.

The regional public health administration will serve. A Public Health Department is established in 1948, reorganised in 1994 into a Health and Welfare Department, and restructured again in 2010 into a Department of Health and Social Care. In RiC this is three corporate bodies (Agent), one activity (public health administration), and triples for succession and for performance of the activity.

```
ex:publicHealth  a rico:CorporateBody ;
    rico:name "Public Health Department" ;
    rico:hasSuccessor ex:healthWelfare .
ex:healthWelfare a rico:CorporateBody ;
    rico:name "Health and Welfare Department" ;
    rico:hasSuccessor ex:healthSocialCare .
ex:healthSocialCare a rico:CorporateBody ;
    rico:name "Department of Health and Social Care" .

ex:phAdmin  a rico:Activity ; rico:name "public health administration" .
ex:publicHealth     rico:performsOrPerformed ex:phAdmin .
ex:healthWelfare    rico:performsOrPerformed ex:phAdmin .
ex:healthSocialCare rico:performsOrPerformed ex:phAdmin .

ex:cdpSeries  a rico:RecordSet ;
    rico:name "Communicable disease prevention" ;
    rico:hasOrganicProvenance ex:publicHealth ;
    rico:documents ex:phAdmin .              # documents this activity
```

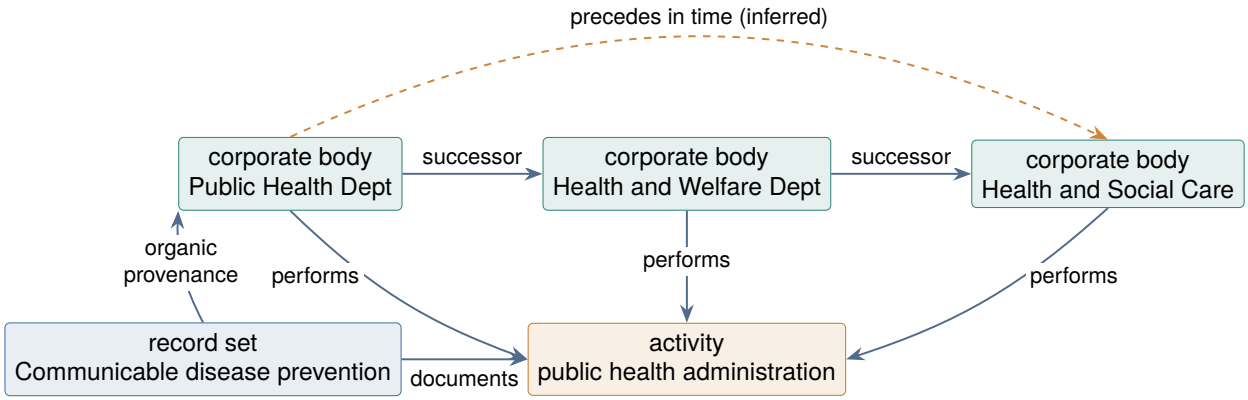


Figure 6.3: A succession of organisations as a graph. From the series to the body that created it, along the chain of succession to the body responsible now, and around the activity to the record sets of every body in the line: every direction can be followed.

Drawn as a graph, the data looks like Figure 6.3.

Several things become possible with this data. Starting from a series of the 1950s, a machine can follow creator (the Public Health Department), then the chain of succession (temporal precedence derived from `rico:hasSuccessor` is transitive), to the department responsible today. Going the other way, from the present department, it can gather every record set that any predecessor left on the same activity.

```
# list the record sets documenting public health administration,
# together with the body that created each
SELECT ?rs ?agent WHERE {
  ?rs  rico:documents            ex:phAdmin ;
       rico:hasOrganicProvenance ?agent .
}
```

Read: for every record set `?rs` that documents the activity of public health administration and has organic provenance `?agent`, list the pairs. The semicolon means what it means in Turtle, "carry on with the same subject `?rs`". The answer is a list of pairs like (Communicable disease prevention, Public Health Department), and where the data for the whole line of bodies is present, four lines of query produce a list running the length of the institutional history, organised around the activity.

In the world of ISAD(G) this information was written in prose in the administrative history of each fonds, and matching it up was work for a person. In RiC the succession is structured data, so a machine can answer. For the archives of public administration, this is where the advantages of a graph and inference show most directly.

6.7 Situation 6: can you add rules of your own?

Everything so far concerned axioms *already built into* RiC-O. This section is about adding vocabulary and rules specific to what you are describing. Take a requirement: find documents written by an ancestor common to two people. Working it through separates what can be done from what cannot.

6.7.1 Adding vocabulary is explicitly allowed

One of the RiC-O design principles is extensibility, and adding subclasses and subproperties of your own in your own namespace is an officially anticipated use (RiC-O documentation, design principle 5). RiC-AG gives an FAQ, "How to extend RiC", to the question, and sets out the recommended

practice: use your own namespace and identifiers; always attach a new component below an existing RiC component; document and publish the extension; and where the need looks general, consider proposing it to EGAD (through a GitHub issue or the users' mailing list). It gives a worked example, defining "has testator" for notarial documents as a subproperty of `rico:hasAuthor`. To distinguish, say, records created by someone acting as a churchwarden, you would write:

```
PREFIX myo: <https://archives.example.org/ontology#>
myo:hasCreatorAsChurchwarden  rdfs:subPropertyOf  rico:hasCreator .
```

Declared as a subproperty, your fine-grained local vocabulary still answers a search on `rico:hasCreator` (the roll-up of situation 3). *Grafting onto the existing hierarchy* is what keeps an extension from becoming a departure from RiC. Invent an isolated property and it is cut off from any search in the standard vocabulary.

6.7.2 The common ancestor: a case that needs only a query

The requirement at the head of this section can in fact be met without adding any rule. RiC-O already defines `rico:hasAncestor` as a transitive property, so writing the parent relations (`rico:hasChild` and the rest) yields ancestry across generations by inference. What remains is a SPARQL query for records created by someone who is an ancestor of both people.

```
SELECT ?doc ?x WHERE {
  ex:margaret  rico:hasAncestor  ?x .   # an ancestor of Margaret (transitive)
  ex:thomas    rico:hasAncestor  ?x .   # and an ancestor of Thomas
  ?x  rico:isCreatorOf  ?doc .          # records that person created
}
```

Read: list every pair of a person `?x` who is an ancestor of Margaret *and* an ancestor of Thomas, with the records `?doc` that `?x` created. What does the work is that the same variable `?x` appears in all three patterns; sharing the variable is what states the condition that the two lines of ancestry meet in one person. Since `rico:hasAncestor` is transitive[2], writing the parent relations alone brings great-grandparents and beyond into range.

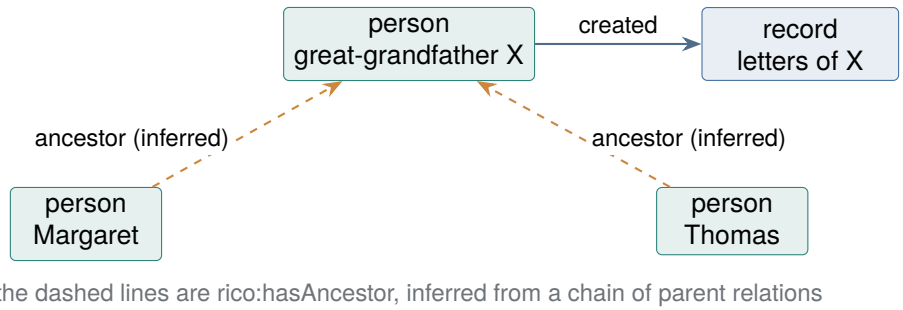


Figure 6.4: Finding documents by a common ancestor. Whether two lines of ancestry meet in the same person X is expressed by sharing a variable in SPARQL: two triple patterns referring to the same `?x`.

So the requirement falls inside what RiC-O covers, and the tools needed are vocabulary (already defined), inference (already defined) and a query (which you write).

[2] In description logic, $\texttt{hasAncestor} \circ \texttt{hasAncestor} \sqsubseteq \texttt{hasAncestor}$.

6.7.3 Some rules cannot be written in OWL

Defining the relation of *having a common ancestor* itself as a new property derived by inference (`myo:hasCommonAncestorWith`, say) cannot be done in OWL 2. Here is the boundary of what the language expresses. A property chain axiom (situation 4) can express only a path followed from A to B in one line. It cannot express the condition that two paths meet at *the same* intermediate node, which in logical terms is a join, the same variable occurring twice on the left-hand side of a rule. To realise a rule of that kind as a derivation, you go outside OWL: to the rule language of the triplestore (SWRL, Datalog-style rules, SHACL rules) or to generating the derived triples with SPARQL CONSTRUCT.

Requirements of your own therefore sort into three layers.

1. **Extending the vocabulary** (adding subclasses and subproperties). Inside OWL, and officially anticipated by RiC-O.
2. **Wanting an answer, and nothing more.** Add no rule; write a SPARQL query. The common ancestor is of this kind.
3. **Derivation rules OWL cannot express.** Implement with a rule engine or with CONSTRUCT. Outside what RiC-O covers, but running such rules over RiC-O data is ordinary technology.

The question one step before this, whether domain-specific knowledge should be held as an extension at all or put somewhere else, is Section 7.5.

6.8 Inside the engine: what actually executes a description

This section looks inside what has so far been lumped together as “the reasoner” and “the triplestore”. However much vocabulary and how many axioms one learns, the relation between axioms, retrieval and performance stays up in the air while the machine that executes them is invisible.

6.8.1 The structure of a triplestore

The inside of a triplestore is easiest to see from a relational database background (Chapter 8). It has roughly these parts.

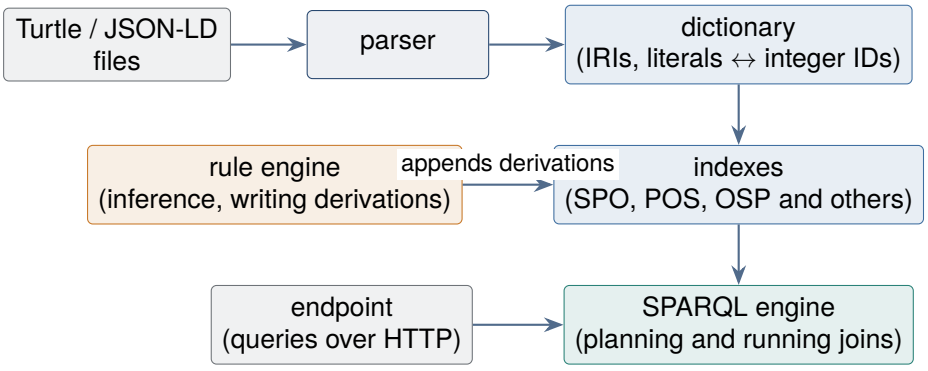


Figure 6.5: The inside of a triplestore in outline. In relational terms, the dictionary and indexes answer to B-tree indexes and the SPARQL engine to the query planner.

Triples that are loaded first pass through the *dictionary*, where IRIs and literals are replaced by internal integer identifiers (comparing long IRIs as strings every time would be slow). They are then stored in *indexes*. A triple has three parts, subject (S), predicate (P) and object (O), so holding redundant indexes in several orders (SPO, POS, OSP) means that any shape of query answers quickly, whether the subject is known or only the predicate. The SPARQL engine estimates in what

order to match the patterns of the WHERE clause so that the joins stay small, and runs them, which is the same idea as query planning in a relational database. Turn inference on and the *rule engine* applies the rules corresponding to the axioms (Datalog-style rules, as in the box in Chapter 5) over and over, appending the derived triples to the indexes.

6.8.2 Computing inference in advance, or at query time

There are two broad implementations. *Materialisation* (forward chaining) applies the rules exhaustively when the data is loaded and writes every derived triple into the indexes. Queries are fast, because the answer is already there, but the store grows, and *deleting* data means recomputing to retract the derivations that depended on the deleted fact. *Query-time expansion* (backward chaining, or query rewriting) does the reverse: it applies the rules in reverse when a query arrives. Updates are cheap and every query costs computation.

Among products, GraphDB is the leading example of the first, letting you choose the rule set (RDFS only, OWL-Horst, OWL 2 RL) in configuration. Apache Jena (Fuseki) combines a general rule reasoner with a store (TDB2) and can be configured either way. Others include Virtuoso, Amazon Neptune and the lightweight Oxigraph. All of them present a SPARQL endpoint, an HTTP interface to which applications send queries.

A different lineage of tool is the *tableau* reasoner (HermiT, Pellet, both callable from the free ontology editor Protégé). The tableau method tests whether a set of axioms is consistent by branching through the possible interpretations. It suits *checking an ontology* (are the axioms consistent, how does the class hierarchy resolve) rather than deriving over large volumes of data. Checking a vocabulary (TBox reasoning) and deriving over data (ABox reasoning) are jobs of different character, and the tools have divided accordingly. The defect in RiC-O 1.0 mentioned in Section 6.9 was exposed by a check of this kind, and fixed in 1.0.2.

6.8.3 Four things you will use in SPARQL

Beyond the basic form, four features come up constantly. They are shown here over the example data used so far.

OPTIONAL: take it if it is there A query that adds the name where there is one and does not drop the row where there is not.

```
SELECT ?r ?name WHERE {
  ex:fonds01  rico:includesTransitive  ?r .
  OPTIONAL { ?r  rico:name  ?name }
}
```

What is inside `OPTIONAL` returns a value if it matches and leaves the row standing if it does not, so records with no name appear in the list with a blank. Under the open world assumption, “no name is written” does not mean “there is no name”, and it matters that this modest behaviour is the default.

FILTER: narrow by value Narrowing by date.

```
SELECT ?r ?d WHERE {
  ?r     a  rico:Record ;
         rico:isAssociatedWithDate  ?date .
  ?date  rico:normalizedDateValue    ?d .
  FILTER(?d < "1900")
}
```

This keeps records earlier than 1900. Comparison of values, matching of strings and tests on language tags can all be written here. (The example compares four-digit years as strings, which is correct only where the digits line up; strictly, typed literals should be used.) The date is reached in two steps because the domain of `rico:normalizedDateValue` is `rico:Date`, not a record resource (Section 4.2.7). Attaching `rico:normalizedDateValue` directly to a record is not valid RiC-O.

Property paths: transitive closure without inference Repetition of a predicate can be written on the query side instead.

```
SELECT ?r WHERE {
  ex:fonds01  rico:directlyIncludes+  ?r .
}
```

The + after a predicate means one or more repetitions, following `directlyIncludes` down any number of levels. So the transitive closure can be computed *without inference*, on the query side. The two have different uses. Making transitivity an *axiom* puts the meaning, that inclusion chains, into the data itself, where every user and every query benefits. Writing a *path* computes it for that one query and does not depend on how the store is configured for inference. A public endpoint may well have no reasoner behind it, so the path notation is one to learn.

CONSTRUCT: build new triples from results Return results as triples rather than as a table.

```
CONSTRUCT { ?rs  rico:hasOrganicProvenance  ?a }
WHERE {
  ?rs  rico:thingIsSourceOfRelation  ?rel .
  ?rel a  rico:OrganicProvenanceRelation ;
       rico:relationHasTarget  ?a .
}
```

Where `SELECT` returns a table, `CONSTRUCT` returns *a new set of triples*. This example generates the shortcut property from a reified provenance relation, running by hand the same derivation that the chain axiom of situation 4 performs. Data conversion (Chapter 7) and the implementation of rules OWL cannot express (situation 6) are usually carried out by `CONSTRUCT`.

6.8.4 OWL 2 profiles: subsets with guarantees

Last, the relation between expressive power and what an engine can do has been institutionalised in the OWL 2 *profiles*. More expressive power makes inference heavier, so subsets have been standardised, each saying: stay inside this range and your data can be processed by this method, at this cost.

Table 6.1: The OWL 2 profiles

Name	Character	Method and typical use
OWL 2 DL	Maximum expressive power ($\mathcal{SROIQ}$)	Tableau reasoners (HermiT and others). Suits checking a vocabulary. Worst-case cost is high
OWL 2 RL	What can be written as rules	Materialisation by a rule engine (GraphDB and others). Suits derivation over large data
OWL 2 EL	Classifying very large vocabularies	Polynomial time. Huge ontologies such as the medical terminology SNOMED CT
OWL 2 QL	Querying over a relational database	The range that can be rewritten into SQL

The axioms RiC-O actually uses (subclass, subproperty, inverse, transitivity, chains) fall broadly inside OWL 2 RL. So the standard arrangement in practice is to load the data into a triplestore with an RL rule engine and materialise, and everything in this chapter runs on that. Axioms outside RL, such as the cardinality restriction on Proxy ("exactly one"), are little used for deriving data and earn their place in checking the vocabulary. Here again the RiC-O policy noted in Chapter 3 shows through: modest in expression, dependable in computation.

6.9 Inference is not magic

Inference derives what follows from the axioms and facts written, and nothing else. Four cautions before relying on it.

Remember the open world assumption OWL inference runs on the premise that what is not written is unknown. A query about *absence*, such as a list of records with no creator recorded, has to be written as a check that a triple does not exist (`FILTER NOT EXISTS` in SPARQL) rather than as a logical negation, and the difference in meaning has to be understood to use them apart.

Inference amplifies errors in the data One relation asserted in error breeds many erroneous derived triples, and transitive properties carry an error a long way. Quality control of the input is the precondition of inference.

Computation is not free Deriving everything in advance (materialisation; Section 6.8) makes queries fast at the cost of storage and update; deriving at query time has the opposite profile. Early releases of RiC-O 1.0 contained a defect in which a combination of axioms made reasoners report an inconsistency. Forty-eight auxiliary properties, present to make the shortcut inferences work, joined an individual of a relation class to itself, and had been declared in a way that asserted every individual stands in that relation to itself. It was fixed in 1.0.2. An implementation using inference will have dealings with technical details of this kind, which is precisely why the system, and not the describer, should be the one to have them.

Inference helps only where the axioms capture the meaning The RiC-O axioms are the product of EGAD's design decisions, and know nothing of your institution's working rules (that reference numbers within a fonds are unique, say). Constraints of that kind are checked with a different technology, such as SHACL (Chapter 8).

Exercises (hints in Appendix A.3)

6.1 Write the transitivity axiom $R \circ R \sqsubseteq R$ as a rule with variables.

6.2 How does OPTIONAL in SPARQL suit the open world assumption? Answer by saying what happens without it.

6.3 Compare materialisation with query-time expansion for data that is updated frequently.

6.4 In the example of situation 5, write out the chain of relations that inference follows from the Communicable disease prevention series to the Health and Welfare Department, naming the axioms used.

Chapter 7

Putting It into Practice

The official guidance on practice is the RiC-AG draft 0.1, published in October 2025. As already noted, the AG states that the variety of use cases makes a step-by-step guide impossible, and offers instead a list of actions that might be considered, with examples. This chapter reassembles the material into a workable procedure, drawing on the AG chapter on implementation strategies and its *Getting started*, on the account of transition in RiC-CM §1.6.4, on the RiC-O design principles and official samples, and on the experience of institutions that have gone first.

7.1 What you need before you can describe

ISAD(G) was a document you could pick up and describe with. Each of the 26 elements came with a purpose and a rule, and what to write and how was written there. RiC-CM is not like that. As Section 1.2 showed, RiC-CM says of itself that it is not a standard for composing descriptive content, and it adds that the value schemas of attributes are "indicative" and neither prescriptive nor proscriptive (§3.5). There is a list of entities, attributes and relations, and the column that ISAD(G) filled with rules is empty.

RiC-CM's own expectations show in the same place. Section 1.6.2 says that what RiC-CM settles is a framework standardising the *input* to a system, leaving output and interface unconstrained, and goes on to express the hope that developers of description software will implement it. Practitioners are to receive the benefits of RiC-CM through the tools that result. It was not designed as a document a describer works from directly.

Getting from RiC-CM to a description therefore means supplying three things yourself.

An application profile Which of the 19 entities, 42 attributes and 85 relations your institution will use, and which will be required. Designing this is step 2 in Section 7.3.

Content rules What form names take, how dates are normalised, which controlled vocabularies are used. The layer that DACS occupies in relation to ISAD(G). RiC does not have one yet. RiC-AG 0.1 does not fill it either, and says as much: it offers advice and examples rather than rules, and states at the outset that it does not give specific guidance for every context of use.

Somewhere to keep the description A system for entering and holding it, or a spreadsheet design standing in for one (Section 7.11).

Dividing it in three is a way of seeing what has not been provided, not a claim that standards divide this way. ISAD(G) combined a list of elements with the rules for writing them in one document, and EAD, while specifying an encoding, also carries decisions about which elements exist. Real standards do several jobs at once. What is unusual about RiC-CM is that it says explicitly that it will do only one of them.

That these three are not provided is why institutions are asked for judgement repeatedly through this chapter, and it also shapes the choice of scenario that comes next.

7.2 Three scenarios

Ways of engaging with RiC fall into roughly three kinds, depending on where an institution stands.

Scenario A: convert and publish existing description Convert catalogue data already held in EAD, spreadsheets or a database into RiC-O and RDF, and publish it as linked open data or contribute it to a cross-institutional search. This has the most experience behind it at present; the Archives nationales de France is the type case.

Scenario B: describe natively in RiC Create new description as entities and relations from the outset. Systems are still few, so this is confined to institutions in front, but the gains are large where organisations change often, as in the records of public administration.

Scenario C: prepare for later Carry on with ISAD(G) and EAD for now, while tightening the discipline of the data so that conversion will be easy later: authority control over creators, unique identifiers, normalised dates. For most institutions this is the realistic answer today.

The AG chapter on implementation strategies gives advice consistent with this division. Start by identifying the purpose (ask what is the most useful thing that RiC will give you, and tie it to the needs of the institution); start small and grow in stages (a pilot project, so that failure costs least); learn RiC as a whole team (archivists, IT staff and data analysts see it from different angles); and estimate the resources and make the business case. On the order of software adoption it sketches several routes: converting the public portal to RiC first; converting existing data one-to-one first and improving the description afterwards; or adding RiC-based functions alongside existing ones.

Note that scenario C is not the same as doing nothing. The hardest part of converting to RiC is not the complexity of RiC but *the untidiness of existing data*: variant forms of one creator's name, duplicated identifiers, dates written freely. Those can be improved within current practice, and the improvement pays whether RiC arrives or not.

How far has adoption got (2026)?

The claim that scenario A has the most experience behind it can be supported. France went first. After the PIAAF pilot on authority interoperability with the national library and the ministry of culture (published February 2018), the Archives nationales (AnF) developed RiC-O Converter with the firm Sparna, released in April 2020. It then published a knowledge graph, the term in this field for a dataset built as a network of entities joined by relations, drawn from the records of 40 of the 122 notarial practices of Paris, with a visual query interface (Sparnatural). The EGAD resource list records this dataset at 57.9 million triples, of which some 37 million were generated by inference. The portal FranceArchives opened a SPARQL repository in 2023; as of 2025 it offers 504 million triples generated from 101,000 finding aids (conforming to RiC-O 0.2), which its developers describe as the largest RDF repository of archival metadata conforming to RiC-O 0.2.

In Switzerland the audiovisual heritage portal Memobase runs in production on a RiC-O-based data model, and the cataloguing software of docuteam (docuteam context, the source of the example in Chapter 5) supports description natively in RiC-O. The SAPA Foundation for the performing arts is another Swiss adopter. Across Europe, EHRI, the research infrastructure for Holocaust-related material, has built a knowledge graph using RiC. Applied research continues elsewhere: Portugal (correspondence of scientists), Greece (court records), Korea (records of the arts), China (an ontology extension for ration coupon material). A "RiC Resource List", compiled by the user community and published under EGAD, collects these, and is the place to follow the current state.

As of 2026, production use is concentrated in a small number of institutions that went first, and much of the rest is still research and demonstration.

7.3 The standard workflow for scenario A

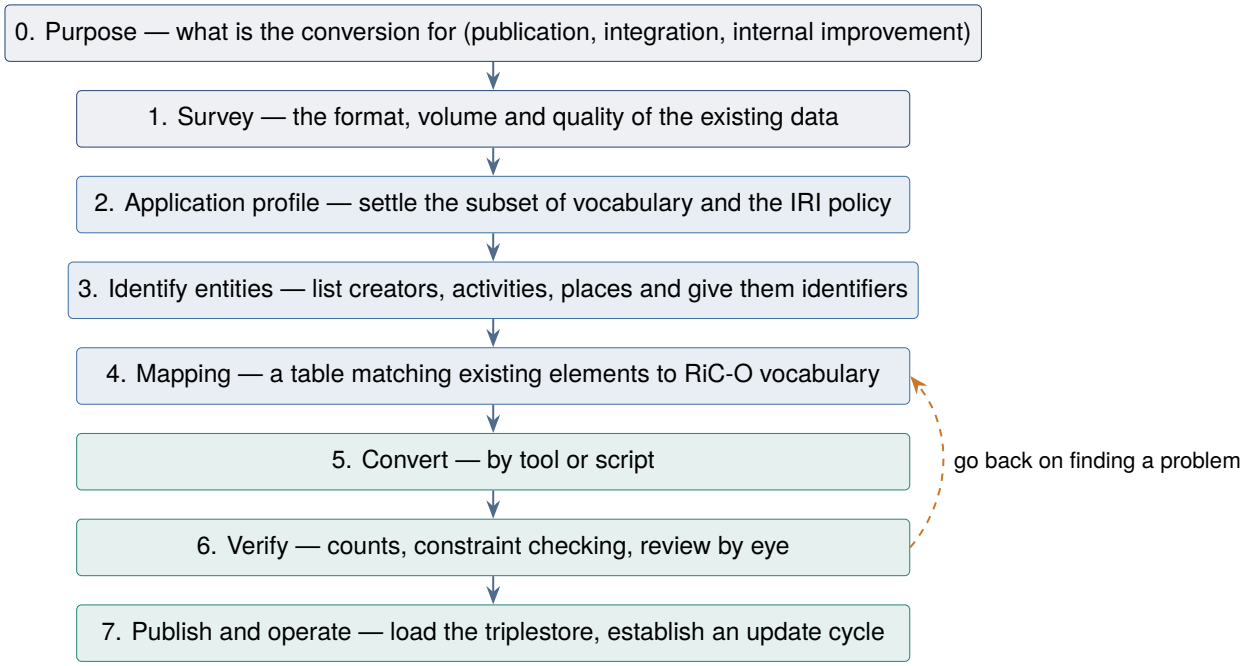


Figure 7.1: The standard workflow for converting existing description into RiC-O and RDF. Steps 2 to 4 are the intellectual centre of the work and need the most archival judgement.

7.3.1 Step 2: designing the application profile

No implementation uses all 480 object properties and 75 datatype properties of RiC-O, and the official documentation states outright that an implementing institution may use whatever subset it needs. So the first thing to settle is the list of classes and properties your institution will use: the application profile. Four decisions carry most of it.

1. **Which entities.** A minimal configuration can start with Record Set and Record, Agent and Activity. Whether to separate the instantiation, and how far to raise rules and events into entities, follow.
2. **Shortcut or exact form** (Section 5.5). Whether dates, types and names are written as literals or as entities. A workable rule is to raise into entities whatever you want to share with the outside world (creators, record set types, places), and leave internal notes as literals.
3. **Direct property or relation class** (Section 5.5.3). Reify only those relations whose date or authority has to be recorded, such as provenance.
4. **IRI design.** The domain, the path structure (`/records/`, `/agents/`), and the undertaking not to change them. Keeping a published IRI alive is the ethic of linked data.

The profile is documented and maintained as an institutional rule. In RiC terms it is a Rule governing the activity of describing, and can itself be described in RiC (RiC-CM chapter 6).

7.3.2 Step 4: doing the mapping

Official tools now exist for this step. Alongside RiC-AG 0.1 came a comprehensive crosswalk from the four existing standards (ISAD(G), ISAAR(CPF), ISDF, ISDIAH) to RiC-CM, and one from the EAD 2002 DTD to RiC-O 1.1, published as spreadsheets (an EAC-CPF crosswalk is said to be in

preparation). The AG states at the same time that some correspondences are not unique (the "date" of ISAD(G) admits several readings) and that adjustment to an institution's own encoding practice is assumed. The table below summarises the main points, in the shortcut form.

Table 7.1: ISAD(G) elements and RiC-O, in the shortcut form

ISAD(G) element	Principal counterpart in RiC-O
3.1.1 Reference code	`rico:identifier` (or an `rico:Identifier` entity)
3.1.2 Title	`rico:title` / `rico:name`
3.1.3 Dates	the `rico:date` properties (or a `rico:Date` entity and a relation)
3.1.4 Level of description	`rico:hasRecordSetType` (to the individuals Fonds, Series, File)
3.1.5 Extent	`rico:RecordResourceExtent` (with `rico:quantity` and others)
3.2.1 Name of creator	`rico:hasOrganicProvenance` (or the wider `rico:hasOrganicOrFunctionalProvenance`) to an Agent entity
3.2.2 Administrative or biographical history	`rico:history` on the Agent (prose), or a set of Events
3.2.3 Archival history	`rico:history` (or Events of custody and transfer)
3.3.1 Scope and content	`rico:scopeAndContent`
3.4.1 Conditions governing access	`rico:conditionsOfAccess` (or a `rico:Rule` entity)
3.5.3 Related units of description	the various record-resource-to-record-resource properties
Hierarchy (above and below)	`rico:isOrWasIncludedIn` / `rico:includesOrIncluded`

As the table shows, most elements have two routes: carry the prose across as prose, or break it into entities. Not everything has to be raised at once. Converting the prose first, to move the contents into the new container, and then raising, in stages, what gains most from it (creators, organisational change), is the realistic approach, and the reason RiC-O keeps the shortcut forms is to make it possible.

7.3.3 Steps 5 and 6: converting and verifying

The starting point for conversion is RiC-O Converter, the open-source tool developed by the Archives nationales de France with Sparna, which converts EAD 2002 and EAC-CPF files into RiC-O-conformant RDF (first released in 2020). Since every institution uses EAD with its own habits, off-the-shelf conversion rules are rarely usable unaltered, and adjustment to local encoding practice is normal. Writing your own means XSLT, the W3C language for transforming XML, or Python. Converting from a spreadsheet or a database, a script using Python and the rdflib library is the standard approach.

Verification runs at three levels at least: counts (do the numbers of units of description and creators in the source match the individuals generated?); mechanical constraint checking (missing required properties, duplicate IRIs; the constraint language SHACL does this); and inspection of a sample (pull the converted result out with SPARQL and set it against the original catalogue). As Chapter 6 said, inference amplifies errors, so publishing without verification is unwise.

7.4 A worked example: catalogue and authority record in RiC

The easiest way to feel what migration means is to see, in actual data, what happens to two things most institutions already do: the *hierarchical description* of ISAD(G) and EAD, and the *authority record* of ISAAR(CPF) and EAC-CPF. The material is the public health example of Chapter 6. The hierarchy comes first, then the link to the authority record.

7.4.1 The old form: two documents joined by an identifier string

Practice has been to maintain the description of the records (a finding aid following ISAD(G), encoded in EAD) and the description of the creator (an authority record following ISAAR(CPF), encoded in EAC-CPF) as separate documents, tied together by an identifier. Simplified:

```
<!-- the finding aid (EAD), for ISAD(G) 3.1.1, 3.1.2, 3.2.1, 3.2.2 -->
<archdesc level="fonds"><did>
  <unitid>P-83</unitid>
  <unittitle>Records of the Public Health Department</unittitle>
  <origination>
    <corpname authfilenumber="AR-0125">Public Health Department</corpname>
  </origination></did>
  <bioghist><p>The Public Health Department was established in 1948 and
    reorganised in 1994 as the Health and Welfare Department. ...</p></bioghist>
  <dsc>  <!-- lower levels: the depth of nesting is the hierarchy -->
    <c01 level="series"><did>
      <unitid>P-83/1</unitid><unittitle>General administration</unittitle></did>
      <c02 level="file"><did>
        <unitid>P-83/1/12</unitid>
        <unittitle>Immunisation programme</unittitle>
        <unitdate normal="1948/1949">1948-49</unitdate></did></c02>
    </c01>
    <c01 level="series"><did>
      <unitid>P-83/2</unitid>
      <unittitle>Communicable disease statistics</unittitle></did>
    </c01>
  </dsc>
</archdesc>
```

The multilevel hierarchy, fonds to series to file, is expressed by the *nesting* of the `c01` and `c02` elements. Look at what is not there. Between a level and the one below it, no reference is recorded at all. Position within the XML document, which element sits inside which, does the work of stating the parent-child relation. In the terms of Section 2.4, for the hierarchy, position has taken the place of reference. It is the direct descendant of a paper catalogue in which indentation and sequence carried the structure.

```
<!-- the authority record (EAC-CPF), for ISAAR(CPF) -->
<cpfDescription>
  <identity><entityId>AR-0125</entityId>
    <nameEntry><part>Public Health Department</part></nameEntry></identity>
  <description>
    <existDates><dateRange>
      <fromDate standardDate="1948">1948</fromDate>
      <toDate standardDate="1994">1994</toDate>
    </dateRange></existDates>
  </description>
  <relations>
    <cpfRelation cpfRelationType="temporal-later">
      <relationEntry>Health and Welfare Department</relationEntry>
    </cpfRelation></relations>
</cpfDescription>
```

Having two documents at all is the achievement of ISAAR(CPF), which made information about a creator reusable across many fonds (Chapter 2). Look at how they are joined, though, and several places are *loose*. The reference from the finding aid to the authority record is the identifier string `authfilenumber="AR-0125"`, and neither the existence nor the uniqueness of the record that string

names is guaranteed by anything mechanical. The character of the relation of being a creator (created or accumulated, and between which dates) is buried in the name of an element and in prose (`bioghist`). The relation to the successor body is, on the authority side, the *string* "Health and Welfare Department", and often not even a machine-readable reference to that body's own authority record. And the same fact, established 1948, reorganised 1994, is written both in the prose of the finding aid and in the authority record, which is how duplicated maintenance and drift begin.

One more consequence of this structure should be fixed in mind: *the direction of a reference is not free* (Figure 7.2; the history of the means of reference is Section 2.4). In a document-based standard a relation is recorded as a field of one document or the other, so the direction of the arrow is settled by which document holds the field. The `origination` of EAD runs finding aid to authority record. The other direction does exist: `resourceRelation` in EAC-CPF lets an authority record point at a record resource. But the two are separate statements in separate documents, and nothing stops one being written without the other, or the two disagreeing. Bidirectional meant writing it twice and keeping the two consistent by hand.

The design can also be compared with the standard practice of a relational database (Figure 7.2(b)). There, a relation in which many descriptions point at one authority is expressed by a foreign key column on the *many* side. The direction is fixed by convention rather than by meaning, but unlike a document, a database has referential integrity: a reference to an authority that does not exist is refused.

RiC changes the situation once more (Figure 7.2(c)). The relation belongs to neither description. It is an independent arc in the graph, or an entity in its own right, and the direction retreats from being a constraint of the structure to being a choice of name, `rico:hasCreator` or `rico:isCreatorOf` (Section 5.1.2). What is pointed at is an IRI rather than an identifier string, so the uniqueness of the naming is guaranteed globally. In fairness, RDF takes the open world assumption and so has no *automatic enforcement* of referential integrity as a relational database does. Pointing at an entity that has not been described is not an error; it is read as "not written yet". Where existence has to be checked, check it explicitly, with SHACL (step 6 of this chapter). The three architectures can therefore be set out as combinations of where the reference is kept (inside a document, in a column on the many side, in an independent arc) and what guarantees it (nothing, enforcement by the database, a global name plus optional checking).

7.4.2 Moving the hierarchy: from nesting to explicit relations

Take the hierarchy first. There is one principle: *turn each parent-child relation that was implicit in the nesting into an explicit relation, one triple at a time*. Every unit of description becomes an entity of its own, and the level is carried by the value of `rico:hasRecordSetType` (the official vocabulary individuals `rst:Fonds`, `rst:Series`, `rst:File`).

```
ex:fondsP83  a  rico:RecordSet ;
    rico:hasRecordSetType  rst:Fonds ;
    rico:identifier  "P-83" ;
    rico:name  "Records of the Public Health Department" ;
    rico:directlyIncludes  ex:s1 , ex:s2 .   # what nesting used to say

ex:s1  a  rico:RecordSet ;
    rico:hasRecordSetType  rst:Series ;
    rico:identifier  "P-83/1" ;
    rico:name  "General administration" ;
    rico:directlyIncludes  ex:f112 .

ex:s2  a  rico:RecordSet ;
    rico:hasRecordSetType  rst:Series ;
```

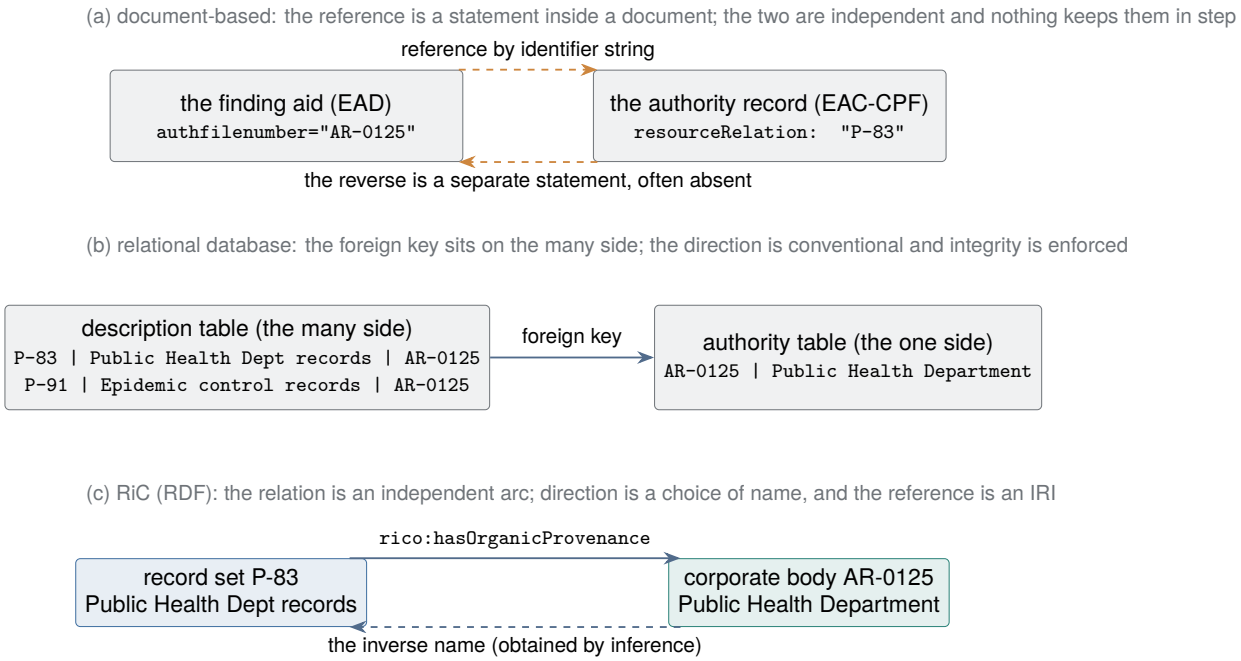


Figure 7.2: Three stages in the architecture of reference. Where a reference is kept moves from a statement inside a document (a), to a foreign key column on the many side (b), to an arc belonging to neither end (c), and fixing the direction changes from a constraint of the structure to a choice of notation.

```
    rico:identifier  "P-83/2" ;
    rico:name  "Communicable disease statistics" .

ex:f112  a  rico:RecordSet ;                # a file is a set of records too
    rico:hasRecordSetType  rst:File ;
    rico:identifier  "P-83/1/12" ;
    rico:name  "Immunisation programme" ;
    rico:isAssociatedWithDate  [ a rico:Date ;
                                 rico:expressedDate  "financial year 1948-49" ;
                                 rico:normalizedDateValue  "1948-04/1949-03" ] .
```

To the eye it is the same tree. Three things about it have changed, though.

The lower units have *become entities in their own right*. A `c02` element in EAD was a part of the fonds document and could not be pointed at from outside (or only as far as an identifier within the document). In RiC the file `ex:f112` has an IRI of its own and can be pointed at directly from an authority record, from another institution's data, from a user's citation.

The hierarchy has *become searchable data*. "Every file below this fonds" was, in EAD, the job of a program walking the document structure; in RiC it is the inference of `rico:includesTransitive` (Section 6.2) or one line of SPARQL property path.

And *the constraint of a single tree has gone*. The same file belonging to several sets, its original series and, say, a subject collection on immunisation, is one more relation (the multidimensional description of Chapter 3).

Something has not changed. The rules of multilevel description in ISAD(G) (describe from the general to the particular, put information at the level to which it applies, do not repeat at a lower level what was said at a higher one) remain good design in RiC. Writing each fact against exactly one entity is a principle a graph can carry through more thoroughly than a document could, since information higher up is reachable by following a relation and there is no reason to repeat it. When

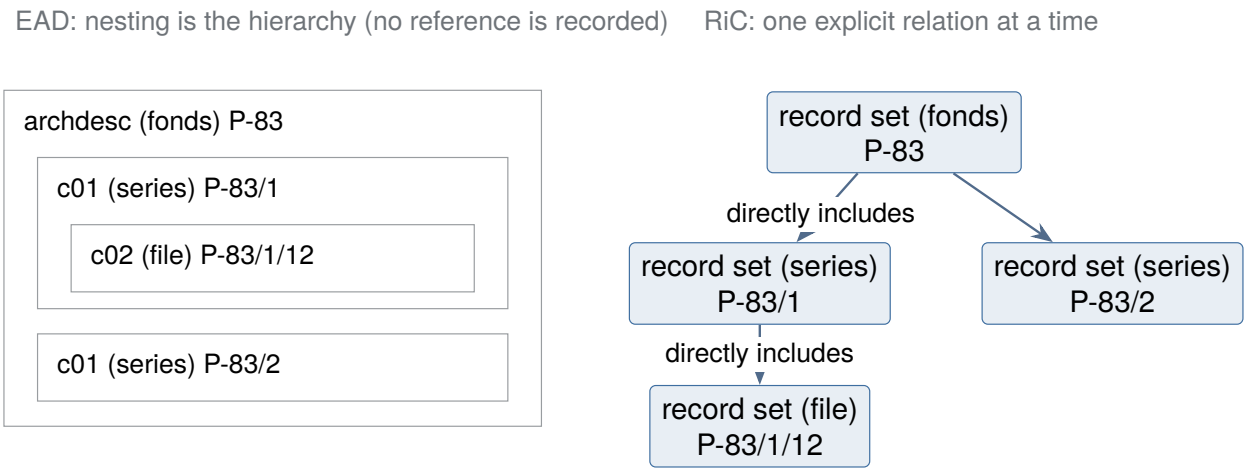


Figure 7.3: Migrating a hierarchical description. In EAD the nesting, that is, position within the document, was the hierarchy; in RiC each unit of description becomes an entity and the parent-child relation is asserted explicitly, one `rico:directlyIncludes` triple at a time.

RiC-CM says that descriptions based on ISAD(G) are accommodated by RiC (§1.6.4), this is what it means. Hierarchy survives, as *one kind of relation among many*.

7.4.3 The authority record: one graph, entities in it

Now the creator. Written in RiC, the same content loses the distinction between two kinds of document, and joins the hierarchy just built as Agent entities and relations in one graph.

```
ex:fondsP83  a  rico:RecordSet ;
    rico:hasRecordSetType  rst:Fonds ;
    rico:identifier  "P-83" ;
    rico:name  "Records of the Public Health Department" ;
    rico:thingIsSourceOfRelation  ex:prov83 .    # the provenance relation, reified

ex:prov83  a  rico:OrganicProvenanceRelation ;
    rico:relationHasTarget  ex:publicHealth ;
    rico:relationHasDate  [ a rico:Date ;
                            rico:expressedDate  "1948-1994" ] .

ex:publicHealth  a  rico:CorporateBody ;
    rico:identifier  "AR-0125" ;
    rico:name  "Public Health Department" ;
    rico:hasSuccessor  ex:healthWelfare .        # the successor is a reference too

ex:healthWelfare  a  rico:CorporateBody ;
    rico:name  "Health and Welfare Department" .
```

Four things have changed. Every reference is now a *direct reference by IRI*: not "an authority record with the identifier AR-0125 must be somewhere" but the entity `ex:publicHealth` itself (the old authority identifier can stay on the entity as an `rico:identifier` attribute). The relation now *has a meaning and a span*: not the position of a "creator" field in a form, but a relation of the organic provenance type, asserted with the dates 1948 to 1994. The successor relation has gone from a string to a relation between entities (`rico:hasSuccessor`), so the inference and retrieval over organisational change seen in Chapter 6 work on it as they stand. And duplicated maintenance goes away in principle: the fact of establishment in 1948 is written once, on the Public Health

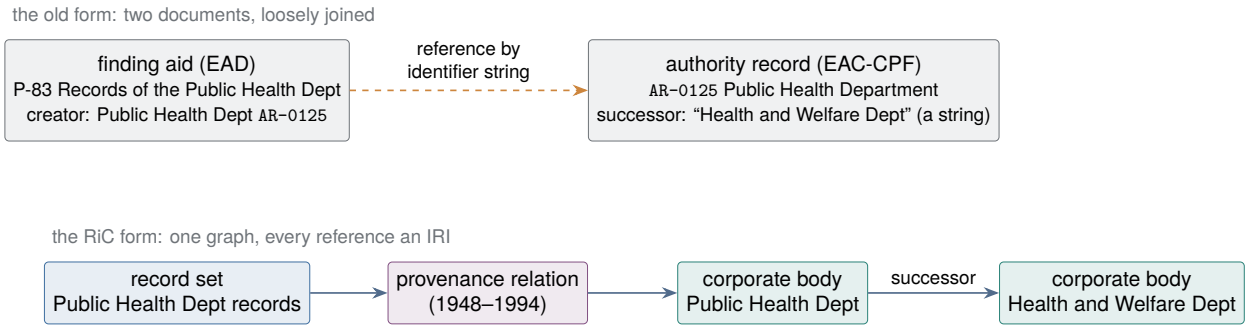


Figure 7.4: Catalogue and authority record, before and after. What was two documents joined by an identifier string becomes entities and relations in one graph.

Department entity, and a finding-aid-style display and an authority-record-style display are both *outputs* generated from the same graph (the separation of input from output in Chapter 3).

Put another way, the job of maintaining authority records does not disappear in RiC. It is promoted into *maintaining Agent entities and their relations*. The content ISAAR(CPF) prescribed (names, dates of existence, history, relations) maps onto the attributes and relations of Agent, and the official crosswalk gives the correspondence. The four kinds of relation in ISAAR (hierarchical, temporal, family, associative) are refined into the many Agent-to-Agent relations of RiC (superior and subordinate, successor, merger and division, family, employment). Only the control area of ISAAR maps not onto an entity but onto “describing description” (RiC-CM chapter 6), since the maintenance information about an authority record is a record of the activity of describing rather than an attribute of the agent.

7.4.4 Will it not all become a muddle? Logic joined, management divided

A word about the practicalities of management. Catalogue and authority record used to be maintained *independently*. Separate documents meant separate staff, separate update cycles, separate quality control. What happens to that independence when everything joins one graph is a fair question.

What clears it up is a distinction: *what is unified is the logic, the model of meaning, not the unit of management*. RiC says that a finding aid and an authority record describe the same world and, in meaning, form one graph. It does not say “manage them in one file, with one person and one update cycle”. And the RDF technology stack has come with a standard tool for dividing management while unifying logic from the start.

The first tool is the **named graph**. A triplestore can in fact store triples as quads, labelled with the graph they belong to (standardised as the RDF dataset of SPARQL 1.0 in 2008, and taken into the data model itself in RDF 1.1 in 2014; the serialisation is TriG). With it, the old division into documents can be reproduced exactly as a division into graphs.

```
GRAPH ex:agents {    # authority side: maintained by the authority team
  ex:publicHealth  a  rico:CorporateBody ;
      rico:name  "Public Health Department" ;
      rico:hasSuccessor  ex:healthWelfare .
}
GRAPH ex:catalog {   # catalogue side: maintained by the cataloguing team
  ex:fondsP83  a  rico:RecordSet ;
      rico:name  "Records of the Public Health Department" ;
      rico:hasOrganicProvenance  ex:publicHealth .   # a reference across the
    boundary
}
```

The authority graph and the catalogue graph can be updated separately, backed up separately and checked separately (SHACL validation can be run per graph). Only at query time are the two composed into a single logical graph. So the old independence of documents is not lost. It changes character, from a *physical accident* (they happened to be separate files) to a *deliberate design* (how to divide for management). Where to put the relation triples that cross the boundary (the `rico:hasOrganicProvenance` above) is itself a design question.

At the level of logic the answer about *relations that cross a boundary* is clear: they belong to *neither*. A relation is a first-class object of description in RiC, and reified it becomes a third entity pointing at both ends by IRI (Section 5.5.3). A relation ceasing to be the property of one description or the other was exactly the change from the old form. At the level of management there are three places to put it, and what decides is *who makes the statement and who maintains it*.

1. **In the catalogue graph.** Relations settled by the cataloguer in the course of processing (that the provenance of this fonds is the Public Health Department) are products of cataloguing work and belong on that side. This agrees with the foreign-key convention that the many side points, and relations generated by conversion from EAD fall here naturally.
2. **In the authority graph.** Relations with an Agent at both ends (`rico:hasSuccessor`, merger and division) are products of authority work. There is no room for doubt here.
3. **In a separate relation graph.** Where the relations have a research and maintenance cycle of their own. The precedent is well established. In a relational database, a many-to-many relation is standardly given a *junction table* belonging to neither table (Section 2.4), and in linked data it is common practice to publish a **linkset** (a set of links between two datasets, as defined by VoID, the vocabulary for describing datasets) as a third managed unit. Cross-institutional identity links (`owl:sameAs`, Section 7.6) almost always take this form. The CRS of the series system (below) was in substance the same thing: a central, independently maintained register of relations between series and agencies.

For a provenance relation spanning catalogue and authority, the first candidate is the catalogue graph. In most institutions establishing provenance is part of processing and cataloguing, so the place matches the person answerable for the statement. If the relation is reified to carry dates, evidence and certainty, and is developed as provenance research on a cycle separate from cataloguing, promoting it to the third place is the right move. Note that a direct property form (one triple) has to go into one graph or another, whereas a reified form moves as a unit, the relation entity and its attributes together, and survives being put in a graph of its own. Relation classes (Section 5.5.3) thus have the additional merit of dividing well. Whichever is chosen, which type of relation goes in which graph is a convention to be written into the application profile.

There is a further point about storage. *The data need not even be in the same store.* Reference by IRI does not require the target to be in the same database. Authority data maintained and published by a national shared authority service, catalogue data by each repository, on separate servers, pointing at one another by IRI: that is not a compromise but the design of linked data itself (Section 7.6). SPARQL also has a mechanism for querying across endpoints (`SERVICE`).

And which description is maintained by whom, and under which rules, can itself be recorded in RiC (describing description, RiC-CM chapter 6). The role played by the control area of ISAAR(CPF) is inherited here.

The mechanism has limits, and they should be stated. A named graph is a mechanism of storage and query, not part of the OWL semantics. Which graphs inference runs over is a matter of configuration (Section 6.8), and write permissions and approval workflows are outside the standard entirely, belonging to institutional regulation and system implementation. In the vocabulary of Section 8.3.1, what RiC standardises reaches as far as the conceptual schema, the logical design of the domain itself; for the internal schema, how it is actually stored, named graphs are the standard tool; and operational control belongs to neither. The standard unifies the logic. Named

graphs divide the storage. Control remains operational work. A muddle is not inevitable, but the independence that a physical document used to grant silently now has to be designed.

7.4.5 The series system and RiC, converging after sixty years

Some readers will have thought, looking at that before-and-after, of the Australian *series system*. Proposed by Peter Scott in the 1960s, it abandons the fonds as a fixed apex: records are described at the level of the *series*, agencies are described independently, and the two are joined by relations *with dates*. Rather than rewriting which body a body of records belongs to at each reorganisation, you add relations. It has run for over half a century as the CRS (Commonwealth Record Series) of the National Archives of Australia.

RiC can be read as the generalisation and international standardisation of that idea. What the series system achieved with two kinds of description (series and agency) and dated relations, RiC extends to arbitrary entities and relations. Read the other way, experience of running a series system is a working model for RiC. Set out as a list, it comes to this, and it is the same structure built in Section 6.6:

- Describe a series as a `rico:RecordSet` of its own, not fixed inside a particular fonds.
- Reify the relations of creation and accumulation (`rico:OrganicProvenanceRelation` and the rest) so that they carry *dates*, and when responsibility passes, *add* a relation rather than rewriting the old one. Not rewriting the past is the thinking behind the *hasOrHad…* naming.
- Hold organisational change on the agency side with `rico:hasSuccessor`, and merger and division with `rico:wasMergedInto` and `rico:wasSplitInto`.
- Where a fonds is wanted, for presentation to users or for exchange, construct it afterwards as a *secondary grouping*. Multiple membership means that the old opposition between the fonds camp and the series camp stops being exclusive.

ISAD(G), which is founded on the fonds, and the series system, which does not raise one, have long been treated as separate schools. A RiC graph expresses either. That is not a compromise but a demonstration, in the form of a model, that the two were different cuts through the same reality.

7.5 Where to put domain knowledge

The hardest judgement in designing an application profile is probably *where* in a RiC description to put knowledge specific to your own field: the concepts around a performance in the performing arts, the documentary forms and parish offices of early modern local records. Broadly there are two roads, describing only facts within the existing RiC-O vocabulary, or adding a vocabulary of your own field to RiC-O. They differ more than they seem to, so some guidance is in order.

In practice there are three places.

Place 1: in the data, within the existing vocabulary Write field-specific information as *values* of existing properties. A type as a string in `rico:type`, an explanation as prose in `rico:scopeAndContent` or `rico:generalDescription`, a subject as a subject relation to a Thing. Nothing in RiC-O is touched.

Place 2: separated out as a controlled vocabulary (SKOS) Develop the field's *system of concepts*, the list of documentary forms, of offices, of works performed, as a controlled vocabulary (SKOS) separate from RiC-O, and *refer* to it from the RiC-O `*Type` classes or from subject relations. Again nothing in RiC-O is touched. This is the road covered by the RiC-AG FAQ "How to use RiC with vocabularies".

Place 3: extending the ontology (subclasses and subproperties) Add the *meaning of a relation*

specific to your field, in your own namespace, below an existing RiC-O component (Section 6.7). This widens the RiC system of rules itself, by grafting.

SKOS: the standard for writing a controlled vocabulary in RDF

Since SKOS keeps recurring from here, here it is in one place. SKOS (Simple Knowledge Organization System) is the W3C standard (recommendation of 2009) for expressing *controlled vocabularies*, that is, knowledge organisation systems such as thesauri, subject heading lists and classification schemes, in RDF. Like OWL it is a vocabulary sitting on RDF, but its role is different.

There is not much to it. Each item of the vocabulary is raised as an *individual* of class `skos:Concept` with an IRI, and named with labels. `skos:prefLabel` is the preferred form (one per language) and `skos:altLabel` the variants and alternative forms (any number). Concepts are joined by `skos:broader` (to a broader concept), `skos:narrower` and `skos:related` (associative), and the range of meaning is filled out with `skos:scopeNote`. The whole vocabulary is gathered into a `skos:ConceptScheme`. A small worked vocabulary shows all the parts (Figure 7.5 draws the same content).

```
vocab:parish  a  skos:ConceptScheme ;
    skos:prefLabel  "Parish records vocabulary"@en .

vocab:register  a  skos:Concept ;
    skos:prefLabel  "Register"@en ;
    skos:inScheme   vocab:parish .

vocab:parishRegister  a  skos:Concept , rico:DocumentaryFormType ; # both
    skos:prefLabel  "Parish register"@en ;
    skos:altLabel   "Register of baptisms, marriages and burials"@en ,
                    "Parish book"@en ;
    skos:broader    vocab:register ;      # to the broader concept
    skos:scopeNote  "A register of baptisms, marriages and burials."@en ;
    skos:inScheme   vocab:parish .

vocab:glebeTerrier  a  skos:Concept , rico:DocumentaryFormType ;
    skos:prefLabel  "Glebe terrier"@en ;
    skos:broader    vocab:register ;
    skos:inScheme   vocab:parish .

# the data side only refers to a concept as a type
ex:r45  rico:hasDocumentaryFormType  vocab:parishRegister .
```

The concepts are declared to be `rico:DocumentaryFormType` as well because that is the range of `rico:hasDocumentaryFormType`. Being a SKOS concept and being a RiC documentary form type are compatible, and one IRI carries both types.

What is easy to confuse with OWL is *what the hierarchy means*. `skos:broader` is not `rdfs:subClassOf`. "Parish register broader Register" is above and below on a map of subjects; it asserts no logical containment (that every instance of the narrower is an instance of the broader). A `skos:Concept` is an *individual*, a concept, not a class, and so has no instances at all (recall the punning example in Chapter 5, where `rst:Fonds` is an individual and also a SKOS concept). This lightness of semantics is deliberate, not careless. Being light is what allows an existing controlled vocabulary, such as a library subject heading list, to be published as RDF without dragging heavy logic in with it.

The rule of thumb is easy. *The structure of the world*, the kinds of entity and their relations, goes in OWL, which is what RiC-O is; *the map of a field's concepts*, the classification of things

and their subjects, goes in SKOS. Many SKOS vocabularies are already published and can be referred to by IRI, among them the Library of Congress Subject Headings and the Getty AAT (Art and Architecture Thesaurus). Check whether an existing published vocabulary will do before building one. Why an OWL vocabulary and a SKOS vocabulary can be mixed in the same data, namely that everything is RDF triples and vocabularies are of equal standing, was explained in Section 5.3.

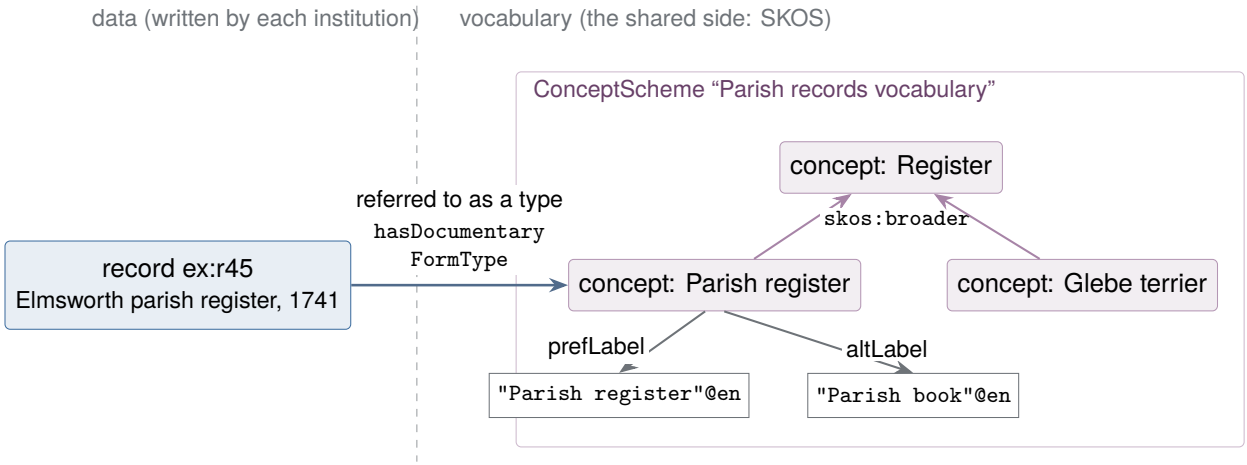


Figure 7.5: The structure of a SKOS vocabulary. A concept is an individual with an IRI, carrying labels (prefLabel, altLabel) and broader and narrower links, and the whole gathers into a ConceptScheme. The data side has only to refer to a concept as a type; variant forms of the name are absorbed by the vocabulary.

7.5.1 Choosing between the three

What decides is whether the knowledge is a particular fact, a system of concepts, or a kind of relation.

A particular fact goes in the data “This record was made in 1741”; “this performance took place out of doors”. These are facts about particular things and are written with the existing attributes and relations. A date in an unfamiliar calendar is the standard case, and the existing machinery covers it: `rico:expressedDate` for the date as the source expresses it, `rico:normalizedDateValue` for the normalised form. No new rule is needed.

A system of concepts is separated into a controlled vocabulary A list of documentary forms (parish register, glebe terrier, churchwardens’ accounts), a system of offices (churchwarden, overseer of the poor, parish clerk), a catalogue of works in a repertoire: these are not facts about particular records but a *map of concepts* shared across a field. Knowledge of this kind should not be written into RiC-O but built as a SKOS vocabulary of its own and referred to as instances of `rico:DocumentaryFormType` or `rico:ActivityType`, or as subjects. The gain from separating it is sharing. If several institutions refer to the same parish records vocabulary, a search across them becomes possible; and the vocabulary can carry labels in several languages and broader and narrower relations (a parish register is a kind of register), so that specialists in the field can develop the field’s knowledge themselves. The RiC-O side needs to know only that there is a type. Most domain knowledge, being classification of things, takes this road.

A kind of relation may justify an extension Only when the existing relations are too coarse, and a distinction you want for retrieval or inference is lost, is it time to consider extending. The official AG example is "has testator" for notarial documents: joining a will to a person with `rico:hasAuthor` alone makes "find the testator" unsearchable, so `hasTestator` is added as a subproperty of `rico:hasAuthor`. By the same judgement, "created as churchwarden" for parish records, or "gave the first performance of" in the performing arts (below `rico:performsOrPerformed`), become candidates. But an extension costs design, documentation and maintenance, and brings an obligation to follow revisions of RiC. Observe the practice set out in Section 6.7 (graft onto an existing component) and an institution that knows nothing of your extension can still search at the level of `rico:hasAuthor`; but the meaning of the extension itself reaches only those you share it with. So extend only when the meaning of the relation is genuinely needed, and publish what you make. In fairness, the arrangements to support that publishing do not exist. RiC-AG goes no further than saying that "it may be important in the future to maintain a registry of RiC-CM and RiC-O extensions", and for now the practice rests on goodwill and on adding an entry to the RiC Resource List. The next section takes that problem head on.

7.5.2 One example written three ways

The abstract statement is hard to hold on to, so here is one fact written in all three places. The material is a parish register: the register of the parish of Elmsworth, made in 1741 by Samuel Hart, churchwarden.

Written in place 1 First, as values of existing properties only.

```
ex:r45  a  rico:Record ;
    rico:name  "Elmsworth parish register" ;
    rico:type  "parish register" ;          # the documentary form as a string
    rico:isAssociatedWithDate  [ a rico:Date ;
        rico:expressedDate       "12 February 1740/1" ;  # as the source has it
        rico:normalizedDateValue "1741-02-12" ] ;        # normalised
    rico:hasCreator  ex:hart .

ex:hart  a  rico:Person ;
    rico:name  "Samuel Hart" ;
    rico:generalDescription  "Churchwarden of Elmsworth, 1738-1745." .  # prose
```

The facts survive. As with the date, where the two-part machinery of `rico:expressedDate` and `rico:normalizedDateValue` handles a form of dating the reader may not know, much is adequately served at this level. The weaknesses show, though. "Documents created by a churchwarden" cannot be searched; that knowledge is buried in prose. And searching by documentary form is string matching only, so another institution writing "register of baptisms" has written something else.

Written in place 2 The documentary form and the office are lifted out into a SKOS vocabulary *in a separate file*, and the data refers to it.

```
# --- the vocabulary (shared; developed by specialists in the field) ---
vocab:parishRegister  a  skos:Concept , rico:DocumentaryFormType ;
    skos:prefLabel  "Parish register"@en ;
    skos:altLabel   "Register of baptisms, marriages and burials"@en ,
                    "Parish book"@en ;             # absorbs the variants
    skos:broader    vocab:register .               # a kind of register

vocab:churchwarden  a  skos:Concept ;
```

```
    skos:prefLabel  "Churchwarden"@en ;
    skos:altLabel   "Church warden"@en , "Chapelwarden"@en ;
    skos:scopeNote  "A lay officer of a parish; in a chapelry, chapelwarden."@en .

# --- the data (written by each institution) ---
ex:r45  rico:hasDocumentaryFormType  vocab:parishRegister .
ex:cw45  a  rico:Position ;  rico:name  "Churchwarden of Elmsworth" .
ex:hart  rico:occupiesOrOccupied  ex:cw45 .
```

The *knowledge of the field*, that a chapelwarden is a churchwarden and that a parish book is a parish register, is written once, on the vocabulary side, and can be referred to from any institution's data. The vocabulary absorbs the variants, so a search for "parish book" lands on the same concept. The moment two institutions refer to the same vocabulary, a search across them works. And because the office has become a Position entity, that Hart was churchwarden is structured too.

Written in place 3 One thing still is not written: that Hart made this register *as churchwarden*, in the duties of that office. That is the meaning of the relation of creating. If it is wanted for retrieval or inference, extend.

```
# --- the extension (defined once, documented, published) ---
myo:hasCreatorAsChurchwarden
    rdfs:subPropertyOf  rico:hasCreator ;
    rdfs:label  "was created by, acting as churchwarden"@en .

# --- the data ---
ex:r45  myo:hasCreatorAsChurchwarden  ex:hart .
```

Now "every document created by a churchwarden" is one property away, and thanks to the roll-up of subproperties (Section 6.4) an institution that knows nothing of the extension still sees it at the level of `rico:hasCreator`. And where there is no such need, no concrete search or inference for which "created" is too coarse, stopping at place 2 is correct.

7.5.3 The order to try them in

The guidance follows. Try place 1 first (it will usually do). If what you have is a concept shared across a field, lift it out to place 2. Go on to place 3 only when the meaning of the relation is genuinely required, and even then, before building anything, check whether RiC-O already has it (its vocabulary is larger than the CM's) and whether it could be proposed to EGAD (as the FAQ recommends). The order is the principle of the smallest sufficient means, and also the principle that domain knowledge should be kept out of RiC-O proper and put somewhere shareable. RiC is the *common language* of archival description, not the encyclopaedia of every field. Separate a field's knowledge into a vocabulary or an extension and publish it, and it becomes an asset to other institutions; embed it in prose or in local rules and it stays inside your walls.

7.6 Who provides the shared infrastructure?

The guidance to "put it somewhere shareable" assumes something: that there is *infrastructure* to receive what is shared. At present that assumption largely fails. This section states the awkward fact, and then shows how far RiC-O gets without the infrastructure.

Table 7.2: Three places for domain knowledge

	1. In the data	2. Controlled vocabulary (SKOS)	3. Extension
Suits	particular facts	a system of concepts	the meaning of a relation
Change to RiC-O	none	none (reference only)	grafted on
Example	a date, a place of performance	lists of forms, offices, works	has testator; created as churchwarden
Cost	least	building and maintaining a vocabulary	design, documentation, keeping up with revisions
Shareable	poorly (prose cannot be reused)	well (the vocabulary is itself an asset)	if published

7.6.1 What is missing

Sharing extensions is not institutionalised. As the previous section showed, what the AG recommends stops at documenting, publishing and adding an entry to the Resource List. There is no mechanism for registering, reviewing, reconciling or versioning extensions, nothing answering to a package registry in programming or to vocabulary management in libraries. Where two institutions in the same field build separate extensions, no procedure exists for matching them up.

More serious is the shared infrastructure for identifiers. RiC's whole point, a web that crosses institutional boundaries, requires that the same thing be given the same IRI, or IRIs mapped to one another. Libraries have VIAF (an international hub gathering national author authority files) and ISNI (identifiers for people and bodies involved in creative works, ISO 27729). Archives have next to nothing equivalent. For creators there is SNAC, a cooperative authority project centred in the United States, agent authority data published by the Archives nationales de France, and Wikidata, which functions as a de facto hub (a shared structured data platform anyone can edit and refer to). None approaches VIAF in coverage or in international coordination. For the entities specific to archives (activities, rules, positions, and record sets themselves), which lie outside the scope of library authority work, shared identifiers are not even at the design stage. Each institution therefore mints IRIs on its own domain (Section 7.3), and the normal state of affairs is that one person or one organisation has as many different IRIs as there are institutions describing them. Mapping them afterwards with `owl:sameAs` or `skos:exactMatch` (declaring that two concepts in two vocabularies match) is technically possible, but the reconciliation itself is expensive and uncertain work, and who is to do it has not been settled.

7.6.2 How far RiC-O gets without it

The answer is: half way. Splitting the value RiC-O promises in two makes it clear.

The value that stays inside one institution (structured description, description in several dimensions, input separated from output, retrieval and inference within the institution) is available without any shared infrastructure. A dataset closed within one institution's own IRIs still yields every benefit of the inference of Chapter 6. The implementations in front today, the Archives nationales de France among them, in practice run at this level. The reasons for a single institution to adopt RiC-O exist independently of whether the infrastructure does.

The value of connecting across institutions (a web of context extending past institutional boundaries, interconnection as linked data) does not exist without shared identifiers or after-the-fact reconciliation. And the RiC sources do not solve this. Designing and maintaining IRIs is left to each institution (a restatement of the general principle of linked data), and the AG says no more than that institutions with authority records and controlled vocabularies in order are at an advantage.

So RiC supplies *a common vocabulary* (the same name for the same concept) and leaves *common reference* (the same identifier for the same person) to institutions. That is a boundary of scope rather than a defect of the standard, but the land outside the boundary is vacant.

There is a historical precedent. In libraries, too, cataloguing codes and MARC were standardised first, and decades passed before an international hub such as VIAF was actually running. Standard first, infrastructure afterwards (and made possible by the standard) is the usual order. Nothing guarantees the same order in archives, but the existence of RiC as a common vocabulary has satisfied one precondition for an archival VIAF.

7.6.3 What to do meanwhile

What can be done without waiting comes down to one thing: *mint your own IRIs, and record the mappings to existing external identifiers from the start*. For agents, where a Wikidata, ISNI or VIAF identifier exists, or one from a national name authority file, assert the mapping. For places, refer to an existing gazetteer or geographical authority. A mapping is a few triples of `owl:sameAs`, and whatever shared infrastructure eventually appears, those triples are the bridge to it. A set of isolated IRIs with no mappings, by contrast, will have the whole reconciliation to do when the day comes. In the terms of Table 7.2, this is investing in shareability ahead of the infrastructure that would reward it.

7.6.4 Aggregators, and what they do not fix

Portals that aggregate across institutions are the nearest thing to shared infrastructure that actually exists, and it pays to be precise about what they do and do not settle. Europeana is the largest example in Europe: material is contributed by institutions through national and domain aggregators, mapped into a common model (the Europeana Data Model, EDM), and published as linked open data. Comparable national and regional portals exist elsewhere, and archives are usually one contributing sector among galleries, libraries and museums.

Two things follow for RiC. The first is genuinely useful. Where an aggregator normalises the values of names, places and periods and gives each a URI, those URIs are candidates for exactly the mapping recommended above, alongside Wikidata and VIAF. Aggregation has, in effect, produced identifier hubs as a by-product.

The second is a limit. An aggregator's data model is built around the item as the unit of description, which differs from a RiC network of entities and relations in granularity and in purpose alike. Record resources, agents, dates and places map across after a fashion; activities, rules, mandates and positions have nowhere to go. Aggregation solved the sharing of *descriptions*. It did not solve the sharing of *identifiers for the entities specific to archives*, which is the gap this section is about.

The identifiers themselves are more fragile than they look. Institutions that assign a URI per record have sometimes said at the time that the URI was an identifier for access rather than a permanent name, and that continuity through system changes would need care. What follows is predictable enough: a generation of the system is replaced, the form of the URI changes, and some proportion of the old URLs stop working. What that shows is that an identifier is permanent only where permanence is a stated policy backed by an institution willing to carry it, which is the same conclusion this section reached from the other direction.

7.7 Hard cases: names, scripts and dates

The examples in the sources are mostly straightforward European material of the modern period. Two areas where they are not much help, and where RDF has settled conventions, are names in

more than one script and dates in more than one calendar. Neither problem was created by RiC.

7.7.1 Names in several languages and scripts

RDF literals take language tags (BCP 47), and several may hang from the same predicate.

```
ex:archive  rico:name  "Rossiyskiy gosudarstvennyy arkhiv"@ru-Latn ,
                       "Russian State Archive"@en .
# the form in Cyrillic script would carry the plain tag @ru
```

The subtag after the hyphen names the script, so `ru-Latn` marks a romanised form of Russian and a bare `ru` the form in Cyrillic. The same applies to `ja-Latn`, `el-Latn`, `ar-Latn` and the rest. A catalogue in a language written in a non-Latin script commonly has to carry a transliteration, and sometimes a phonetic reading distinct from both, and this is the cheap way to hold them.

Where the kind of a name (real name, pen name, name in religion, married name, transliteration) or the period of its use has to be managed, there is the exact way: raise the name itself into an entity. RiC-O provides the `rico:AgentName` class.

```
ex:person12  a  rico:Person ;
    rico:hasOrHadAgentName  ex:name12a , ex:name12b .
ex:name12a  a  rico:AgentName ;
    rico:textualValue  "Mary Ann Evans"@en ;
    rico:type  "name at birth" .
ex:name12b  a  rico:AgentName ;
    rico:textualValue  "George Eliot"@en ;
    rico:type  "pen name" .
```

Choosing between the shortcut and the exact form goes as it did for dates and types (Section 5.5).

7.7.2 Variant spellings and forms of a character

Variation in how a name is written (Smith and Smyth, Elmsworth and Elmesworth, a place spelt three ways across two centuries of registers) is *not solved by standardising the string*. The RiC way is to solve it by identity of IRI. There is one entity, and the written forms are absorbed as multiple names (the `rico:AgentName` entities above, or `skos:altLabel` in a vocabulary). Not having to rely on string matching for identity is the direct return on turning references into IRIs (Section 2.4).

Alongside that, normalise Unicode (to NFC) when data is loaded. Characters that look identical but have different code points cause search failures that nobody notices: precomposed and decomposed accented letters, a long s transcribed as itself, ligatures, the several dashes. Treating historical and modern forms as equivalent at search time (finding the archaic spelling from the modern one) is the work of the index and application layer, not of the data.

7.7.3 Dates in another calendar

Where the two-part treatment of dates proves its worth is material dated in a calendar other than the one the catalogue uses. Write the source's own form in `rico:expressedDate` and a machine-readable value in `rico:normalizedDateValue`. RiC-O defines the latter as an expression in a standard format that a program can process, and its scope note names ISO 8601 and its extension **EDTF** (Extended Date/Time Format, standardised as ISO 8601-2). EDTF can write the uncertain dates that cataloguing constantly produces: `1741?` for uncertain, `1741~` for approximate, `174X` for a decade, and `1948-04/1949-03` for a span.

```
ex:r45  rico:isAssociatedWithDate  [ a rico:Date ;
    rico:expressedDate       "12 February 1740/1" ;
```

```
    rico:normalizedDateValue "1741-02-12" ] .

ex:f112  rico:isAssociatedWithDate  [ a rico:Date ;
    rico:expressedDate       "financial year 1948-49" ;
    rico:normalizedDateValue "1948-04/1949-03" ] .
```

Three kinds of difficulty recur, and all three are reasons to keep the expressed form.

The year did not always begin on 1 January. In England and its possessions the legal year began on 25 March until 1752, so a document of what we would call February 1741 was dated 1740 at the time. The convention in transcription is the double year, "12 February 1740/1", which is what the example above holds in `rico:expressedDate`. Similar conventions exist wherever the start of the year has moved.

The calendar itself changed. The Gregorian reform was adopted at different dates in different countries, spread over centuries, so two documents bearing the same date may be days apart in fact, and correspondence between two jurisdictions may bear two dates. Where the material is dated in a lunisolar calendar, or in a calendar with intercalary months, normalising below the level of the year requires real conversion, and an approximation to the year is often the sensible stopping point. Where a calendar was replaced entirely, as the French Republican calendar was, a conversion table is needed and its edge cases need checking.

Dates were expressed by something other than a number. A regnal year ("in the fourteenth year of the reign of…"), a feast day ("Michaelmas", "Lady Day"), a term of court, an era name: all of these require reference to a table before they become a number, and the table sometimes has more than one reading. Normalising destroys the information about which system the source used, and that information is frequently what tells a reader where a document came from. Keeping the expressed form means the normalisation policy can be changed afterwards, which is the point of holding both.

7.7.4 Connecting to external authorities

For people and corporate bodies, VIAF, ISNI and the national name authority files publish authority data with IRIs. Building a bridge with `owl:sameAs` from your own Agent entities to those is what "what to do meanwhile" (Section 7.6.3) amounts to in practice.

7.8 Personal data and access restriction

The more precise a RiC description becomes, the more structured data about individuals it holds. Worse, the mechanisms this book has been recommending (identity by IRI, inverse properties, transitive closure) are also risk factors for privacy, because information that was effectively invisible for being scattered gets gathered by inference and search. Data published as linked open data is copied and harvested, and withdrawing publication does not really work. So "how much to write" and "how much to publish" have to be designed as separate questions.

There are four tools. The first is *describing the restriction*: the shortcut is prose in `rico:conditionsOfAccess`, the exact way is to make the governing regulation a `rico:Rule` entity joined with `rico:isOrWasRegulatedBy` (so that a change to the regulation is one update). The second is that *publication is controlled by dividing the graph* (Section 7.4.4). Keep an internal graph and a published graph, and build the published graph as a *product* of the internal data: select what may be published and emit it, rather than writing everything into one graph and then hiding parts. Describing a restriction (now machine-readable) and enforcing it (implementing access control) are different things, and the second is the system's work, through permissions at the level of named graphs. The third is to *keep entities for living people minimal*: a name and a role, or stopping at the Position (`rico:Position`) without raising the individual at all (the layers of Section 4.5 help here; note that the work history of the describing archivist is personal data too, and deserves the same

care). The fourth is that requests for deletion are met, by linked data convention, by keeping the IRI and emptying the content (a tombstone). Deleting the IRI takes external data that referred to it down as well.

Where an access regime exists in law, being able to describe its categories of restriction as machine-readable `rico:Rule` entities is a practical contribution RiC can make. Once the correspondence between the governing regulation and the records it covers survives as a graph, both checking the consistency of review decisions and detecting the expiry of a restriction (a query, as in Chapter 6) become things that can be automated.

7.9 Retention, appraisal and disposal

Everything in this chapter so far has concerned records already in an archive. But the life of a record begins before transfer. Setting a retention period, appraising, and carrying out the disposition when the period expires (transfer or destruction) are the core of records management while records are still current, and what arrives at the archive is decided there. RiC-CM places its primary audience in the archival community while stating that it intends to be of interest to records managers as well (§1.3), and the AG gives one whole FAQ (§6d) to this subject. Mandate (E17), defined but not used in Section 4.2, does its first work here. Rule (E16) has already been used in Section 4.5 and Section 7.8, and RiC-CM itself supplies a precedent for writing retention as a rule: in the chapter on describing description it gives a figure in which a rule, a records management policy, is expressed by a record, a retention schedule (§6.2, Figure 4).

7.9.1 Make the process an activity

The skeleton the AG offers is simple to the point of being anticlimactic, and general in the same measure. Raise the process as one *Activity* and surround it with four relations. What the process affects is `rico:affectsOrAffected` (inverse `rico:isOrWasAffectedBy`); the decision or product is `rico:resultsOrResultedIn`; who carries it out is `rico:isOrWasPerformedBy`; when is `rico:occurredAtDate`. The range of the first is Thing in general, so what is affected may be a record resource or an instantiation, which matters later. There is also the option of placing a particular activity as a subevent (`rico:isOrWasSubeventOf`) of a general one. The AG takes that route in its appraisal example, and says that doing so builds a bridge, by way of a rule, to the act of entering the result of the appraisal in a register. The AG offers this as an *alternative* to giving the activity a type.

The same pattern serves for digitisation as for appraisal, and the AG says it generalises to many other kinds of process (§6d.1). Put the other way round, RiC has *no* dedicated machinery for appraisal. None of the 42 attributes is a retention period, and for destruction there is only `rico:hasDestructionDate`, joining a date (with the shortcut `rico:destructionDate` where no date entity is raised). Describing a process means assembling it yourself from the general activity pattern. RiC-O ships controlled vocabularies for two things only, record set types and documentary form types. No controlled values for activity type (the value of `rico:hasActivityType`) come from RiC, so an existing standard vocabulary has to be used, or one built.

7.9.2 An example from Belgium: the register entry as the warrant for destruction

What AG §6d.3 offers is an example from a records management agency in the Flanders region of Belgium, discussed on the RiC mailing list[1]. An agency appraises a series R, and the result is

[1] Flanders (*Vlaanderen*) is the Dutch-speaking region of northern Belgium, with its own parliament and government. On the obligation to enter results in the register, described below, the AG says only that it is required by law and does not name the legislation.

a decision that, say, ten years hence a file F within the series is to be destroyed. R is entered in a government register, and recording the results of appraisal (including the destruction of F) in that register is a legal requirement. Figure 7.6 shows the structure.

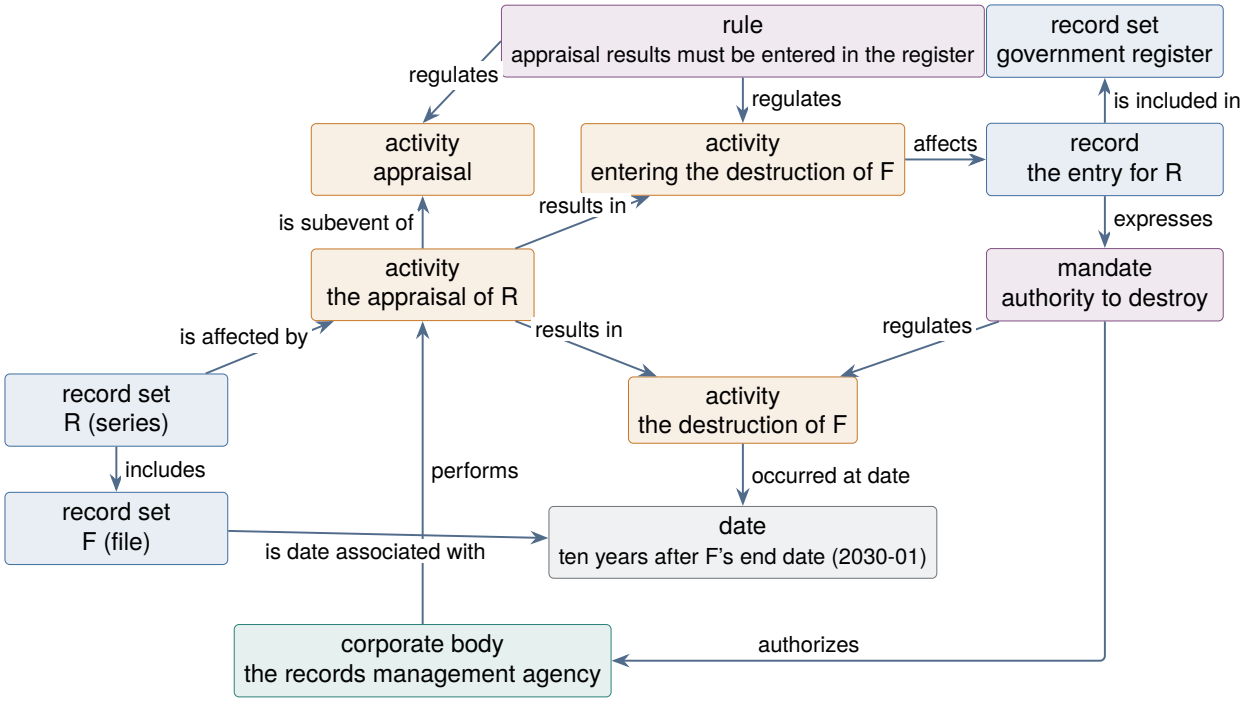


Figure 7.6: The structure of the Belgian example, from the diagram in AG §6d.3 redrawn in the notation of this book. Entity types and attributes (record set type, normalised date value), the end date of the file, the relation naming the records-management organisation as manager, and two relations by which the agency also performs the entering and the destruction, are omitted to keep the drawing legible. The authority to destroy is written in three steps: the register entry *expresses* a mandate, and that mandate governs the activity of destruction and authorises the agency.

What the AG itself calls the key point is that appraisal is formulated as an activity and surrounded by the relations of the previous section, "entirely analogously" to digitisation. The AG names three of them there, two in the inverse form (`rico:isOrWasAffectedBy` and `rico:performsOrPerformed`, alongside `rico:resultsOrResultedIn`); the date is absent from that list, though it appears in the diagram. Direction does not matter, since these are inverse properties. On top of that, the AG says that the register entry *can be regarded as* giving the mandate for destruction, and the reservation should be kept. The permission to destroy is tied to the appraisal result being entered in a register required by law. The AG adds that in this case the agency that appraised is also likely to carry out the disposition itself; in the diagram, all three activities of appraising, entering and destroying are performed by the same body.

Look closely at the figure and the "bridge by way of a rule" becomes visible. The rule that forms the bridge is *the obligation to make the entry*. The requirement in law that appraisal results be entered in the register is raised as one rule, and it governs both the general activity of appraisal and the particular activity of entering the destruction of F. Placing the particular appraisal as a subevent of the general one was what let the rule reach it. The authority side runs in three steps: the register entry, a record, *expresses* a mandate; that mandate governs the *activity* of destroying F, and authorises the agency. In this diagram what the mandate targets is an activity and an agent, not a record.

The AG models the register as a record set and the entry about R as a record. That follows from the interest lying on R's side; make the register itself the subject, as the record of appraisal in

that region, and the division into record set and record comes out differently (AG §6b discusses judgements of this kind at length).

Two technical notes. `rico:hasDestructionDate` is a subproperty of `rico:hasEndDate`, with record resources and instantiations as its domain and dates as its range. The AG says that, if RiC-O is being used, it would be "a bit more precise" than writing *is date associated with*. That is R068, which takes a date as its domain and corresponds to `rico:isDateAssociatedWith`; the inverse, from a thing to a date, is `rico:isAssociatedWithDate` (R068i), which is what the examples in this book use. The diagram itself uses the general date relation, and `rico:hasDestructionDate` appears only as a suggestion in a note, because a diagram drawn in the vocabulary of RiC-CM and the choices made when implementing in RiC-O are different things. In the AG diagram the label on this arrow is the R068 form, while the arrow runs from the record set to the date. Being inverse properties, nothing turns on this, but it is a reminder that a label and an arrow need not line up. The property was added in RiC-O 1.0 and has no counterpart in RiC-CM: for all that the previous section said about there being no dedicated machinery, the ontology got there first on the date of destruction. On the same reasoning, the example below writes the expiry of the retention period with `rico:hasEndDate` (a subproperty of `rico:isAssociatedWithDate` and the parent of `rico:hasDestructionDate`). Choose the narrowest relation available. The other note is that *a future date may be written*. That some relation names carry a past form is a habit of RiC naming, not a constraint, and the AG observes that these forms may change in a future version, with a first step taken in RiC-O 1.1.

7.9.3 The general pattern

Regimes for retention and disposal differ from country to country. The shapes they take are broadly similar, though, and it is the shape rather than any one statute that repays modelling. Many regimes contain most of the following.

- A *retention period* attached to a class of records, usually by a schedule that is itself a published document.
- A *date of expiry* computed from the period, often from a cut-off that is not the date of the record itself but the end of the year or the financial year in which the file was closed.
- A *disposition* decided for expiry, typically transfer to an archive or destruction.
- An *entry in a register* recording all of this, in some jurisdictions open to public inspection.
- Sometimes, an *authorisation* without which destruction may not proceed: the consent of an archival authority, or of a supervising body.

Written in RiC, each of those is an entity, and the important move is the first one: *a retention period is not an attribute of the record but a rule that governs it*.

```
# the retention period is a Rule governing the file, not an attribute of it
ex:retentionA12  a  rico:Rule ;
    rico:name  "Retention period for file A-12 (10 years)" ;
    rico:regulatesOrRegulated  ex:fileA12 ;
    rico:hasEndDate  [ a rico:Date ;                # the date of expiry
                       rico:normalizedDateValue  "2037-03-31" ] .

# the activity that decides the disposition
ex:appraisalA12  a  rico:Activity ;
    rico:name  "Decision on the disposition of A-12 at expiry" ;
    rico:isOrWasSubeventOf  ex:appraisalGeneral ;  # the general activity
    rico:hasActivityType  ex:atAppraisal ;
    rico:isOrWasPerformedBy  ex:agency ;
    rico:occurredAtDate  [ a rico:Date ;
```

```
                            rico:normalizedDateValue  "2027-04-01" ] ;
    rico:affectsOrAffected  ex:fileA12 ;
    rico:resultsOrResultedIn  ex:dispositionA12 .
```

What the activity produced is itself a rule, and what that rule governs is not the record but the future activity of destroying it. The register entry is then *a record that expresses* the rule, joined with `rico:expressesOrExpressed`. As Chapter 4 warned, a rule and the document evidencing it are different entities.

```
ex:dispositionA12  a  rico:Rule ;
    rico:name  "A-12 is to be destroyed on expiry of its retention period" ;
    rico:issuedBy  ex:agency ;
    rico:regulatesOrRegulated  ex:destroyA12 .

# the register: a record set, whose members are the entries
ex:fileRegister  a  rico:RecordSet ;
    rico:name  "Register of files" ;
    rico:includesOrIncluded  ex:registerEntryA12 .

ex:registerEntryA12  a  rico:Record ;
    rico:name  "Register entry for A-12" ;
    rico:expressesOrExpressed  ex:retentionA12 , ex:dispositionA12 .
```

One entry expresses both the retention rule and the disposition rule. The division into a record set and records here follows the Belgian example, and for the same reason: the interest lies with the individual file. Make "how this body manages its records" the subject instead, and the register itself becomes the protagonist and the cut comes out differently.

What remains is the destruction. It has not happened, and it is raised as an activity all the same, so that the authorisation can be attached to it. Where a regime requires the consent of another body before destruction, that consent is written as a Mandate, the subentity of Rule, and the body it names as acting is given with `rico:authorizes`.

```
# the destruction (which has not yet happened), and the authority for it
ex:destroyA12  a  rico:Activity ;
    rico:name  "Destruction of A-12" ;
    rico:isOrWasPerformedBy  ex:agency ;
    rico:affectsOrAffected  ex:fileA12 .

ex:consentA12  a  rico:Mandate ;                    # a subentity of Rule
    rico:name  "Consent to the destruction of A-12" ;
    rico:issuedBy  ex:supervisor ;
    rico:authorizes  ex:agency ;
    rico:regulatesOrRegulated  ex:destroyA12 .

ex:fileA12  a  rico:RecordSet ;
    rico:hasRecordSetType  rst:File ;
    rico:identifier  "A-12" ;
    rico:hasDestructionDate  [ a rico:Date ;        # a future date is allowed
                               rico:normalizedDateValue  "2037-04" ] .
```

A caution about `rico:authorizes`. Its definition is the agent that a mandate gives authority to act, and a consent of this kind is often not a grant of authority at all: where the duty to dispose is laid on the holding body directly by law, the source of the authority is the law, and the consent is a negative control on its exercise. Writing it as a mandate stretches the definition. It is defensible, because the relation "this activity cannot proceed without an act by another agent" is exactly what

needs to be recorded, but the stretch should be conscious, and using `rico:regulatesOrRegulated` alone, without `rico:authorizes`, is the safer choice.

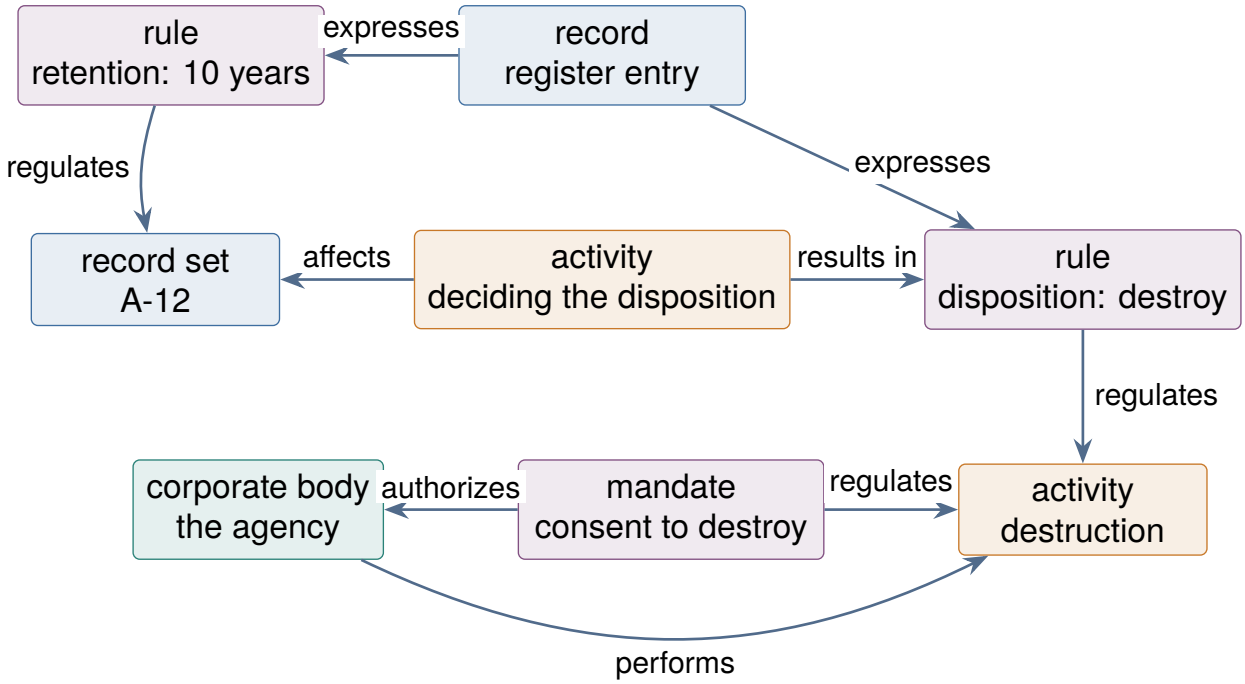


Figure 7.7: Retention and disposal. Neither the retention period nor the disposition is an attribute of the record: both are rules, and what authorises the activity of destruction is a separate mandate. The register entry is placed as a record expressing the rules.

What this graph makes possible can be said in the vocabulary of Chapter 6. Detecting expiry is not inference but a query comparing dates (`FILTER`, Section 6.8.3). But questions such as "of those that have expired, separate the ones marked for transfer from the ones marked for destruction", "list the destructions scheduled for which consent has not been obtained", and "list every record governed by this rule" can be written straight into SPARQL, provided the disposition is entities and relations rather than a paragraph in a field. Aggregating across an organisational hierarchy brings transitivity into play as well. So long as this information stays in a free-text column of a register, it can only be counted by eye. Those three queries are included, in runnable form, in the sample data distributed from this book's website (Section 7.12).

The pattern travels; the content does not

The AG example rests on the law of one region of Belgium, and that should not be forgotten. Reading a register entry as the mandate for destruction works in a regime where the entry is a legal requirement. Under another regime, which document generates which authority will be different: the schedule itself may carry the authority, or an archival authority may have to approve each disposal, or the decision may rest entirely with the creating body. The modelling pattern is shareable. What goes into the pattern has to be read off each regime separately, and the cautions about `rico:authorizes` above apply with it.

7.9.4 Digitisation takes the same pattern

The AG treats digitisation alongside appraisal in §6d to show that the same pattern serves both. Digitisation has one point of its own, though: it is written as an activity affecting not the record resource but *the instantiation*. The note at AG §6d.2 gives the reason (the background argument

is at §6c.2). The intellectual side of a record resource, the information it holds, is not essentially changed by digitisation. Digitisation moves one inscription of that information into another. The effects, that it becomes more widely usable or more robustly preserved, concern how a particular inscription of the information is used; in the first instance at least, the information itself is not affected. The reason for distinguishing record from instantiation in Chapter 3 has its practical consequence here.

There is a second thing to notice in the AG example. It gives "scanning" as the single activity type (the value of `rico:hasActivityType`) and, separately, raises *both* digitisation and scanning as activities in their own right. Activity type is an attribute for classification, and RiC-CM makes it non-repeatable (RiC-A02); from that the AG reads an expectation that the narrowest available classification is chosen, and its diagram does so. Meanwhile, having both digitisation and scanning as activities lets you write that one is a special case of the other, and lets a particular digitisation be connected to a much wider context. The AG says that this second point is the more important, because relations and attributes can then be extended from either the digitisation side or the scanning side. It is an instance of the RiC habit of writing the classification and the individual separately.

```
ex:digitization2024  a  rico:Activity ;
    rico:name  "Scanning of the Elmsworth parish register" ;
    rico:isOrWasPerformedBy  ex:archivesOffice ;
    rico:occurredAtDate  [ a rico:Date ;
                           rico:normalizedDateValue  "2024" ] ;
    rico:affectsOrAffected  ex:paperInst ;    # the original, an instantiation
    rico:resultsOrResultedIn  ex:digitalInst .
```

On the record's side, all that appears is two relations to two instantiations. Nothing written about the record itself changes across the digitisation, not one character.

```
ex:r45  a  rico:Record ;                      # the record itself is unchanged
    rico:hasOrHadAnalogueInstantiation  ex:paperInst ;
    rico:hasOrHadDigitalInstantiation    ex:digitalInst .
```

The two properties used just above, for an analogue and a digital instantiation, are subproperties of `rico:hasOrHadInstantiation`, and the AG notes that they complement the Representation Type attribute. Being subproperties, they leave anyone querying on `rico:hasOrHadInstantiation` free to ignore the distinction (the roll-up of Section 6.4).

Digitisation does sometimes produce a new record. The AG's case is a digitised instantiation in a form that admits annotation, which is then annotated. The test for that judgement is discussed at §6c.3, and §6d.2 gives a separate diagram for it. "Digitisation affects the instantiation" is the default reading, not a claim that there are no exceptions.

7.10 Scenario B: a workflow for describing natively

Describing natively in RiC changes the unit of work from "finish writing one finding aid" to "register entities and assert relations". Restated in the language of entities, accession to publication runs like this.

1. **At accession**, register the transferring body (Agent), the transfer as an activity (Activity or Event) and the body of records received (Record Set), and join them with relations. A coarse description, one record set and a handful of relations, is enough at this stage.
2. **During processing**, build the arrangement (series, files) as nested record sets, add as entities the creators, activities and places that emerge from the content, and assert the relations.

Where an existing authority (another institution's, a national one, Wikidata) already holds the same entity, refer to it rather than making your own, or assert the correspondence.

3. **At each point of description**, the system should ideally record automatically who described what, when and on what basis (describing description, RiC-CM chapter 6).
4. **Publication** is the generation of output from the accumulated network of entities and relations. A conventional finding aid can be generated as one of those outputs.

The level of a first attempt is set out in the AG's *Getting started*. RiC has no required elements, but most entities are more useful with a name and an identifier; the six elements ISAD(G) made essential (reference code, title, creator, dates, extent, level of description) are a sound starting point in RiC too; and prose such as scope and content, or history, can be carried across as it stands, with structured relations for subjects and events *added* afterwards. A worked conversion of a real catalogue from The National Archives of the UK into RiC-CM, with a list of the entities, attributes and relations used, is included, and can be worked from directly.

The change of mindset that matters is that *description stops being finished*. In place of a completed finding aid there is a network that grows and is revised continually, and output is a snapshot of it at a moment. Versioning and revision history (themselves description of description) become correspondingly more important.

7.11 The toolbox

The main tools usable in practice as of 2026 (see also the RiC Resource List, maintained by the community).

- **Reading the ontology**: the official RiC-O HTML documentation and the CSV lists. For serious reading, open the RDF file in Protégé (the free ontology editor from Stanford) and follow the hierarchy and the axioms interactively.
- **Conversion**: RiC-O Converter (from EAD and EAC-CPF; version 3.0.0 is published, with a record of converting over 30,000 EAD finding aids of the Archives nationales de France into a RiC-O 1.1 dataset), rdflib (Python), XSLT. For defining the mapping, the official crosswalks accompanying RiC-AG are the starting point.
- **Storage and query**: a triplestore. GraphDB (commercial, with a free edition, strong on inference) and Apache Jena Fuseki (open source) are the standards. Both provide a SPARQL endpoint.
- **Validation**: a SHACL processor (built into GraphDB; pySHACL and others).
- **Visualisation**: there is no visualisation tool specific to RiC, so general RDF tools are used according to purpose. For the ontology itself, WebVOWL (a web tool that draws classes and properties from an OWL file) or the visualisation plugins of Protégé will show the structure of RiC-O. For data, the Visual graph feature of GraphDB (displaying SPARQL results or a neighbourhood as an interactive graph) and linked data browsers such as LodView and LodLive (following IRIs and visualising the neighbourhood) are the easy options. As a query interface, Sparnatural (which builds a query visually, without writing SPARQL) has been demonstrated over RiC-O data in the work around the Archives nationales de France. One caution: an archival graph is large, and drawing the whole of it on one screen is of no use. Choosing a starting point and showing a few hops around it, which is what the figures in this book do, is the practical mode. The RiC Resource List has the current state of tooling.
- **Description systems**: docuteam context (Switzerland; an archival information system built on RiC-O as its metadata standard) and a few others are appearing. On RiC support in AtoM, the leading open-source cataloguing system, the RiC-AG FAQ is candid: software development is outside EGAD's remit; the relational database on which AtoM rests sits awkwardly with RDF

and OWL data formats; and a set of APIs (interfaces through which a program calls functions or data) may be more fruitful, in the open-source software of this field, than a full application. Check the Resource List for the current position before making a decision.

Until the tools arrive: writing RiC in a spreadsheet

Many readers will have had the same thought. An ISAD(G) catalogue could be written in a spreadsheet without any specialist system: one row per unit of description, one column per element, and the hierarchy expressed by a "parent identifier" column holding the reference code of the level above. With RiC tools still thin on the ground, the same approach still works. A traditional register looks like this.

Reference code	Title	Dates	Level	Parent
P-83	Public Health Dept records	1948–1994	Fonds	
P-83/1	General administration	1948–1994	Series	P-83
P-83/1/12	Immunisation programme	1948-49	File	P-83/1
P-83/2	Communicable disease statistics	1948–1994	Series	P-83

Why this worked needs establishing. In a tree each node has *exactly one* parent, so the connection could be recorded in *one column*. A row per entity, a column per attribute, references by identifier: in the terms of Chapter 2, the spreadsheet register was already a respectable tabular description of a graph. What RiC changes is the relations. They become many-to-many, typed and dated, and *they no longer fit in one column*.
The answer is not more columns but *more sheets*. One sheet per kind of entity (record sets, agents, places), a row per entity, an identifier column first. And relations lifted out into a *relations sheet*, a row per relation, with columns for source identifier, type of relation, target identifier, start, end and evidence.

Record sets sheet

ID	Type	Name	Directly included in
P-83	Fonds	Public Health Dept records	
P-83/1	Series	General administration	P-83
P-83/1/12	File	Immunisation programme	P-83/1

Agents sheet

ID	Class	Name
AR-0125	CorporateBody	Public Health Department
AR-0230	CorporateBody	Health and Welfare Department

Relations sheet

Source	Type of relation	Target	Start	End
P-83	hasOrganicProvenance	AR-0125	1948	1994
AR-0125	hasSuccessor	AR-0230		

Many-to-many is now just more rows. And look at the shape of the relations sheet: source, type, target, three columns *stacked*, which is the table of triples from the box in Chapter 5. As some readers will have seen, it is also the junction table of a relational database, the relation entity of RiC (Section 5.5.3), and the spreadsheet version of the "separate relation graph" of Section 7.4.4. (A plain hierarchy, where the parent is unique, can stay as one column on the entity sheet, as above.) Most institutions' registers have in any case grown ad hoc multiple relations already, in the shape of a creator column or a related-material column, and moving

those into a relations sheet is nothing more than normalisation. A spreadsheet of this form works perfectly well as an *editing front end* for a graph.
Five disciplines make it convertible by machine. One row per entity and per relation, one value per cell (do not pack several values into a cell with commas; add rows). No meaning in appearance (merged cells, colour and indentation are lost in conversion). An identifier on every row, and references by identifier rather than by name. Fixed column names, with a table of "column name to RiC-O property" on the first sheet (which is a small application profile in itself). And care with dates, since spreadsheets convert them silently; hold anything unusual as a string (Section 7.7).
With that in place, conversion to RiC-O is mechanical, with a short rdflib script or with OpenRefine (a free tool for cleaning and reconciling tabular data); the exercise in the next section is exactly this. And when description systems do mature, a register kept with this discipline will migrate at the least possible cost, which is scenario C, the discipline that prepares for later, practised on a spreadsheet. What has been lost is only the luck of the tree fitting in one column. The row-by-row way of working is compatible with RiC.

7.12 A first step, as an exercise

For learning purposes no institutional plan is needed. Try this.

1. Take a body of records near at hand (seminar papers, family documents, a society's box of files) and pick out about ten entities: two or three record sets, two or three records, two or three people or bodies, one or two activities, and any rules or places.
2. Draw the graph on paper (imitating the figures in this book is fine). Choose the relations from the list in RiC-CM §5.6.
3. Write it out in Turtle (using the examples of Chapter 5 as a template; `ex:` will do for the IRIs).
4. Load it into a free triplestore (Fuseki) and use SPARQL to produce "the records in this set" and "the entities this person is involved with".
5. If you have the appetite, enable inference in GraphDB and confirm that triples you did not write (inverses, transitive closure) appear in the results.

One turn through that will make Chapters 4 to 6 something done with your own hands, and leave you with the feeling that describing in RiC means making data of entities and relations.

If starting from a blank page is too much, begin instead from the sample data published on this book's website[2]. It contains the examples from this book in loadable form: the worked example of this chapter, the parish records of Section 7.5, the retention and disposal of Section 7.9, the genealogical example of Chapter 6 with fourteen SPARQL queries, and the shapes and test data of Section 8.4. Instructions for running it under Fuseki and pySHACL are in the accompanying `README.md`. It carries the same CC BY 4.0 licence as the book, so it may be adapted for teaching. The included `check.py` verifies mechanically the syntax of every file, the existence of the `rico:` names used, domains and ranges, the number of rows each query returns (with and without inference), the SHACL report, and agreement with every Turtle listing in the book.

What is still missing

The publication of the AG draft advanced practical guidance considerably. Even so, the AG describes itself as a draft and a living document and is soliciting feedback, and it deliberately

[2]The archive expands into a directory containing the examples. The RDF and CSV of RiC-O 1.1 are included, so inference and validation run without anything further.

declines to be a step-by-step guide. A table of answers, on the granularity of record sets or on a minimum set of required attributes, is not to be expected; what the AG gives is examples and material for judgement. The AG also acknowledges that its examples are weighted towards UK, Scandinavian and Spanish material, so applying them elsewhere calls for testing. The procedure in this chapter is a reconstruction from the AG and from early adopters as things stand, and should be updated as the AG is revised and as national guidance appears. For a learner, the durable investment is not memorising settled answers but learning to tell where the official documents stop and the implementer's judgement begins.

Exercises (hints in Appendix A.3)

7.1 Design, as a list, the minimum application profile for a small imaginary archive: which entities are used, which relations are reified, and the IRI policy.

7.2 Convert by hand into Turtle the series "Communicable disease statistics" from the EAD fragment in Section 7.4, including its relation to the parent fonds.

7.3 When authority data and catalogue data are divided into named graphs, in which graph do the relation triples that cross the boundary belong? Decide, and give your reasons.

7.4 Write the date "financial year 1933-34" in both forms, the expressed form (`rico:expressedDate`) and a machine-readable EDTF value (`rico:normalizedDateValue`).

7.5 Choose three of the tools in Section 7.8 and use them to state a policy for publishing, as linked open data, a body of records created by a living person.

Chapter 8

How Much Technology Do You Need?

"Do you have to be able to program to do RiC?" is the question that comes up most, and this chapter answers it. The short answer is that it *depends on the role*. What everyone needs is an understanding of the concepts; implementing the technology can be divided up. For the division to work, though, enough of a shared language is needed to talk across the boundary. This chapter is an inventory of that shared language.

8.1 What each role needs

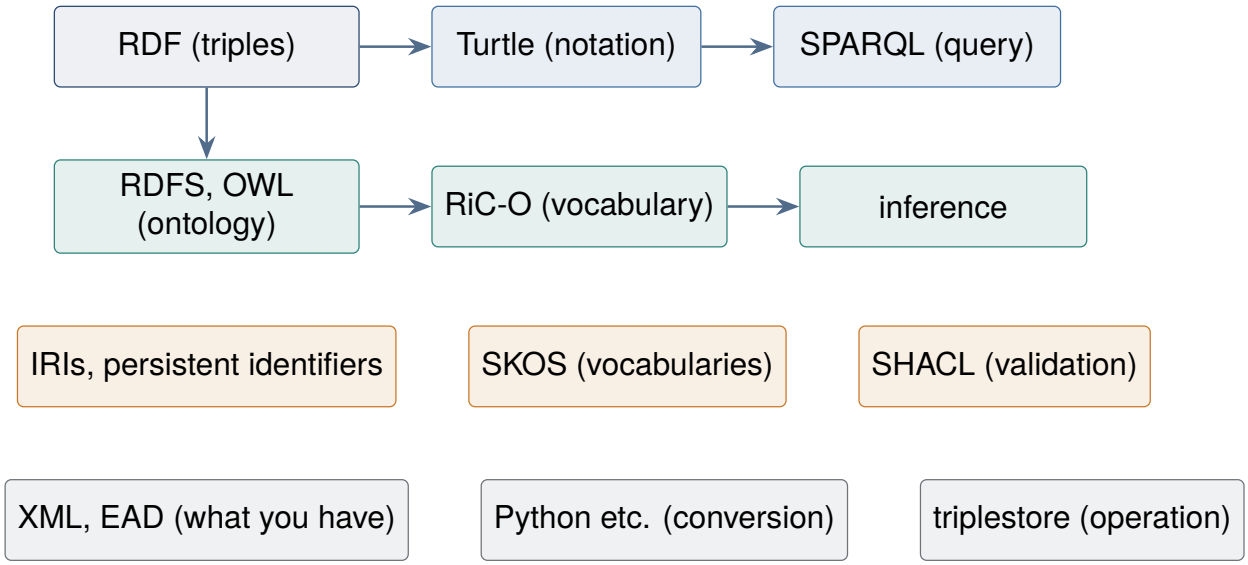
Table 8.1: A guide to technical grounding by role (● = learn it, ○ = be able to read and use it, · = know roughly what it is, — = not needed)

Technology	Descriptive archivists	Data modellers	Developers	Managers
RiC-CM concepts (entities, relations)	●	●	○	○
Reading Turtle	○	●	●	·
Writing Turtle	·	●	●	—
SPARQL (reading, simple queries)	○	●	●	·
Reading OWL axioms	—	○	●	—
XML and EAD	○	●	●	·
Designing IRIs and persistent identifiers	·	●	●	○
Running a triplestore	—	·	●	·
Scripting (Python and the like)	—	○	●	—
Validation with SHACL	—	○	●	—

Look at the column for archivists who write description. Almost none of the writing technology is needed. What is needed is to understand the conceptual model, to be able to *read* the data (Turtle) and the queries (SPARQL) that the system produces and know what they mean, and to be able to picture how a descriptive judgement (record or record part, raise it to an entity or not) will show up in the data. EGAD says as much itself: the complexity is for the user interface to mask (RiC-CM §1.3). The data modeller, on the other hand, the person who builds the institution's application profile and defines the mappings, needs both reading and writing. This intermediate role, at home in archival science and in information technology, is where a RiC implementation is won or lost.

8.2 A map of the technologies

The technologies of Chapters 5 to 7, arranged with an eye to the order of learning them.



top row: the basis of the data; middle row: the layer of meaning; bottom row: conversion and operation. Arrows show roughly the order of learning.

Figure 8.1: A map of the technologies behind a RiC implementation. All are W3C standards or widely adopted technologies; nothing here is peculiar to archives.

Briefly, what each amounts to, and where it touches archival work.

RDF and Turtle Chapter 5. That the atom of data is subject, predicate, object, and that things are named with IRIs. With those two understood, the details of the syntax can be looked up as needed.

SPARQL The query language for RDF. It looks like SQL (`SELECT ... WHERE`) but matches patterns in a graph rather than rows in a table. An archivist who can read it will find looking things up at a public endpoint, and explaining a requirement to a developer, very much easier.

RDFS and OWL The languages on the defining side. Using RiC-O needs no depth here, but five terms are the minimum for following the inference of Chapter 6: `subClassOf`, `domain` and `range`, `inverseOf`, transitivity, property chains.

IRIs and persistent identifiers The ground of linked data. This includes the thinking behind persistent identifier schemes such as DOI and ARK, and the operational discipline that a published identifier is neither changed nor deleted. It is institutional design rather than technology, and continuous with the archivist's professional competence.

SKOS The standard for expressing a controlled vocabulary in RDF. The instances of the RiC-O `*Type` classes are meant to be developed as SKOS vocabularies (the official individuals such as Fonds are SKOS concepts as well). Experience of managing subject headings or material types transfers directly.

SHACL The constraint language for checking that data satisfies an institution's rules (required elements, the form of values). Where OWL inference (Chapter 6) widens meaning, SHACL tightens shape; it is a quality control tool. Section 8.4 shows one.

XML, EAD and conversion scripts The tools of migration from what you already hold. Understanding the structure of EAD is the precondition of designing a conversion, and Python

(rdflib) or XSLT is how it gets implemented.

Triplestores Databases for RDF, providing a SPARQL endpoint and inference. Running one (backups, performance, access control) is the same kind of work as running any other information system.

8.3 What relational database experience is worth

A reader who has worked with relational databases and SQL can carry most of that experience across. Here is the correspondence, and where it breaks.

Table 8.2: Relational concepts and their RDF and RiC-O counterparts

Relational	RDF / RiC-O	Watch out for
Table definition	class	an individual can belong to several classes
Row	individual (resource)	
Column	datatype property	the same property may be used more than once
Foreign key, join	object property	it has a direction, and an inverse is defined
Primary key	IRI	unique worldwide; once published, unchanged
Schema (constraints)	ontology (axioms) plus SHACL (constraints)	an OWL axiom is an inference rule, not a constraint
NULL (no value)	absence of a triple	closed world against open world (Chapter 5)
SQL	SPARQL	the thinking is close. SPARQL with inference is better at recursive search
Normalisation (separating entities)	raising attributes to entities	very nearly the same idea; the instinct transfers

Experience of normalisation is the most valuable part. “Do not repeat the creator’s name as a string in every row; separate it into a creators table and refer to it by identifier” is the standard move of relational design, and it is the same motive that makes RiC raise attributes into entities (RiC-CM §3.3); the worked steps are in the box in Section 2.4. The design in which a many-to-many relation becomes a junction table, and a column such as “role” is added to that table, corresponds directly to reifying a relation in RiC. Anyone who has drawn an ER diagram will read the chapters of RiC-CM on entities, attributes and relations with a sense of recognition. What does trip up someone from a relational background is the open world assumption, and the fact that the schema does not reject data; those two are the parts to relearn.

8.3.1 What happens to dependency and independence

The family of concepts around dependency in database theory transfers to reading RiC as well.

Existence dependency and weak entities In the ER model, an entity that cannot exist without another, and whose identity (its key) depends on the parent, is a weak entity, and the relation joining them is an identifying relationship. The wording RiC-CM uses to explain its three kinds of record resource reads as a restatement of exactly that: the identity of a record derives from the record itself; the identity of a record part derives from the record it belongs to; the identity of a record set derives from its members (§2.2.2). A record is a strong entity, a record part a weak entity with a record as parent, and a record set is existence-dependent on its members (looser than the classical weak entity in having several parents). The instantiation is joined to the record by an existence condition too: a record does not exist without at least one instantiation. In RiC-O such

dependencies are sometimes written as OWL cardinality restrictions. `rico:Proxy` is axiomatised as standing for exactly one record resource (`rico:proxyFor`) and lying within exactly one record resource (`rico:proxyIn`), an existence-dependent entity that means nothing without what it stands for. The instinct that tells a designer which entities are weak works, unchanged, for telling which RiC entities stand alone and which lean on others.

Functional dependency and uniqueness Declarations answering to functional dependency in normalisation theory (fix the value of X and the value of Y is determined) are nearly absent from RiC-O. OWL has functional properties, limiting a property to at most one value, and RiC-O 1.1 does not use them. Names, dates and creators are all repeatable in principle, and nothing enforces "at most one". This is not an omission but a choice that follows from the nature of archival description (variant names, changes of name, several creators, uncertain dates recorded side by side) and from the open world assumption (the same thing may appear twice under two IRIs, and enforcing uniqueness of value in logic invites unintended inferences of identity). A working functional dependency such as "the identifier is unique within a record" is therefore upheld not in the ontology but in the layer of validation, with SHACL. The work of maintaining consistency that a relational schema did on its own is, in the RiC world, split into two layers, axioms (inference) and constraints (validation). Holding that correspondence in mind makes the division of labour much clearer.

Data independence ANSI/SPARC divided database design into three schemas: the external schema, which settles how each user sees the data; the conceptual schema, the logical design of the domain itself; and the internal schema, how it is actually stored. Only with that division can one speak of data independence, the property that a lower layer can change without the upper layers being rewritten. The idea has counterparts in the design of RiC. Separating input from output (Chapter 3) is the acquisition of independence from presentation; separating the record (the intellectual content) from the instantiation (the inscription on a carrier) insulates the description of content from changes of carrier and format. A format migration leaves the description of the record untouched; a new instantiation is simply added. The design principle taught in database textbooks, separate what changes often from what does not, has been stretched to the timescale of archives, which is centuries.

8.4 Tightening the shape: writing one SHACL constraint

The previous section said that a working constraint such as uniqueness is upheld in the layer of validation rather than in the ontology. What does that work is **SHACL** (Shapes Constraint Language, a W3C recommendation of 2017). The name has appeared repeatedly since Chapter 5, but the difference from OWL does not sink in until an actual shape is in front of you. Here is a short one.

The data below has two problems: `ex:rec2` has no name, and `ex:rec3` gives a *place* as its creator.

```
ex:rec2  a  rico:Record .                    # no name

ex:rec3  a  rico:Record ;
    rico:name  "Quarantine records" ;
    rico:hasCreator  ex:marchford .        # the creator is a place

ex:marchford  a  rico:Place ;
    rico:name  "Marchford" .
```

Start by asking what a RiC-O reasoner makes of it. The range of `rico:hasCreator` is declared to be `rico:Agent`. But an OWL range declaration is *a warrant for inference, not a check* (Table 8.2).

So the reasoner reports no error. It *derives a new triple* instead: that `ex:marchford` is also an agent. RiC-O declares person, group, position and mechanism to be pairwise disjoint, but does not declare place and agent disjoint, so no inconsistency is detected either. Wrong data quietly multiplies. The missing name is not a problem either: under the open world assumption it is not "there is no name" but "it has not been written yet".

SHACL runs the other way. Write the shape you want to check as a **shape**, and nodes that do not fit are reported as violations.

```
PREFIX sh:   <http://www.w3.org/ns/shacl#>
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>

ex:RecordShape  a  sh:NodeShape ;
    sh:targetClass  rico:Record ;          # check instances of Record
    sh:property [
        sh:path      rico:name ;
        sh:minCount  1 ;                   # a record must have a name
        sh:message   "A record must have a name" ] ;
    sh:property [
        sh:path      rico:hasCreator ;
        sh:class     rico:Agent ;          # a creator must be an agent
        sh:message   "Only an agent may be given as creator" ] .
```

Run that through `pySHACL` or the validation built into GraphDB and two violations are reported, `ex:rec2` for the missing name and `ex:rec3` for the creator being a place. The same data, the same RiC-O vocabulary: the reasoner says nothing and SHACL names two problems. That difference is the content of the formula repeated through this book, that OWL widens meaning and SHACL tightens shape.

Without the vocabulary loaded, correct data fails too

The `sh:class` check above has a trap in it. A correct description whose creator is a `rico:Person` will also be reported as a violation, as things stand. `sh:class` checks whether the value node belongs to the named class *within the graph being validated*. The fact that a person is a kind of agent is written on the RiC-O side, so a processor given the data alone does not know it. There are two ways round: load the RiC-O definitions (at least the class hierarchy) together with the data, or run inference first so that `a rico:Agent` has been derived. That SHACL counts in a closed world also means that *what you hand it is the whole of that world*. What Section 5.3 said, that the meaning of a vocabulary is written on the data side, turns up here as a practical trap.

How can SHACL say that something is absent? The key is `sh:targetClass`. That one line fixes what is being checked, and within that range SHACL *counts what is in front of it*. SHACL brings the closed world assumption in locally, for the temporary purpose of validation. This does not contradict the open world assumption of OWL; they are at different layers. The meaning of the data stays open, while "the shape of data my institution will accept" is judged as a closed world. When Section 6.9 said that a query about absence is written as a check on the data rather than as a logical negation, this division of labour is what it meant.

What SHACL cannot write in one line either

The basic vocabulary expresses conditions on one node and its neighbourhood. A condition that *compares several nodes*, such as the uniqueness of reference numbers within a fonds mentioned in Section 6.9, goes past that. SHACL provides an escape, `sh:sparql`, which embeds a SPARQL query inside a shape, and constraints of that kind are written there. The

pattern, that there is a range writable declaratively and a range where a query has to be written, is much like the boundary of OWL expressive power in Section 6.7. Seeing where a tool stops, and knowing the means of going past it, is needed in the same way for both.

The shape and the test data of this section are in the sample data published on this book's website, as `shapes.ttl` and `shacl-data.ttl`. That the two violations really are reported, and that without RiC-O even the correct record is reported, can both be confirmed at your own desk (Section 7.12).

An application profile (Section 7.3) is not observed merely by being decided. Only when what was decided is written out as shapes, and the validation is run at every conversion, does the policy reach the data. SHACL is the tool for that, and the closest description of it is that it is *the machine-readable part* of an institution's application profile.

8.5 A learning route

An unhurried order, for a reader beginning in archival science.

1. **The concepts** (Chapters 1 to 4 of this book, plus RiC-CM itself). No technology at all. Read the entities and their scope notes against a body of records you know.
2. **Getting used to graphs** (Chapter 5, with paper and pencil). Practise drawing a body of records you know as entities and relations. Still no technology, and yet the core of thinking in RiC is acquired here.
3. **Reading and writing Turtle and SPARQL** (Chapters 5 and 6, plus the official RiC-O samples). A text editor and free tools are enough. The exercise at the end of Chapter 7 is at this level.
4. **One turn through with real data** (Chapter 7). Convert, load, query and reason over a few dozen records of your own. Learning the beginnings of Python alongside (useful well outside this field) widens the options.
5. **Going deeper where needed.** Towards implementation, that means OWL, SHACL and running systems. Towards descriptive practice, what is needed from here is keeping the concepts and keeping enough vocabulary to talk to developers.

One last thing, said plainly. The most important competence RiC asks for is not programming but *modelling*: the ability to divide a piece of the world into entities and relations, and to explain why it was divided that way. And that is essentially the same competence as the traditional training of archival science, which consists of establishing provenance, designing an arrangement, and accumulating judgements about description. RiC does not so much impose a new kind of work on archivists as ask them to write out, in an explicit form that can be shared with a machine, the intellectual work they have been doing all along.

Exercises (hints in Appendix A.3)

8.1 Name three differences between a foreign key in a relational database and an object property in RDF, attending to referential integrity, direction, and the treatment of the schema.

8.2 Choose one role (archivist, IT staff, researcher) and pick from the map in this chapter the two technologies to learn next, with your reasons.

8.3 In the work of validating the shape of data, how do the roles of SHACL and OWL differ? Answer using the phrase "open world assumption".

8.4 Following the shape in Section 8.4, write the constraint that a record set must have at least one identifier (using `rico:RecordSet` and `rico:identifier`).

Appendix A

Appendices

A.1 The entities of RiC-CM 1.0

ID	Entity	Definition (summarised)
E01	Thing	Any idea, material thing, or event within the realm of human experience
E02	Record Resource	Information produced or acquired and retained by an agent in the course of life or work activity
E03	Record Set	One or more records gathered by an agent on the basis of shared attributes or relations
E04	Record	Discrete information content inscribed on a carrier in a persistent, recoverable form
E05	Record Part	A component with information content of its own, contributing to the record's intellectual completeness
E06	Instantiation	The inscription of information on a carrier; the physical or digital appearance of a record
E07	Agent	A thing that performs activities in the world
E08	Person	An individual human being
E09	Group	Two or more agents that act together as an agent
E10	Family	Two or more people joined by birth, marriage, adoption or other social bond
E11	Corporate Body	An organised group recognised in law or society
E12	Position	The functional role of a person within a group
E13	Mechanism	A process or system created by a person or group that performs an activity
E14	Event	Something that happens in time and space
E15	Activity	An event that an agent designs and carries out with a purpose
E16	Rule	Conditions governing the existence, responsibility or authority of an agent or the performance of an activity, or contributing to the distinct characteristics of what an agent creates or manages
E17	Mandate	The delegation of responsibility or authority from one agent to another
E18	Date	Chronological information associated with an entity that contributes to its identification and contextualisation
E22	Place	Bounded, named geographic area or region

E19 to E21 are vacant: the subentities of Date in version 0.2 were dropped in 1.0.
The full definitions, with scope notes and examples, are in RiC-CM §2.2. The 42 attributes are

listed in §3.6 and the 85 relations, with domain, range, cardinality and the broader and narrower relations of each, in §5.4; keep both open while designing a description; neither is reproduced here.

A.2 Frequently used RiC-O vocabulary

Term	Use
`rico:RecordSet` / `rico:Record` / `rico:RecordPart`	the three kinds of record resource
`rico:Instantiation`	the instantiation
`rico:Person` / `rico:Group` / `rico:CorporateBody` / `rico:Family` / `rico:Position` / `rico:Mechanism`	agents
`rico:Activity` / `rico:Event` / `rico:Rule` / `rico:Mandate` / `rico:Date` / `rico:Place`	the entities of context
`rico:Proxy`	a record resource's stand-in within one particular set (carries order and context)
`rico:name` / `rico:title` / `rico:identifier`	the basic literal attributes (the shortcut, without entities)
`rico:generalDescription` / `rico:scopeAndContent` / `rico:history`	the prose attributes
`rico:includesOrIncluded` / `rico:isOrWasIncludedIn`	membership of a record set
`rico:directlyIncludes` / `rico:includesTransitive`	direct inclusion; transitive closure (for inference)
`rico:hasCreator` / `rico:hasOrganicProvenance` / `rico:hasAccumulator`	provenance and creation
`rico:performsOrPerformed` / `rico:documents`	performing an activity; documenting one
`rico:hasSuccessor` / `rico:isSuccessorOf`	change of agent over time
`rico:hasOrHadInstantiation` (analogue and digital variants)	record resource to instantiation
`rico:thingIsSourceOfRelation` / `rico:relationHasTarget`	the ends of a reified relation
`rico:isAssociatedWithDate` / `rico:normalizedDateValue`	relating a date; the normalised value

A.3 Hints for the exercises

Chapter 1 1.1: the figure of the four parts in Section 1.2. 1.2: the CM is the concept (stable), the O an implementation (which gets corrected); Section 1.2 and Section 5.5. 1.3: the table of sources in Section 1.3, and the note on readers in the preface.

Chapter 2 2.1: Section 2.4.2. Having an identifier is being able to be the target of a reference. 2.2: the EAD example in Section 7.4 can be reused (`authfilenumber` and `relationEntry`). 2.3: the fourth period in Section 2.4.4, and Section 7.4. 2.4: the paragraph on libraries in Section 2.6, and Section 7.6.

Chapter 3 3.1: Table 3.1, freedoms against burdens. 3.2: point (6) of Section 3.6. One creator relation written as a direct property and as a relation entity, for instance. 3.3: the box in Section 3.1.

Chapter 4 4.1: the box in Section 4.2 (the anthology). 4.2: the right-hand side of Figure 2.2, or Figure 7.3, for the shape. 4.3: Section 4.5. Verifiability and revision history. 4.4: the last paragraph of Section 4.5.

Chapter 5 5.1: the syntax box and Section 5.4. `rico:isOrWasIncludedIn` is "is or was included in". 5.2: the three equivalent forms in the syntax box. 5.3: the definition of R^{-} in Section 5.2.1. 5.4: the box on classes as subjects in Section 5.2.

Chapter 6 6.1: Section 6.1 and the composition axiom in Section 5.2.1. 6.2: Section 6.8.3. 6.3: Section 6.8.2; attend to recomputation on deletion and update. 6.4: the figure and listing of situation 5. Provenance relation, then corporate body, then successor.

Chapter 7 7.1: step 2 in Section 7.3. 7.2: the Turtle in Section 7.4.2 is the model. 7.3: the end of Section 7.4.4; the convention that the many side points. 7.4: normalising a financial year in Section 7.7; the span is `1933-04/1934-03`. 7.5: the four tools in Section 7.8.

Chapter 8 8.1: the table in Section 8.3 and Figure 7.2. 8.2: the table of roles in Section 8.1. 8.3: Section 8.4 and Section 6.9. An OWL axiom is a warrant for inference, not a constraint. 8.4: copy the `sh:minCount` part of the shape in Section 8.4, changing `sh:targetClass` to `rico:RecordSet` and `sh:path` to `rico:identifier`.

A.4 Glossary: words that shift meaning between fields

Most of the terms below are ordinary in one field and mean something else in a neighbouring one. Those are the ones worth having in a glossary; the rest are here because this book uses them in a settled sense.

Archival terms

provenance The *context of creation*: whose records these are, and out of what undertaking they came. In the wording of RiC-FAD §5, "created, accumulated, and used by a person or group in the course of life and work". The word also has a wider sense covering origin, custody and ownership together, and in the art world it means the history of ownership, which is closer to what archives call custodial history. Establish which sense is in play (Section 3.3). RiC widens provenance beyond the single accumulator, to the several parties involved in creating, accumulating, sending and receiving, and to the activities that produced the records.

archival history, custodial history What became of the records *after* they left the creator: changes of ownership, responsibility and custody. ISAD(G) 3.2.3 calls it archival history and takes the wider view, including the history of arrangement, the production of current finding aids and software migration; DACS 5.1 calls it custodial history and stops at the door of the repository. In RiC it goes into the History attribute (RiC-A21) as prose, and into the management relations (R038 is or was manager of, R039 is or was holder of) when structured. Those are a different type of relation from provenance (the box in Section 3.3).

principle of provenance The principle that puts provenance at the base of description and management, in two parts. **Respect des fonds**: records created and accumulated by one body are kept together and not intermixed with those of another. **Respect for original order**: the order established among the records in the course of accumulation and use is preserved. Where a library classification rearranges material by subject, archives leave the grouping by provenance intact (Chapter 2).

fonds The whole of the records created, accumulated and used by one body in the course of life or work. From French, and the same in the singular and the plural. Traditional description was built as a hierarchy with the fonds at its head; in RiC it becomes one type of record set (Section 3.4).

unit of description In ISAD(G), the grouping that one description is about. The abstraction by which a fonds and a single letter can be described with the same 26 elements. RiC breaks it into three entities: record, record set, record part.

instantiation The appearance of a record, as intellectual content, inscribed on a carrier (paper, film, a digital file). One record may have several. A new entity introduced by RiC (Section 3.5).

GLAM Galleries, libraries, archives and museums, taken together as the sectors that collect, preserve and describe cultural material (Section 2.6).

Words that mean different things in different layers

administrative metadata Metadata about a description itself: who made it, when, under which rules, and when it was revised. In RiC it is descriptive metadata about a record one storey up, the description record, expressed with the same entities and relations (Section 4.5).

reference Naming something else from within a description. It has been strengthened by stages, from a name to a record identifier to an IRI (Chapter 2). How to read the direction of an arrow is Section 5.1.2.

entity In the ERM, a kind of thing of interest (an entity type), or an instance of one. The 19 entities of RiC-CM are types (Section 4.1).

scope note One of the fields RiC-CM uses to describe an entity, attribute or relation, stating the range of application and rules of thumb that a definition alone cannot convey. (An entity has six such fields: Identifier, Name, Definition, Scope Notes, Examples and Comments; for an attribute the field is called Scope.) When this book says "the scope note says", that field is meant. The same phrase is used in thesauri and subject heading lists, so context matters. RiC-O carries content that is the same as, or derived from, the RiC-CM scope notes in `skos:scopeNote`.

class The RDF and OWL counterpart of an entity type. The classes of RiC-O are the entities of RiC-CM plus concepts added for implementation.

individual, resource An instance in RDF and OWL, identified by an IRI.

attribute, property An attribute in the ERM is a characteristic of an entity. RDF divides properties into datatype properties, joining a thing to a value, and object properties, joining a thing to a thing; the latter answer to what the ERM calls relations. The same words name different things in different layers.

relation A connection between entities in the ERM. In an RDF implementation it appears as an object property or, reified, as a relation class.

"is a kind of" and "belongs to" The entity hierarchy asserts only "is a kind of" (a record is a kind of record resource), which is classification. A record *belonging to* a record set is an ordinary relation (R024 includes or included, from the set's side; R024i is or was included in, from the record's). There is no "is a kind of" relation between record and record set (the box in Chapter 4).

domain, range The entity, or class, that can stand at the start and at the end of a relation or property.

triple The unit of RDF data: subject, predicate, object.

a (the Turtle predicate) The abbreviation of `rdf:type`, meaning "is an instance of the class" (the syntax box in Chapter 5).

IRI Internationalized Resource Identifier. A globally unique name, borrowing the uniqueness that domain registration guarantees. That a page opens at it is not a condition (Chapter 5).

namespace The shared front part of an IRI, saying which institution's vocabulary the name belongs to. In Turtle it is abbreviated with a short prefix (`rico:`) declared by `PREFIX`.

literal A raw value: a string, a number, a date.

ontology A vocabulary defining the classes and properties of a domain in machine-readable form. Not metaphysics, though the word is the same.

axiom One of the declarations written in an ontology: not a fact about particular data but a commitment holding wherever the vocabulary is used, such as "`rico:Person` is a subclass of `rico:Agent`" (Section 5.2.1).

inference Deriving, mechanically, triples that were not written, from the data and the axioms.

reification Raising a relation, or the value of an attribute, into a node of its own. Used when the relation needs attributes.

graph A structure of points (nodes) and lines joining them (arcs). Not a bar chart. The same shape as a transit map or a family tree; a tree is the special case (Section 2.3).

transitive closure The whole relation obtained by following a direct relation as far as it goes. The transitive closure of "directly includes" is "includes" (Section 6.2).

property chain An axiom declaring in the vocabulary that if A leads to B and B to C, a shortcut property holds from A to C. A description written as the long path yields the shortcut by inference (Section 6.5).

roll-up A statement written with a narrow subproperty being drawn up by inference into the broader property, so that describing in detail does not lose you a coarse search (Section 6.4).

open world assumption The position that what is not written is unknown rather than false. OWL takes it.

closed world assumption The position that what is not written is false. Relational databases and the knowledge representation of the expert-system era take it, and rules with exceptions can only be written under it. In a domain where the data is incomplete it produces false negations, which is why OWL took the other side (the box in Section 3.1).

Semantic Web The programme proposed in 2001 for writing information on the web so that machines as well as people can make sense of it, and having them follow links and reason. RDF, OWL and SPARQL all descend from it (the box in Section 3.1).

linked open data Published RDF data named by IRIs and linked to one another; also the practice of publishing that way.

owl:sameAs The OWL property declaring that two IRIs name the same thing. Used to tie your own Agent to an external authority such as VIAF (Section 7.6).

application profile A documented decision about which of the RiC-O vocabulary an institution uses and which parts are required. RiC says "either will do" often enough that institutional policy has to be gathered somewhere, and this is where (Section 7.3).

SPARQL The query language for RDF graphs.

SHACL The constraint language for validating the shape of RDF data.

SKOS The W3C standard for expressing a controlled vocabulary (thesaurus, subject heading list, classification) in RDF. Concepts are raised as individuals with `prefLabel` and `altLabel`, joined by `broader` and `narrower`. `broader` is not `subClassOf` (the box in Section 7.5).

named graph Storing and querying triples with a label saying which graph they belong to (the RDF dataset of SPARQL, from SPARQL 1.0 in 2008). It allows authority data and catalogue data to be managed separately and joined logically (Section 7.4.4).

triplestore A database that stores RDF and provides SPARQL querying, often with inference.

A.5 Primary sources and further resources

- RiC-FAD 1.0 (2023) — `github.com/ICA-EGAD/RiC-FAD`
- RiC-CM 1.0 (2023) — `github.com/ICA-EGAD/RiC-CM` (also from the ICA website)

- RiC-O 1.1 (2025) — `github.com/ICA-EGAD/RiC-O`
- RiC-AG 0.1 draft (released 29 October 2025 at the ICA congress in Barcelona) — `github.com/ICA-EGAD/RiC-AG` (the text; the official crosswalks from the four existing standards to RiC-CM and from EAD 2002 to RiC-O 1.1; the diagrams)
- RiC-O documentation site — `ica-egad.github.io/RiC-O/` (About, Why use RiC-O?, Diagrams, Examples)
- RiC Resource List — `ica-egad.github.io/RiC-ResourceList` (implementations, tools, publications and datasets, compiled by the user community and maintained under EGAD)
- RiC-O Converter — the EAD and EAC-CPF conversion tool from the Archives nationales de France and Sparna (first released 2020) — `github.com/ArchivesNationalesFR/rico-converter`
- AnF Sparnatural demonstrator (visual querying of the notarial knowledge graph) — `sparna-git.github.io/sparnatural-demonstrateur-an`
- Records in Contexts users — the user community (Google Group)
- The earlier standards: ISAD(G) 2nd ed. (2000), ISAAR(CPF) 2nd ed. (2004), ISDF (2007), ISDIAH (2008), all published on the ICA website
- Related standards: ISO 23081 (records management metadata), PREMIS (preservation metadata), EAD and EAC-CPF (encoding), and the W3C specifications for RDF, OWL, SPARQL, SKOS and SHACL

Index

Records in Contexts: An Introduction

Understanding the new international standard for archival description

Author	Tetsuji Kuboyama (Graduate Program in Archival Science and Computer Centre, Gakushuin University)

Published	2026 (English edition 1.0)
Website	`https://tkub.github.io/edu/ric/en/`
Contact	`tkuboyama@tk.cc.gakushuin.ac.jp`

This is the English edition of *Records in Contexts: An Introduction*, first published in Japanese in 2026. Large language models (Claude Fable 5 and Claude Opus 5, Anthropic) were used to assist with the translation and with drawing the diagrams. Responsibility for the structure and content of the book rests with the author. Typeset with pdfLaTeX.